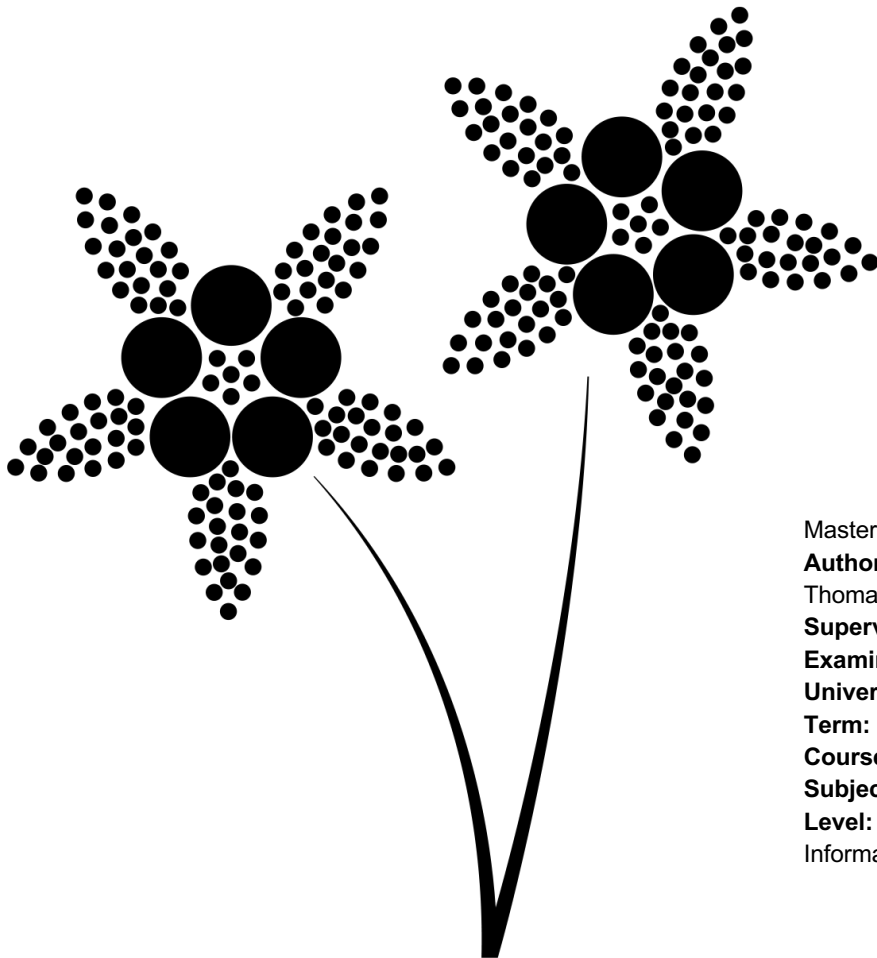


Evaluating Generative AI Coding Assistants in Enterprise Software Development

A SPACE and Task–Technology Fit (TTF) Analysis



Master's Thesis

Authors: Louise Bergman & Reena Thomas

Supervisor: Jan Aidemark

Examiner:

University: Linnaeus University

Term: Spring 2026

Course code: 5IK50E

Subject: Informatics

Level: Graduate Department of Informatics

Abstract

This master's thesis examines developers perceived productivity when using Generative AI coding assistants in enterprise software development, with a focus on Amazon Q Developer and AWS Kiro in a large telecom organization. Guided by a Human-Centered AI (HCAI) perspective, the study evaluates how software developers perceive these tools across the five dimensions of the SPACE framework: Satisfaction and Well-being, Performance, Activity, Communication and Collaboration, and Efficiency and Flow. It also investigates whether perceived Task–Technology Fit (TTF) helps explain variation in these productivity outcomes.

The study uses a mixed-methods survey design based on responses from 160 software developers, combining Likert-scale items with open-ended questions. The quantitative analysis includes descriptive and regression-based analysis, while the qualitative analysis applies thematic analysis to developers' written responses. The findings indicate generally positive perceptions of coding assistants, with the strongest reported effects on Satisfaction and Efficiency and Flow, followed by Activity. Performance is more moderate, while Communication and Collaboration is the weakest and most variable dimension. Perceived TTF is positively associated with all examined productivity dimensions, whereas utilization alone is not a statistically significant predictor.

Comparisons between Amazon Q Developer and AWS Kiro suggest broadly similar perceived productivity outcomes, although AWS Kiro shows somewhat broader reported use across task types. The qualitative findings indicate that coding assistants are especially valued for well-defined tasks such as code comprehension, troubleshooting, refactoring, documentation, and testing support. At the same time, recurring constraints include limited access to relevant context, weak workflow integration, correctness and reliability issues, operational instability, and increased verification effort.

Overall, the study contributes a multidimensional, enterprise-grounded understanding of AI coding assistants by combining the SPACE framework with TTF. The findings suggest that organizations may realize more sustainable value by focusing not only on adoption and usage, but also on improving task–tool fit, workflow integration, governance, and developer support.

Keywords

Coding Assistants
SPACE Framework
Task–Technology Fit (TTF)
Productivity
Generative AI
Software Development
Human-Centered AI (HCAI)

Acknowledgments

We would like to express our sincere gratitude to our supervisor, Jan Aidemark, for his valuable guidance, constructive feedback, and continuous support throughout this thesis. His insights helped us refine both the research design and the presentation of our findings.

We also thank our host organization for providing the opportunity and environment to conduct this study, and for supporting access to relevant tools and participants. We are especially grateful to our supervisors from the organization for their excellent academic and technical guidance. We want to also thank survey respondents who took the time to share their experiences and perspectives; their contributions were essential to this work.

Finally, we would like to thank our families and close friends for their patience and encouragement during this period. We also thank each other for a supportive collaboration and for sharing the work throughout the thesis process.

Table of Contents

1	Introduction.....	3
1.1	Background and problem.....	4
1.2	Topic justification.....	5
1.3	Purpose Statement and Research Questions.....	5
1.4	Thesis Organization.....	6
2	Literature Review	7
2.1	AI Coding Assistants in Software Development.....	7
2.2	The SPACE Framework for developer productivity	14
2.3	Task–Technology Fit (TTF) theory.....	17
2.4	Synthesis: Integrating SPACE and TTF.....	21
3	Theoretical Framework.....	22
3.1	Human-Centered AI - Overarching perspective.....	23
3.2	SPACE Framework	23
3.3	Task–Technology Fit.....	24
3.4	Theoretical Framework and Analysis.....	24
3.5	Conceptual Model and expected relationships	25
4	Methodology	26
4.1	Philosophical Tradition.....	26
4.2	Methodological Approach	27
4.3	Data Collection.....	27
4.4	Definition of Task.....	30
4.5	Participants and Distribution	31
4.6	Data Analysis.....	31
4.7	Integration of Findings	34
4.8	Reliability & Validity	35
4.9	Ethical Considerations.....	36
5	Empirical Findings	37
5.1	SPACE- Quantitative Findings.....	37
5.2	SPACE- Thematic Analysis of qualitative data	44
5.3	Mixed-methods integration.....	50
6	Discussion	53
6.1	Overview and answers to research questions	53
6.2	Interpreting perceived productivity through the SPACE dimensions	55

6.3	Task–Technology Fit as an explanatory mechanism.....	57
6.4	Organizational factors for effective use and realized value	58
6.5	Interpreting similarities and differences between coding assistants.....	60
6.6	Implications for theory and practice	61
7	Conclusion	62
7.1	Conclusions	62
7.2	Contributions	62
7.3	Limitations.....	64
7.4	Future Research	64
	References	67
	Appendix A: Report on the use of Generative AI in Master Thesis.....	1
	Appendix B: Description of the work division between the authors.....	3
	Appendix C: Survey Questions	4
	Appendix D: Screenshots.....	7
	Appendix E: Coding Assistants Usage Patterns.....	10
	Appendix F: Coding Assistants Comparisons	12
	Appendix G: Multiple linear regression diagnostic tables	15
	Appendix G: Multiple linear regression robustness check tables	25
	Appendix H: Literature Review Search Summary.....	30

1 Introduction

Recent advancements in digital technology are reshaping both everyday life and organizational practices, with software engineering increasingly influenced by Artificial Intelligence (AI), particularly Generative AI (GenAI) and Large Language Models (LLMs) (Banh et al., 2025, p. 1). In enterprise software development, this shift is especially visible in the growing use of LLM-based coding assistants that support tasks such as code generation, explanation, refactoring, debugging, and documentation. These tools are often presented as a new way to improve developer productivity by accelerating routine work and supporting broader parts of the software development lifecycle (Fischer & Lanquillon, 2024, p. 5).

At the same time, software development is not limited to writing code. Prior research shows that developers spend much of their working time on activities such as program comprehension, information seeking, coordination, documentation, and problem solving, while direct coding occupies only a relatively small share of the workday (Meyer et al., 2021, pp. 863-864; Minelli et al., 2015, pp. 30–32). This means that evaluating the value of coding assistants requires a broader perspective than simple output-based indicators such as lines of code or number of commits. If these tools are to contribute meaningfully in enterprise settings, they must support the wider task portfolio that characterizes real software engineering work.

This challenge is particularly important in organizations that are investing in coding assistants with the expectation of improving developer productivity. Although the potential benefits of GenAI are widely discussed, it remains unclear how developers perceive the actual contribution of these tools in everyday enterprise work, where task complexity, codebase maturity, organizational constraints, and collaboration pattern all shape outcomes. In such environments, the value of coding assistants is likely to depend not only on their technical capabilities, but also on how well they fit developers' tasks, how easily they can access relevant context, how reliably they perform, and how well they integrate into existing workflows.

Against this background, this thesis examines developers perceived productivity when using coding assistants in enterprise software development, with a particular focus on Amazon Q Developer and AWS Kiro in a large telecom organization. The study investigates how these tools are perceived across different dimensions of developer productivity and how their use is shaped by task fit and organizational conditions. In doing so, it treats coding assistants not simply as automation tools, but as socio-technical tools whose value depends on the interaction between human work, task demands, and enterprise context.

To study this, the thesis applies a mixed-methods survey design based on responses from 160 software engineers. The analysis combines quantitative and qualitative evidence to examine how coding assistants are experienced in practice. The quantitative part draws on the SPACE framework as a multidimensional lens for understanding developer productivity and on Task–Technology Fit (TTF) as an explanatory perspective for why productivity outcomes may differ across users and contexts. The qualitative part adds insight into the practical conditions that shape use, including strengths, limitations, and recurring challenges reported by developers.

By combining these perspectives, the thesis aims to contribute to a more nuanced and enterprise-grounded understanding of AI coding assistants. Rather than assuming that increased use automatically leads to greater value, the study examines how perceived productivity depends on the extent to which these tools support real development tasks, reduce friction in daily work, and fit within the broader organizational environment in which software engineering takes place.

1.1 Background and problem

Assessing the value of GenAI coding assistants in software development is inherently complex. Traditional productivity proxies, such as lines of code (LoC), commit counts, or pull requests, often fail to capture meaningful gains because AI-generated output may be verbose, fragmented, or rapidly produced without necessarily improving outcomes. Examples of these metrics are illustrated in Appendix E. Additionally, many evaluations of coding assistants are based on synthetic or isolated tasks that do not reflect the contextual richness and interdependencies of real-world software engineering. In enterprise environments, developers work within large and evolving codebases, under architectural constraints, quality standards, and established workflows that require deep situational understanding. Current AI systems often lack the tacit and undocumented knowledge that experienced developers rely on in such settings (Becker et al., 2025, pp. 2, 19).

This challenge becomes even more significant when considering the broader nature of software development work. Developers spend only a limited share of their time writing code, while substantial effort is devoted to program comprehension, collaboration, meetings, information seeking, documentation, and problem solving (Meyer et al., 2021, p. 864). As a result, the value of coding assistants cannot be understood solely in terms of faster code generation. Any meaningful assessment must instead account for how these tools influence a wider portfolio of development tasks and whether they support productive work across multiple dimensions.

In enterprise software development, this creates a particular evaluation problem. Organizations increasingly invest in coding assistants with the expectation of improving developer productivity, yet the actual contribution of these tools depends on more than technical capability alone. Their value is also shaped by how well they fit the tasks developers perform, how much relevant context they can access, how reliably they perform, and how well they integrate into existing workflows. In practice, even tools that appear useful for local or well-defined tasks may provide limited value when they struggle with cross-repository context, domain-specific conventions, or organizational processes such as code review and issue tracking.

In addition, the long-term implications of GenAI-assisted development remain uncertain. Although coding assistants may accelerate routine tasks, they can also introduce hidden costs. AI-generated code may increase verification effort, affect maintainability, introduce security risks, or create technical debt that becomes visible only later in testing or after release. These issues raise important questions about whether apparent productivity gains are sustainable over time, or whether they partly reflect a redistribution of effort from code production to review, validation, and correction. Concerns about cognitive burden, over-reliance, and long-term learning further complicate the picture, particularly when developers must continuously verify outputs that appear plausible but are not consistently correct.

Taken together, these considerations indicate that the value of coding assistants cannot be inferred from tool statistics or output-based metrics alone. Robust evaluation requires a broader and more integrated perspective that accounts for developer-perceived productivity across multiple dimensions, while also considering task fit, context access, workflow integration, reliability, and longer-term sustainability. In enterprise software development, where work is shaped by complex codebases, organizational constraints, and the continued need for human oversight, coding assistants must therefore be assessed not simply by how much code they generate, but by how well they support real development work in practice.

1.2 Topic justification

The growing use of GenAI coding assistants in software development makes the topic of this thesis both timely and relevant. Although these tools are increasingly promoted to improve software engineering work, the empirical evidence remains limited, fragmented, and still heavily influenced by the rapid pace of technological change. Much of the available research has focused on emerging tools, short-term experiments, or controlled benchmark tasks, while fewer studies have examined how coding assistants are perceived and used in real enterprise software development environments.

This creates an important research need. Existing studies often evaluate coding assistants using narrow indicators such as coding speed, lines of code, or benchmark completion, even though such measures do not adequately reflect the broader nature of software engineering work. Prior research has shown that software development involves not only implementation, but also comprehension, debugging, coordination, documentation, review, and problem solving. As a result, assessments based only on output or speed risk overlooking important dimensions of developer productivity, including work quality, flow, collaboration, and well-being.

The topic is also justified by persistent uncertainty in the literature regarding the net value of coding assistants. Previous research points to possible benefits in routine or well-defined tasks, but also highlights risks related to correctness, maintainability, security, and increased verification effort. Studies have further raised concerns that apparent gains may sometimes reflect spurious productivity, where faster code generation is offset by downstream debugging, review, and rework. In addition, much prior research has relied on synthetic tasks or simplified settings that do not capture the complexity of enterprise development, where large codebases, domain-specific knowledge, governance requirements, and established workflows strongly shape how useful such tools can be in practice.

A further justification for this study is that the existing literature provides limited explanation for why coding assistants appear beneficial in some contexts but less useful in others. Research has increasingly recognized the need for broader and more human-centered approaches to evaluation, but there is still a lack of studies that combine multidimensional productivity assessment with an explanatory perspective on task-tool alignment. This is particularly relevant in enterprise settings, where the usefulness of coding assistants is likely to depend not only on technical capability, but also on how well they fit developers' actual tasks, workflow conditions, and organizational environment.

Against this background, this thesis addresses an important gap by examining developer-perceived productivity in a real enterprise context using the SPACE framework and TTF as complementary analytical lenses. By focusing on Amazon Q Developer and AWS Kiro in a large telecom organization, the study contributes to a more nuanced understanding of how coding assistants are perceived across multiple productivity dimensions and why those perceptions vary. The topic is therefore justified not only by the growing organizational relevance of coding assistants, but also by the need for more enterprise-grounded, multidimensional, and explanatory research on their role in software development.

1.3 Purpose Statement and Research Questions

The purpose of this thesis is to examine developer-perceived productivity when using AI coding assistants in enterprise software development, with a focus on a large telecom organization. Specifically, the study investigates how tools such as Amazon Q Developer and AWS Kiro are perceived across the five dimensions of the SPACE framework and assesses whether perceived TTF helps explain variation in these productivity outcomes.

In addition, the study examines the organizational conditions that shape how developers use these tools and derive value from them in their day-to-day work. By combining the SPACE framework with TTF, the thesis aims to provide a multidimensional and enterprise-grounded understanding of how AI coding assistants support software development work in practice.

Ultimately, the findings are intended to provide evidence-based insights that can inform organizational decision-making regarding the selection, deployment, and support of AI coding assistants in large-scale software engineering environments. In this context, organizational factors may include access and onboarding, workflow integration, governance, security constraints, training, and team norms related to code review and tool use.

The research questions guiding this study are:

RQ1. How do software developers perceive the impact of AI coding assistants across the five SPACE dimensions of productivity?

RQ2. To what extent does Task–Technology Fit (TTF) predict developer-perceived productivity across the SPACE dimensions?

RQ3. What organizational factors influence how software developers use AI coding assistants and realize value from them in their daily work?

1.4 Thesis Organization

This thesis is structured into seven chapters.

Chapter 1 introduces the study by presenting the background and problem, justifying the topic, stating the purpose and research questions, and outlining the structure of the thesis.

Chapter 2 reviews the relevant literature on Generative AI coding assistants, developer productivity, the SPACE framework, and TTF and identifies the research gap addressed by the study.

Chapter 3 presents the theoretical framework. It introduces Human-Centered AI (HCAI) as an overarching perspective and explains how the SPACE framework and TTF are used together as analytical lenses for examining developer-perceived productivity.

Chapter 4 describes the methodology, including the mixed-methods survey design, data collection, operationalization of constructs, participants, data analysis procedures, and considerations of reliability, validity, and ethics.

Chapter 5 presents empirical findings. It reports usage patterns, quantitative results across the SPACE dimensions, qualitative themes from open-ended responses, regression analysis, and the integration of quantitative and qualitative findings.

Chapter 6 discusses the findings in relation to the research questions, the theoretical framework, and prior research. It interprets perceived productivity across the SPACE dimensions, examines Task–Technology Fit as an explanatory mechanism, discusses organizational conditions shaping effective use, and outlines implications for theory and practice.

Chapter 7 concludes the thesis by summarizing the main findings, presenting the study's contributions, discussing limitations, and suggesting directions for future research.

References and appendices are included at the end of the thesis to provide full source of information and supporting material.

2 Literature Review

The following sections present three separate literature reviews: one on AI coding assistants in software development, one on the SPACE framework, and one on the TTF framework. A description of the search strategy used to identify and select the sources included in the reviews, together with the inclusion and exclusion criteria, PRISMA chart, keywords, date of search, and limitations, can be found in Appendix H.

2.1 AI Coding Assistants in Software Development

The integration of Large Language Model (LLM)-powered coding assistants has initiated a fundamental shift in software engineering, evolving from simple next-token autocompletion to complex agents capable of generating entire functions and managing project-level tasks (Lau & Guo, 2025; Yu, 2025). These tools aim to automate repetitive implementation tasks so that developers can focus on higher-level architectural logic (Mendes et al., 2024). The focus is on Github Copilot, Amazon Q Developer, AWS Kiro and Claude Code.

2.1.1 Core Functionality and IDE Integration

Github Copilot is an AI-powered programming assistant released in 2021 by GitHub in collaboration with OpenAI, primarily powered by the Codex model, a version of GPT-3 fine-tuned on an extensive collection of source code from public GitHub repositories (Dang et al., 2026, p. 2). It integrates directly into popular IDEs such as Visual Studio Code and JetBrains, where it functions as an "AI pair programmer," shifting the paradigm of code completion from simple next-token prediction to translating natural language prompts or function signatures into entire implementations (Mendes et al., 2024, pp. 2-3). Research using the SPACE framework identifies two primary interaction modes: acceleration mode, in which the programmer uses the tool for rapid execution of a known plan, and exploration mode, in which the developer uses it to brainstorm solutions for unfamiliar problems. Beyond standard autocomplete, the ecosystem extends to Github Copilot Chat for natural language debugging and code explanation, Github Copilot Workspace for generating implementation plans from issues, and Github Copilot Edits for applying suggestions across an entire repository (Hassan et al., 2026, p.1; Lau & Guo, 2025, pp. 10-11; Weber et al., 2024, pp. 3-5).

Amazon Q Developer was formerly known as Amazon CodeWhisperer, is a GenAI-powered assistant for software development released by Amazon in 2022, built on transformer-based LLMs trained on extensive source code repositories and documentation. It provides two primary interaction mechanisms which are, inline code completion and conversational chat (Mendes et al., 2024, pp. 2-3; Stalnaker et al., 2026, pp. 3, 10, 13). The inline completion engine offers real-time, context-aware suggestions ranging from single tokens to full functions, with specific optimization for Python, Java, and JavaScript (Pirzado et al., 2024, p. 5), while the conversational assistant supports natural language interactions for coding, refactoring, and debugging (Dang et al., 2026, p. 4; Weisz et al.,

2025, pp. 2, 7). The tool integrates seamlessly into cloud-native environments, enabling developers to remain focused without switching to external sources such as Stack Overflow (Yu, 2025, p. 9).

AWS Kiro is a next-generation AI-powered IDE developed by Amazon/AWS, launched in 2025 and built on Code OSS, the open-source core of Visual Studio Code. The fundamental distinction between AWS Kiro and Amazon Q Developer lies in their architectural philosophies and their placement within the software engineering evolutionary timeline. While Amazon Q Developer is characterized as a reactive assistant following the Software Engineering 2.0 (SE 2.0) paradigm, AWS Kiro represents a proactive, Software Engineering 3.0 (SE 3.0) approach defined by intent-centric, specification-driven development (Hassan et al., 2026, pp. 1-2; Marcus, 2026, pp. 1, 5).

It introduces a paradigm shift from reactive coding to proactive, specification-driven development through its mandatory Spec-First workflow (Swaminathan & Singh, 2025, p. 29). This workflow ensures that every line of code is backed by a clear plan, effectively replacing disorganized vibe coding with a rigorous system of checks and balances (Marcus, 2026, p. 2). The three core documents function as follows:

The requirements file defines what the software needs to do by listing user stories and acceptance criteria. It uses the EARS (Easy Approach to Requirements Syntax) format to provide a highly structured and unambiguous language for these goals. By starting here, the developer ensures that the AI understands the business intent before any technical work begins.

Once the requirements are set, AWS Kiro translates them into a technical blueprint or how the system will be built. The design file includes critical architectural details such as TypeScript interfaces, API contracts, and structural diagrams that typically require manual drafting by a senior architect.

This tasks file provides the action plan, breaking the design down into a granular to-do list. Each task is an atomic unit, meaning it is small enough to be executed and tested independently by an AI agent. Because these tasks are testable, AWS Kiro's agents can work through them autonomously while verifying that each step matches the original requirements (Sabahi, 2025, pp. 2, 12).

This structured approach acts as a human gate, requiring the developer to review and approve the specs at each stage, which significantly reduces logic errors and prevents code drift, a common issue where the final product fails to match the original user needs (Marcus, 2026, pp. 8, 9). By formalising these steps into a mandatory sequence, AWS Kiro shifts the developer's role from writing every line of code to orchestrating and certifying the AI's technical decisions (Hassan et al., 2026, p. 9).

Claude Code is a specialized AI coding assistant developed by Anthropic and categorised within the agentic era of development tools. Distinguishing itself from IDE extensions like Github Copilot, Claude Code is engineered as a Command-Line Interface (CLI) tool, allowing developers to interact with the AI directly through the terminal.

The tool is classified as an autonomous agent, meaning a single natural language prompt can trigger a sequence of independent actions. These include traversing the codebase, reading and editing multiple files simultaneously, and executing terminal commands on the user's machine.

Claude Code is noted for its superior complex reasoning capabilities and effectiveness in multi-step planning. This makes it particularly suitable for high-level architectural tasks and autonomous workflows.

A core feature is support for Project Rules. These are persistent instructions written in Markdown files that define coding standards, domain-specific knowledge, and required

library versions, ensuring the AI's output remains consistent with existing project conventions.

As it utilizes multimodal foundation models, Claude Code can process visual inputs, such as screenshots or videos of a UI, to generate the corresponding code or interpret application behavior.

Unlike the Spec-First mandate of AWS Kiro, Claude Code follows a more flexible mixed-initiative approach; while it is capable of complex planning when requested, it does not strictly enforce a specification-driven workflow.

Despite its advanced capabilities, research identifies specific trade-offs. Its reliance on a terminal-first interface may be less accessible to developers who prefer a polished graphical UI/UX. Additionally, while it can manage entire repositories, its context persistence is largely session-based, meaning its immediate memory is often limited to the active interaction thread. Consequently, it is regarded as a powerful tool for professional software engineers managing complex, existing codebases. It represents the next generation of AI IDEs, pushing the boundaries of autonomous agency and intent-driven development (Lau & Guo, 2025, pp. 19-36; Hassan et al., 2025, p. 1). Going forward, specialized agents like Claude Code will handle complex, multi-file execution while humans provide creative vision and ethical oversight. It represents a future where the developer functions as a high-level architect overseeing a fleet of AI agents (Marcus, 2026, pp. 13-18; Pezzè et al., 2025, pp. 41, 42)

Standalone AI IDEs is a new class of programming tools where the AI is built directly into the software's foundation, usually by modifying existing platforms like Visual Studio Code (Weber et al., 2024, p. 2). This built-in design allows for much deeper features than a simple plugin, such as graph-based engines that remember the structure of your entire project and shadow workspaces for testing code safely in the background (Marcus, 2026, pp. 2, 4).

Cursor is a leading example that uses this integrated approach for editing many files at once and a vibe coding style where the AI translates a developer's broad goals into working code (Marcus, 2026, p. 4; Sabahi, 2025, pp. 1, 2).

Devin AI represents the shift toward autonomous agents that can act like a digital software engineer by planning their own steps, navigating folders, and using the terminal to complete tasks independently (Lyu et al., 2024, p. 2; Lau & Guo, 2025, pp. 11-15).

2.1.2 Efficiency Gains

Github Copilot has consistently been shown in empirical studies to deliver significant improvements in speed and efficiency. In controlled experiments, developers completed JavaScript programming tasks 55.8% faster than those without AI support. When working with legacy codebases (brownfield tasks), students using Github Copilot were 34.9% faster and made 50% more solution progress. At the workflow level, the tool reduces time spent on manual code entry by 10.6% and web searches by 11.6%, with AI assistants effectively replacing the browser as a primary information source (Shihab et al., 2025, pp. 2-21; Weber et al., 2024, pp. 6, 16, 25).

Amazon Q Developer has established itself as an integral component of professional software engineering, seeing widespread adoption as a primary daily workflow tool (Klemmer et al., 2024, p. 1; Stalnaker et al., 2026, p. 12). Empirical assessments utilizing the SWE-bench (a comprehensive evaluation framework) demonstrate the significant evolution of the platform's technical capabilities. Specifically, on the SWE-bench Lite dataset, which comprises 300 self-contained software defects, the v2 iteration of Amazon

Q Developer successfully resolved 29.67% of problems, representing a substantial quantitative advancement over the original version's resolution rate of 20.33%.

In industrial settings (such as telecommunications), Amazon Q Developer has been associated with more modest, situational productivity gains. Research indicates that users of industrial coding assistants like Amazon Q experience average time savings of approximately 13%, primarily in routine tasks like boilerplate generation and refactoring (Yu, 2025, p. 5).

AWS Kiro can achieve a 70% reduction in development cycles, according to empirical performance data. Case studies show that development teams using AWS Kiro can complete complex projects in approximately 45 minutes that traditionally require four hours of manual effort. By separating the design phase from implementation, AWS Kiro's workflow has been shown to reduce logic errors by 23% to 37% compared to reactive chat-based generation (Marcus, 2026, p. 17). The tool maintains a 95% accuracy rate for translating specifications into functional code. AWS Kiro is hypothesized to reduce a developer's mental demand and overall workload by 20% to 30%. This is achieved by automating repetitive implementation details, allowing developers to focus on business logic and architecture (Sharma, 2025, p. 24, 26).

Claude Code functions as an autonomous agent capable of multi-step reasoning and multi-file edits, unlike standard IDE extensions. A single natural language prompt can trigger the AI to independently traverse a repository, read relevant files, and apply changes across the codebase without constant human intervention.

As a Command-Line Interface (CLI) tool, it allows for a highly scriptable and immersive workflow for terminal-native developers (Lau & Guo, 2025, pp. 19, 31; Marcus, 2026, p. 13). Claude Code is noted for its superior complex reasoning capabilities and its effectiveness in executing multi-step execution plans. This makes it particularly efficient for high-level architectural refactoring and systematic debugging across large codebases (Marcus, 2026, pp. 15, 16).

While specific percentage metrics for Claude Code alone are less frequently cited, the tool belongs to a category of autonomous agents that preliminary studies suggest can provide 60% to 80% efficiency gains for specific software engineering tasks (Sharma, 2025, p. 2; Weber et al., 2024, p. 6). However, more studies and research are needed to validate this.

2.1.3 Code Quality

Github Copilot has demonstrated superior code quality, achieving high maintainability ratings in specialized machine learning pipeline benchmarks. It produced zero code duplications in specific churn and survival prediction tasks (Flamourtzoglou et al., 2025, p. 13). Conversely, broader industry data indicates a contrasting trend where AI-assisted workflows have facilitated a fourfold increase in duplicated code, potentially introducing technical debt that degrades long-term maintainability (Roy, 2025, p. 2). Beyond its role in code synthesis, Github Copilot functions as a knowledge transfer agent by providing proactive reminders of critical implementation details, such as missing database commit statements, that developers might otherwise omit during manual implementation (Welter et al., 2024, p. 15).

Amazon Q Developer is engineered for enterprise-grade structural robustness, prioritizing maintainability, and high-quality documentation (Flamourtzoglou et al., 2025, p. 5). Static analysis reveals high adherence to Python best practices, with scores reaching 9.14 out of

10 in specific machine learning benchmarks (Flamourtzoglou et al., 2025, p. 11). Structural assessments via Radon and SonarQube consistently grant the tool a maintainability rating of "A," characterized by low cyclomatic complexity and cognitive complexity scores (Flamourtzoglou et al., 2025, pp. 11–13). While industrial practitioners find it highly effective for routine tasks such as boilerplate implementation and Javadoc generation, they observe that its output often requires expert refinement for complex tasks involving organization-specific frameworks and nuanced architectural rules (Yu, 2025, pp. 9, 15–16).

AWS Kiro code quality is fundamentally driven by its architectural paradigm, which prioritizes systematic planning to ensure functional and structural integrity. This methodology achieves a 95% accuracy rate in translating technical specifications into production-ready code (Cursor IDE, 2025, pp. 2, 8). By decoupling the design phase from implementation, AWS Kiro reduces logic errors and hallucinations by 23% to 37% compared to reactive chat-based interfaces (Marcus, 2026, pp. 1, 17). In comparative assessments, the tool earned a code quality score of 94 out of 100, characterized by high architectural consistency and comprehensive test coverage (Cursor IDE, 2025, p. 26). Projections further indicate that the autonomous agentic workflow can increase automated test coverage by 6.7% while reducing bug density to 3.6 defects per 1,000 lines of code (Sharma, 2025, p. 22). It was observed that while AWS Kiro is exceptionally effective for generating maintainable documentation, maintaining quality in complex, domain-specific environments still require iterative prompt refinement and expert human oversight (Yu, 2025, pp. 1, 12, 16).

Claude Code ensures structural integrity by independently traversing repositories to read and edit multiple files. It operates as a terminal-native autonomous agent, thereby maintaining project-wide logic consistency (Lau & Guo, 2025, pp. 15, 31; Marcus, 2026, p. 13). The efficacy of its underlying engine is evidenced by Claude 3.5 Sonnet achieving a solve rate exceeding 50% on the SWE-bench Verified benchmark when utilized in structured frameworks (Xia et al., 2025, p. 2). However, it is noted that the quality of reasoning may face degradation in massive projects where the necessary context exceeds approximately 32,000 tokens (Marcus, 2026, p. 2). Claude Code transition software quality from reactive snippet generation to a model of autonomous logic where the agent independently executes complex terminal commands and file modifications to fulfill high-level intent (Lau & Guo, 2025, p. 19; Marcus, 2026, p. 13).

In summary, AWS Kiro is the optimal choice for projects requiring high architectural robustness; Claude Code is superior for complex algorithmic reasoning, and Amazon Q Developer provides the most reliable standardized maintainability metrics.

2.1.4 Security, Legal, and Ethical Risks

Several security, legal and ethical risks have been highlighted with respect to the use of coding assistants.

Github Copilot may generate code containing critical security vulnerabilities, such as Common Weakness Enumerations (CWEs). Empirical research indicates that approximately 40% of its generated code may contain such issues, primarily because the underlying LLMs are trained on unvetted public datasets (Pearce et al., 2022, p. 754; Salah, 2025, p. 2). Legally, the tool is subject to ongoing litigation, which alleges that the assistant violates open-source licenses by reproducing copyrighted code verbatim without proper attribution (Stalnaker et al., 2026, p. 4; Klemmer et al., 2024, p. 15). Ethically, Github Copilot has raised concerns regarding memorization, where the model outputs identical

copies of existing works, and the potential for deskilling, where developers may become overly reliant on AI suggestions and fail to master fundamental problem-solving skills (Stalnaker et al., 2026, p. 5; Pezzè et al., 2025, p. 15).

Amazon Q Developer is engineered for enterprise-grade security, emphasizing structural robustness and integrated safety features. It provides real-time security scanning and reference tracking to identify potential vulnerabilities and notify developers if generated code matches specific public datasets, thereby mitigating copyright risks (Flamourtzoglou et al., 2025, p. 9; Marcus, 2026, p. 9). While it benefits from the underlying AWS security posture, research suggests it may exhibit an optimistic bias, systematically under-predicting the severity of vulnerabilities during automated assessments (Özer & Dündar, 2026, p. 1). Ethically, Amazon Q Developer is distinguished as the only major assistant capable of performing comprehensive fairness and bias evaluations by computing disparate impact ratios for sensitive variables like gender, which enhances the reliability of software in regulated sectors (Flamourtzoglou et al., 2025, p. 18).

Claude Code leverages the superior reasoning of models like Claude 3.7 and 4.0 Sonnet to maintain project-wide logic consistency; however, its security posture is largely session-based. This means that its context and auditability are often limited to active interaction threads rather than integrated enterprise systems (Marcus, 2026, pp. 3, 15; Lau & Guo, 2025, p. 29). It faces potential reasoning degradation in projects where the required context exceeds 32,000 tokens (Lau & Guo, 2025, p. 28; Marcus, 2026, p. 2). Ethically, Claude Code transitions the developer's role from reactive snippet generation to a model of autonomous logic, requiring the human to act as a high-level orchestrator and certifier of independent AI decisions (Marcus, 2026, p. 13; Pezzè et al., 2025, p. 41).

AWS Kiro is positioned as the most secure solution for regulated industries due to its compliance inheritance model and architecture. By ensuring that code never leaves the AWS infrastructure, AWS Kiro provides a closed-loop environment that inherits certifications such as SOC 2, HIPAA, and PCI-DSS, while maintaining permanent audit trails via AWS CloudTrail (Cursor IDE, 2025, p. 31; Marcus, 2026, p. 3). Legally, AWS Kiro's structured workflow, separating requirements and design from implementation, reduces logic errors and hallucinations by 23% to 37%, addressing the trust gap inherent in reactive coding (Marcus, 2026, p. 17). Ethically, AWS Kiro addresses the accumulation of ethical debt by enforcing a "Human Gate" where developers must explicitly review and approve design blueprints and task lists before any code is generated, ensuring human accountability remains central to the autonomous lifecycle (Marcus, 2026, p. 2; Salah, 2025, p. 3).

2.1.5 Future Directions: Agents and Context Awareness

The transition toward autonomous AI agents and deep context awareness represents a fundamental paradigm shift in software engineering, moving from reactive autocomplete tools to proactive, collaborative partners that operate at the repository level (Hassan et al., 2026; Lau & Guo, 2025).

AI Software Engineering Agents

The evolution of AI coding assistants is categorised into distinct eras, with the current trajectory aimed at the development of Autonomous AI Agents between 2024 and 2025 (Lau & Guo, 2025, pp. 11, 30). Unlike earlier reactive autocomplete tools, these advanced agents perform multi-step reasoning, navigate complex, project-wide file systems, and

independently execute terminal commands to modify repositories on a user's behalf (Lau & Guo, 2025, p. 25; Sharma, 2025, p. 2).

Future directions envision a fundamental shift toward Software Engineering 3.0 (SE 3.0), an AI-native paradigm shift where development is driven by high-level human intents expressed through iterative, conversation-oriented interactions, allowing AI teammates to autonomously orchestrate the entire software lifecycle from design to implementation. Development becomes intent-centric and conversation-oriented rather than code-centric, with the AI partner driving the code-creation loop by synthesizing high-level human goals into runnable software (Hassan et al., 2026, pp. 5, 7).

Emerging approaches further include the orchestration of multiple agent instances operating in parallel, effectively functioning like a fleet of software engineers capable of managing large-scale architectural projects (Lau & Guo, 2025, p. 19; Pezzè et al., 2025, p. 4).

The framework known as Agentless can be conceptualized as a structured assembly line for software fixes, rather than a free-roaming robot that independently decides its own actions. Instead of allowing an AI to autonomously choose which tools to use or which files to visit, it follows a rigid, three-stage sequence to solve problems (Xia et al., 2025, pp. 1, 3, 10). This first phase called Localisation, involves systematically narrowing down hundreds of files to the specific classes, functions, or lines of code responsible for the error (Xia et al., 2025, pp. 7, 8). The second is Repair, where once the error is located, the system generates targeted code edits (patches), which focus on small, cost-efficient changes using a simple search-and-replace format rather than rewriting entire files. The final phase is patch validation which involves automatically creating and running tests to ensure the proposed fix resolves the problem while verifying that existing code functionality remains intact (Xia et al., 2025, pp. 9, 10).

Advancements in Context Awareness

Next-generation tools are being engineered to manage complex repository-level operations, such as systematic debugging and project-wide refactoring, which necessitate a granular comprehension of cross-file dependencies and architectural call graphs (Lau & Guo, 2025, p. 21; Nghiem et al., 2024, p. 10).

A foundational pillar of this era is the integration of Theory of Mind (ToM) frameworks; these enable AI teammates to infer a developer's mental state, internalize assumptions, and learn from historical interactions to dynamically align with unique coding styles or project constraints (Hassan et al., 2026, pp. 13–14; Pezzè et al., 2025, p. 18).

To sustain these reasoning-heavy capabilities in resource-constrained environments, context-aware management strategies like CACE (Context-Aware CodeLLM Eviction) come into play. CACE operates as a smart cache by intelligently managing limited accelerator memory to ensure that high-priority AI models remain ready for immediate use. Unlike standard strategies that simply remove the oldest models, CACE integrates specific factors, predicted developer demand, and task urgency, to minimize the lag associated with starting up a model from scratch. By keeping models essential for real-time tasks resident in memory, this framework reduces the initial response delay, or Time-to-First-Token (TTFT), by up to 70%, facilitating a significantly more responsive and fluid coding experience (Thangarajah et al., 2025, pp. 1, 2, 15).

Proactive Interactions and Vibe Coding

The future includes a definitive shift toward proactive AI behaviour, where the system acts without explicit prompting. For example, independently generating documentation when a user pauses editing or by highlighting project-wide architectural inconsistencies (Eibl et

al., 2025, p. 13; Lau & Guo, 2025, p. 16). Emerging platforms further enable vibe coding, a paradigm shift allowing users, including non-programmers, to create functional prototypes through high-level natural language prompts while remaining entirely abstracted from the underlying code implementation (Hassan et al., 2026, p. 20; Lau & Guo, 2025, pp. 13, 28). To address the persistent challenge of intent alignment, future agents will incorporate guided prompting, which utilizes a structured conversational stage to clarify ambiguous requirements and ensure the AI partner's internal model is aligned with human goals before the autonomous code generation cycle begins (Eibl et al., 2025, p. 14; Hassan et al., 2026, p. 8).

AWS Kiro and Amazon Q Developer possess vibe coding functionality through their conversational AI interfaces, but they prioritize developer control and architectural integrity (Marcus, 2026, p. 3; Sabahi, 2025, p. 5). Emerging tools like Lovable represent a more specialized and advanced application of the vibe coding paradigm, focusing on end-to-end app generation and deployment through high-level intent alignment (Hassan et al., 2026, pp. 5, 21; Lau & Guo, 2025, p. 28).

Challenges and Research Roadmaps

Despite the potential of next-generation agents, several technical and structural barriers remain. Independent benchmarks indicate that in some tools reasoning capabilities often degrade significantly once the context window exceeds a certain number of tokens (Marcus, 2026, p. 2). There is also an inherent trade-off between AI autonomy and user control, as higher levels of autonomy can reduce the human's ability to debug mistakes effectively or comprehend the underlying rationale behind complex, multi-file changes (Hassan et al., 2026, p. 2; Lau & Guo, 2025, p. 21). Future research must therefore establish formal methods for 'programming with trust,' a paradigm shift where autonomous agents provide verifiable evidence of correctness for generated code to ensure the necessary security and qualitative integrity (Hassan et al., 2026, p. 20; Pezzè et al., 2025, pp. 27, 29).

2.2 The SPACE Framework for developer productivity

2.2.1 Theoretical Origin

The SPACE framework was introduced in 2021 by a multidisciplinary team of researchers from Microsoft Research, GitHub, and the University of Victoria. It provides a holistic and multi-dimensional approach to understanding developer productivity (Forsgren et al., 2021, p. 20). The framework originated as a critical response to the long-standing challenges of conceptualising and measuring productivity in software engineering (Mohamed et al., 2025, p. 3).

2.2.2 Motivation

The SPACE framework was developed primarily as a response to the challenges of defining and measuring developer productivity using simplistic, traditional metrics (Forsgren et al., 2021, p. 20; Kaul et al., 2024, p. 2). The following sections detail the core motivations behind its creation as described in the sources:

Limitations of Traditional Metrics

Before the introduction of the SPACE framework, both industry and academic evaluations of developer productivity often relied on narrow, volume-based indicators such as Lines of Code (LoC), commit frequency, and task completion counts. Researchers have argued that these traditional metrics are inadequate because they capture only isolated snapshots of

work. It often incentivised undesirable behaviours, such as rewarding verbosity over efficiency, and failed to reflect the true value of a developer's contribution (Forsgren et al., 2021, pp. 20–21; Mohamed et al., 2025, p. 3). In addition, such measures can be easily manipulated; for example, commit counts may be inflated through trivial changes, while LOC-based measures may favour quantity over quality (Kaul et al., 2024, p. 2). More fundamentally, speed and output volume alone do not provide a complete account of productivity, since essential aspects of software development, including design, problem-solving, and knowledge sharing, are equally important to successful outcomes (Afroz et al., 2026; Mohamed et al., 2025, p. 3).

Software Development as a Socio-Technical System

Software development is a multi-dimensional socio-technical system, and productivity arises from the interplay among human, technical, and organizational factors (Forsgren et al., 2021, p. 20). This motivated a shift toward a framework that could account for qualitative human-centered factors, such as well-being, team collaboration, and psychological focus, which traditional telemetry-driven approaches largely ignored (Afroz et al., 2026; Mohamed et al., 2025, p. 2).

Holistic and Outcome-Driven Perspective

A key motivation was the belief that no single metric can capture productivity (Kaul et al., 2024, p. 2; Mohamed et al., 2025, p. 3). The framework was designed to acknowledge the trade-offs inherent in engineering work; for instance, high Activity does not necessarily indicate improved Performance unless it is accompanied by quality, sustained Satisfaction, and Efficient Flow (Afroz et al., 2026; Borg et al., 2025, p. 4). By providing a multi-dimensional organizing model, SPACE allows organizations to identify where gains in one area might introduce hidden costs in others (Mohamed et al., 2025, p. 3).

Adaptation for the Age of AI

While originally intended for traditional workflows, the rise of AI coding assistants provided new motivation for using SPACE (Mohamed et al., 2025, p. 3). Most initial efforts to measure AI's value focused narrowly on coding speed. However, research showed that developers only spend about 14% of their time coding new features, making speed-only metrics an incomplete measure of AI's impact (Houck et al., 2025, p. 2).

Furthermore, the non-deterministic nature of AI outputs means that speed gains in code generation (Activity) often shift the burden to code review and verification (Performance and Efficiency). This necessitates a framework that can capture these spurious productivity shifts and redistributed effort (Afroz et al., 2026; Chen et al., 2026, p. 2; Tomaz et al., 2026, pp. 2, 9).

2.2.3 Empirical applications of SPACE framework

The SPACE framework is increasingly being used in empirical research as a multidimensional lens for examining developer productivity, particularly in the context of AI-supported software engineering.

In longitudinal industrial studies, for example, a 13-month investigation of agile teams at a global consulting firm used SPACE to identify a “Performance–Activity–Efficiency divergence.” It showed that that team-level value could increase even when raw developer activity remained stable (Tomaz et al., 2026, p. 2).

The framework has also been applied to research on human factors and well-being, where it has been used to demonstrate that the impostor phenomenon has a statistically significant negative effect across all five SPACE productivity dimensions among professional engineers (Guenes et al., 2024, p. 3).

In addition, systematic literature reviews have adopted SPACE as an organizing model to synthesize findings from dozens of peer-reviewed studies, showing that Satisfaction, Performance, and Efficiency are the dimensions most frequently examined in current AI research (Mohamed et al., 2025, p. 3).

Repository-mining studies have likewise used the framework to examine whether automated telemetry, such as commit interaction frequency and pull request merge times, can capture dimensions like communication and efficiency that have traditionally been regarded as more qualitative in nature (Kaul et al., 2024, p. 4).

Finally, large-scale mixed-methods practitioner surveys conducted at institutions such as Microsoft and BNY Mellon have used SPACE to show that AI coding assistants tend to improve perceived satisfaction more clearly than objective time savings, which are often found to be more modest (Chen et al., 2026, p. 4; Houck et al., 2025, p. 3).

2.2.4 Limitations of SPACE framework

While the SPACE framework is widely recognized as an industry standard for providing a holistic view of developer productivity, empirical research highlights several significant limitations regarding its scope, measurement accuracy, and application to AI-driven workflows.

The following sections detail the limitations of the SPACE framework identified in the sources:

Conceptual and Scope Gaps

Research suggests that the original SPACE framework, which was designed for traditional software engineering contexts, does not fully capture several dynamics introduced by AI coding assistants.

One limitation is its emphasis on short-term productivity outcomes, which means it does not explicitly account for important long-term effects such as the erosion of technical expertise, the development of junior engineers, or the maintenance of collective code ownership.

In addition, SPACE does not address self-sufficiency, even though AI assistants can significantly influence a developer's independence by shifting reliance away from teammates or documentation and toward the tool itself (Chen et al., 2026, p. 8).

A further limitation is that, because the framework was not originally developed for human-AI collaboration, researchers must make subjective interpretive judgments when mapping AI-related findings onto its sub-dimensions, which may introduce researcher bias (Mohamed et al., 2025, p. 30).

Challenges in Data Collection and Measurement

In practice, applying the SPACE framework can be challenging because it is often difficult to identify reliable measures for all five dimensions.

Many studies rely on self-reported survey data, but such measures do not always reflect objective outcomes (Afroz et al., 2026). They are also vulnerable to perceptual biases, such as the “novelty effect,” where satisfaction is temporarily inflated because the technology is new. Also, the “Hawthorne effect,” where participants change their behaviour because they know they are being observed (Tomaz et al., 2026, p. 22). In addition, developers may evaluate their productivity more positively simply because they believe in the value of AI (Borg et al., 2025, p. 59).

Automated repository data also has clear limitations. Qualitative dimensions such as Efficiency and Flow or Communication are difficult to capture through repository mining alone. For example, pull request merge time is often used as an efficiency metric, but this

can be misleading because it does not account for factors such as pull request size, reviewer workload, or whether delays were caused by refactoring rather than poor code quality. Likewise, attempts to measure Satisfaction through sentiment analysis of commit messages are often unreliable due to domain mismatch. Models trained on general-language data, such as Twitter, may misinterpret technical expressions; for example, the phrase “killing a program” could be classified as negative sentiment even though it refers to a routine technical action (Kaul et al., 2024, pp. 8-10).

Implementation and Statistical Constraints

Although the SPACE framework promotes a holistic view of productivity, most empirical studies do not assess all five dimensions at the same time.

In fact, 86% examine no more than three dimensions, leaving areas such as Communication and Activity particularly underexplored and resulting in fragmented insights (Mohamed et al., 2025, pp. 2, 22).

In addition, there is no scientific consensus on how to combine the five dimensions into a single composite productivity score, since the relative importance of each dimension may vary across organizations (Kaul et al., 2024, p. 11).

Longitudinal applications also face a further challenge. It is difficult to separate productivity gains attributable to AI from improvements that arise naturally as teams mature and become more familiar with their projects over time (Tomaz et al., 2026, p. 23).

Systematic Risks

The framework can sometimes obscure a redistribution of constraints rather than reveal genuine overall improvement. For example, gains in the Activity dimension, such as writing code faster, may be offset by hidden costs in other dimensions, including higher verification demands in Performance and increased cognitive load in Satisfaction.

As a result, if an organization focuses only on the dimensions in which AI appears to improve outcomes, it may conclude that the tool is effective while overlooking the potentially spurious nature of those gains (Afroz et al., 2026).

2.3 Task–Technology Fit (TTF) theory

2.3.1 Origin

Theoretical foundation and origin of TTF

TTF was originally introduced by Goodhue and Thompson (1995). The central premise of TTF is that information technology enhances an individual’s performance only when the technology’s functionality aligns closely with the specific task it is intended to support (Goodhue & Thompson, 1995, p. 214).

TTF is defined as the degree to which a technology can assist an individual in performing a desired task (Goodhue & Thompson, 1995, p. 216). More specifically TTF is defined as the correspondence between the 3 components: task requirements, individual abilities and the functionality of the technology. The theory suggests that performance benefits arise when the technology offers functionality and support that “fits” the task at hand. Moreover, it is also suggested that if the gap between functionality and the task increases, the fit will simultaneously decrease (Goodhue & Thompson, 1995, pp. 214, 218).

Tasks are broadly defined by the authors as the actions performed by an individual to transform inputs into outputs. Task characteristics of interest refers to aspects of a task that

determine which technological features a user relies on, as some tasks require certain features of a technology more than others. Individual abilities refer to the characteristics of the individual that could affect how easily or well they are able to utilize technology. Lastly, technology refers to the tools individuals use to carry out their tasks. In this study, the technologies refer to information systems, more specifically GenAI (Goodhue & Thompson, 1995, p. 216).

Technology-to-Performance Chain (TPC)

TTF is at the core of the broader conceptual model Technology-to-Performance Chain (TPC) (Chang & Huang, 2026, p. 2). TPC postulates that the contribution of technology to an individual's performance does not depend solely on its utilization, but rather on a chain of factors, especially the alignment between the technology's capabilities and the requirements of the task (Marikyan & Papagiannidis, 2023, p. 6; Bazine & Salahddine, 2025, p. 241). To get a positive effect on performance, it is necessary that the technology has a strong fit with the task and that it is actually utilized by the user (Marikyan & Papagiannidis, 2023, p. 6).

Definition of "FIT"

Within the framework, the word "fit" is defined as the extent to which a piece of technology can assist an individual in performing their portfolio of tasks (Bazine & Salahddine, 2025, p. 241).

Moreover, the theory is grounded in the assumption of a rational user, it assumes that experienced individuals select and value tools that reduce the cost of performing a task and that provides the greatest benefit for its completion (Dishaw & Strong, 1998, p. 109).

Operationalization and measurements

To operationalize TTF, Goodhue originally developed an instrument based on a decision-making model, resulting in a set of dimensions used to assess fit (Goodhue, 1998, pp. 105, 106). These dimensions include, but are not limited to including data quality, accessibility, ease of use, and system reliability (Dishaw & Strong, 1998, p. 155). These dimensions have demonstrated strong validity in explaining how users evaluate systems and how fit, in turn, predicts perceived performance (Goodhue, 1998, p. 105).

2.3.2 Empirical applications of TTF

Generative AI and the software development life cycle

In recent years, GenAI tools, such as ChatGPT and Github Copilot, have become more prominent in software development (Chang & Huang, 2026, p. 1). Moreover, Passmore et al. (2025, p. 41) states that TTF has been widely applied in human-computer interaction research and recently extended to AI applications. Chang & Huang (2026, pp. 1, 4) executes a study that examines how the alignment between AI tools and developers' tasks influences perceived performance in systems development.

In the context of this study software development is not treated here as a single isolated task, but rather as a set of activities included in the entire development lifecycle (Chang & Huang, 2026, pp. 3, 4).

The empirical findings showed that TTF had a direct and statistically significant positive effect on system development performance (Chang & Huang, 2026, p. 9). The authors argue that a high degree of fit arises when the capabilities of AI tools, such as code generation and debugging, align with developers' mental models and workflows (Chang & Huang, 2026, p. 4).

According to Dishaw and Strong (1998, p. 155), the concept of task can be understood at the level of a portfolio of tasks rather than as a single isolated task. They argue that applying TTF to a portfolio of tasks rather than to individual activities is fully consistent with the model's theoretical definition, in which the value of technology is assessed based on how well it supports an individual in performing their overall set of tasks. Moreover, they propose that application of task-system fit, to differentiate, even though the framework stays the same.

Impact on individual performance and productivity

The ability of the TTF framework to predict performance has been confirmed across a range of digital contexts in which users are confronted with complex information. In a study about smartphone usage in the context of library systems, it was demonstrated that TTF has a significant positive impact on individuals perceived performance (Vongjaturapat, 2018, pp. 40, 51). The study establishes that if the technology does not align with the user's specific needs, this leads to a lower evaluation of the system's usefulness and effectiveness (Vongjaturapat, 2018, p. 51).

This is further supported by research conducted by Lin (2012, p. 498, 503), which integrated TTF with theories of system continuance and found that perceived fit is one of the most important factors of both the intention to continue using a system and actual individual performance. These studies provide empirical support for TTF as a robust variable for measuring how technology influences productivity in software development.

Methodological practice: TTF as an aggregated construct

In applying TTF to newer technologies, it has become increasingly common to simplify the model's operationalization. Marikyan and Papagiannidis (2023, p. 10) note in their review that it has become established practice ("fit-as-match") to treat TTF as a unidimensional first-order construct. This means that users directly assess how well the technology as a whole, fits their tasks, rather than evaluating all 12 original dimensions separately.

This simplified approach has been successfully applied to examine a range of technologies, from e-books to knowledge portals, where it has been found that perceived fit directly improves work quality, productivity, and job performance (Marikyan & Papagiannidis, 2023, p. 10).

2.3.3 Limitations of TTF

Despite its widespread use, the Task-Technology Fit model and its extensions have been identified as having several theoretical and methodological limitations that are important to consider when evaluating modern technologies.

Theoretical scope and explanatory power

A recurring criticism of the TTF model is its relatively low explanatory power in accounting for actual technology utilization, compared to attitude-based models such as the Technology Acceptance Model (TAM). Studies have shown that TTF typically explains a substantially larger proportion of variance in performance than in utilization, suggesting that a good fit does not automatically ensure that a tool will be used (Marikyan & Papagiannidis, 2023, pp. 5, 6). Furthermore, the model has been criticized for being overly technology-focused and for largely neglecting individual psychological factors, such as motivation, personality traits, and computer skills (Marikyan & Papagiannidis, 2023, pp. 13, 14).

The interaction between the individual and the task

A significant theoretical gap in the original TTF model is the lack of attention to the interaction between the individual and the task. Research indicates that failed IT projects are often a result of a fundamental lack of acceptance of the task itself rather than the tool itself. If the user is not motivated to perform the task, the fit of the technology plays a secondary role.

In addition, TTF tends to conceptualize fit as a static state, whereas more recent research emphasizes that alignment is a dynamic process that changes over time as users' skills develop and the technology evolves (Ammenwerth, Iller, & Mahler, 2006, pp. 3, 10).

Methodological challenges and simplifications

The operationalization of TTF often involves complex measurement instruments with multiple dimensions, making empirical testing difficult to conduct without the study becoming overly extensive.

To increase the model's generalizability, researchers have often resorted to using one-dimensional scales ("fit-as-match"), which may reduce the model's ability to explain specific factors that hinder or facilitate performance (Marikyan & Papagiannidis, 2023, p.13).

Another methodological limitation is the reliance on self-reported data, which introduces a risk of inconsistencies between users' subjective perceptions and objective reality (Marikyan & Papagiannidis, 2023, p.3).

Furthermore, due to the perceptual nature of most TTF measures, it remains difficult to establish causal direction, as users who experience positive outcomes may retrospectively evaluate the technology as a better fit. This introduces ambiguity in whether perceived fit drives performance, or whether perceived performance influences evaluations of fit.

These limitations are particularly relevant in the context of this study. As both Task–Technology Fit and productivity are operationalized as perceived constructs, the relationship between them may reflect subjective evaluation patterns rather than purely causal effects. This highlights the importance of interpreting the findings with caution, as perceived alignment and perceived productivity may be closely intertwined.

In addition, the assumptions underlying TTF may be challenged in the context of GenAI tools. Unlike traditional information systems, GenAI coding assistants operate in environments where both task requirements and technological capabilities evolve during use. This dynamic and iterative nature of interaction makes the notion of a stable task–technology fit more difficult to define, suggesting that the explanatory power of TTF may be more complex in AI-assisted software development contexts.

2.3.4 A comparison of TTF and TAM in relation to developer productivity

Within research on technology adoption, the Technology Acceptance Model (TAM) is a dominant framework that focuses on users' attitudes, particularly perceived usefulness and perceived ease of use, to explain their intention to use a system (Ammenwerth et al., 2006, p. 3). However, an important limitation of TAM in professional contexts such as software development is its limited attention to the work task itself (Vongjaturapat, 2018, p. 44). Previous research has shown that high system utilization does not necessarily translate into improved performance. In some cases, extensive use of a system may provide limited

productivity benefits when the technology is poorly aligned with task requirements (Goodhue, 1998, p. 107; Marikyan & Papagiannidis, 2023, pp. 5–6).

Whereas TAM focuses primarily on the intention to use a system, TTF addresses the post-adoption phase by examining how well the functionality of a technology aligns with the specific requirements of the tasks being performed (Marikyan & Papagiannidis, 2023, pp. 6–7). In the context of complex, knowledge-intensive work such as software development, TTF is therefore better suited to this study because it emphasizes that productivity gains depend on whether the technology meaningfully supports task-related problem solving (Chang & Huang, 2026, p. 10; Bazine & Salahddine, 2025, p. 249). By using TTF as an explanatory variable in relation to the dimensions of the SPACE framework, this study can examine how perceived alignment between AI tools and developers' tasks and workflows is associated with developer-perceived productivity, rather than merely measuring general attitudes toward the tool.

2.4 Synthesis: Integrating SPACE and TTF

A key insight emerging from the literature is that the SPACE framework and TTF theory address complementary aspects of developer productivity. SPACE provides a multidimensional view of productivity outcomes, whereas TTF offers an explanatory perspective on why those outcomes may vary depending on how well a technology aligns with users' tasks and work conditions.

The SPACE framework conceptualizes developer productivity across five dimensions: Satisfaction and Well-being, Performance, Activity, Communication and Collaboration, and Efficiency and Flow. By explicitly warning against reducing productivity to a single metric, it provides a structured basis for identifying trade-offs between different aspects of work. However, although SPACE is well suited to describing what changes in developer productivity, it is less explicit about why a particular tool leads to positive outcomes in some contexts but not in others.

TTF complements this limitation by proposing that technology improves performance when its capabilities fit the tasks users must perform and the conditions under which those tasks are carried out. In this sense, TTF provides an explanatory mechanism for understanding variation in perceived productivity, while SPACE provides the outcome dimensions through which those effects can be observed.

The combined logic of this thesis is therefore that perceived TTF functions as a key explanatory construct, whereas SPACE provides a structured framework for capturing downstream productivity outcomes. More specifically, TTF helps explain why coding assistants may improve perceived productivity when developers experience them as supportive of their broader task portfolio, including comprehension, debugging, refactoring, and documentation, under the real constraints of enterprise software development, such as large codebases, review processes, architectural dependencies, and security requirements.

This integration also aligns with a HCAI perspective. From this viewpoint, productivity gains should not be interpreted merely as increased output, but rather as the extent to which the tool augments developer capability while preserving human control, confidence in correctness, and a manageable cognitive workload.

This logic can be illustrated by considering how common fit-related factors identified in TTF research and recent evaluations of coding assistants may influence each SPACE dimension:

Satisfaction and Well-being: Fit is reflected in the extent to which the assistant reduces friction in daily work, for example by lowering frustration or increasing confidence. Conversely, misfit may appear as increased cognitive burden when developers must expend substantial effort verifying or correcting generated output.

Performance: Fit is strongest when the assistant supports outcome-relevant work, such as correctness, maintainability, and adherence to standards. Misfit may instead lead to spurious productivity, where apparent gains in output are offset by downstream defects, rework, or reduced code quality.

Activity: Fit may increase task throughput or expand the range of tasks developers can complete with assistance. However, activity alone is not a sufficient indicator of productivity and must be interpreted alongside performance and efficiency to avoid rewarding output volume without considering value.

Communication and Collaboration: Fit may influence whether the assistant reduces help-seeking by supporting faster code comprehension or instead increases coordination costs by generating larger changes that require heavier review and more interaction among team members.

Efficiency and Flow: Fit is closely tied to whether the assistant supports uninterrupted progress and reduces context switching. Misfit, by contrast, may appear in the form of tool instability, prompting overhead, repeated verification cycles, or difficulty maintaining continuity across tasks.

This mapping clarifies why the SPACE dimensions may move in different directions. For example, a coding assistant may improve Efficiency and Flow while offering limited support for Communication and Collaboration if it speeds up individual work but shifts effort into code review and coordination.

Empirical research has increasingly used SPACE to evaluate AI coding assistants, but an important gap remains. SPACE-based studies tend to describe perceived productivity outcomes without a clearly specified explanatory mechanism for why those outcomes differ across individuals, tasks, and contexts.

Conversely, TTF research explains variation in performance through perceived fit but typically does not capture the multidimensional nature of developer productivity emphasized by SPACE.

Integrating SPACE and TTF therefore offers a stronger evaluative logic for this thesis. SPACE provides a comprehensive set of multidimensional outcome variables, while TTF explains variation in those outcomes through the perceived alignment between tool capabilities and task demands. This combined perspective is particularly relevant in enterprise software development, where context availability, workflow integration, governance constraints, and code quality expectations strongly influence whether coding assistants generate sustainable value in practice.

To the best of our knowledge, no prior empirical study has combined these two frameworks in the context of AI coding assistants.

3 Theoretical Framework

The overarching perspective of this study is HCAI, which frames AI as augmenting human capabilities rather than replacing them. To explain variation in developer-perceived productivity, the study draws on TTF as its core explanatory framework. Developer-perceived productivity is conceptualized through the SPACE framework, which provides a multidimensional lens for assessing productivity outcomes.

3.1 Human-Centered AI - Overarching perspective

This study adopts the HCAI perspective as it's overarching theoretical framework. The HCAI perspective represents a fundamental change in how humans view technology through the integration of advanced AI algorithms and a human centered design (Shneiderman, 2022, chap. 2, p. 43). Traditionally, the view of automatization has been one-dimensional, where it has been suggested that as automatization increases, human control decreases (Shneiderman, 2022, chap. 2, pp. 48, 59). The HCAI framework challenges this one-dimensional way of thinking and introduces a two-dimensional model that suggests that high levels of automatization and high levels of human control can coexist (Shneiderman, 2022, chap. 2, pp. 45, 59).

The primary goal with HCAI is not to substitute human intelligence, but instead to increase it through tools and thereby give it more power (Shneiderman, 2022, chap. 2, pp. 69, 81). Furthermore, within HCAI, computers and machines are not viewed as teammates in otherwise human teams, but rather as tools that enhance control over complex situations (Shneiderman, 2022, chap. 2, pp. 57, 78).

When examining developer-perceived productivity in relation to GenAI, the HCAI perspective is particularly relevant. For instance, it puts emphasis on the balance between automation and human control. Within software development, GenAI can automate repetitive programming tasks, but HCAI emphasizes that developers must retain control to ensure system quality (Shneiderman, 2022, chap. 2, pp. 45, 51). Productivity should therefore not be measured solely by the number of generated lines of code, but also by how well the tool enables developers to control the process and quickly identify errors. Moreover, GenAI can sometimes hallucinate; therefore, HCAI promotes technical practices that support human responsibility, fairness, and explainability, which are crucial for assessing a GenAI model's contribution to developer productivity in a professional environment. Code generated by GenAI must be trustworthy, which in this context means that the developers can rely on the output being truthfully grounded and also have the tools to validate it (Shneiderman, 2022, chap. 2, pp. 53, 55).

Lastly, HCAI stresses that assessing AI systems should go beyond technical metrics and consider human experience, agency, and well-being. From this perspective, developer-perceived productivity in relation to GenAI in software development is measured not merely by the accuracy of its outputs or gains in productivity, but also by how it supports users in their tasks, influences their confidence, and integrates into established workflows.

3.2 SPACE Framework

To assess perceived productivity in software development, one-dimensional metrics are insufficient from a HCAI perspective. The SPACE framework is therefore used as a multidimensional lens aligned with the objectives of this study.

As introduced earlier, SPACE captures developer productivity through five dimensions: Satisfaction, Performance, Activity, Communication & Collaboration, and Efficiency & Flow. Together, these dimensions provide a holistic view of developers' work and support a broader assessment of productivity than output-based measures alone.

Satisfaction concerns how fulfilled developers feel in relation to their work, tools, team, and culture, and is relevant because declining satisfaction may signal burnout and reduced productivity (Forsgren et al., 2021, p. 48).

Performance concerns the quality and impact of outcomes rather than output volume alone, since high output does not necessarily imply high value or quality (Forsgren et al., 2021, p. 48).

Activity refers to observable work actions and outputs but should not be interpreted in isolation because many important developer activities are difficult to quantify and activity alone provides only a partial view of productivity (Forsgren et al., 2021, p. 48, 49).

Communication & Collaboration concerns how developers and teams share information and coordinate work, which is central in software development even if many collaborative processes must be assessed through indirect indicators (Forsgren et al., 2021, p. 49).

Efficiency and flow concern the ability to make progress with minimal interruptions, delays, and unnecessary rework, both individually and across systems (Forsgren et al., 2021, p. 49, 50).

Together with the HCAI perspective, the SPACE framework provides the theoretical basis for assessing whether GenAI supports human work in a meaningful way. While HCAI emphasizes augmentation with retained human control (Shneiderman, 2022, chap. 2, p. 59), SPACE helps evaluate whether GenAI enhances human capabilities rather than merely increasing machine-driven activity (Shneiderman, 2022, chap. 2, p. 43).

3.3 Task–Technology Fit

To explain how GenAI usage affects an individual's performance, the TTF theory is used in compliance with the description presented by Goodhue & Thompson (1995). The central premise of TTF is that information technology enhances an individual's performance only when the technology's functionality aligns closely with the specific task it is intended to support (Goodhue & Thompson, 1995, p. 214).

TTF is defined as the degree to which a technology can assist an individual in performing a desired task (Goodhue & Thompson, 1995, p. 216). The theory suggests that performance benefits arise when the technology offers functionality and support that “fits” the task at hand. Moreover, it is also suggested that if the gap between functionality and the task increases, the fit will simultaneously decrease (Goodhue & Thompson, 1995, pp. 214, 218).

Tasks are broadly defined by the authors as the actions performed by an individual to transform inputs into outputs. Task characteristics of interest refer to aspects of a task that determine which technological features a user relies on, as some tasks require certain features of a technology more than others. Individual abilities refer to the characteristics of the individual that could affect how easily or well they are able to utilize technology. Lastly, technology refers to the tools individuals use to carry out their tasks. In this study, the technologies refer to information systems, more specifically generative AI (Goodhue & Thompson, 1995, p. 216).

3.4 Theoretical Framework and Analysis

Figure 1 illustrates the integration of the theoretical frameworks used in this study. The model shows how the HCAI perspective, the SPACE framework, and TTF theory is combined to guide the evaluation of GenAI coding assistants in enterprise software development.

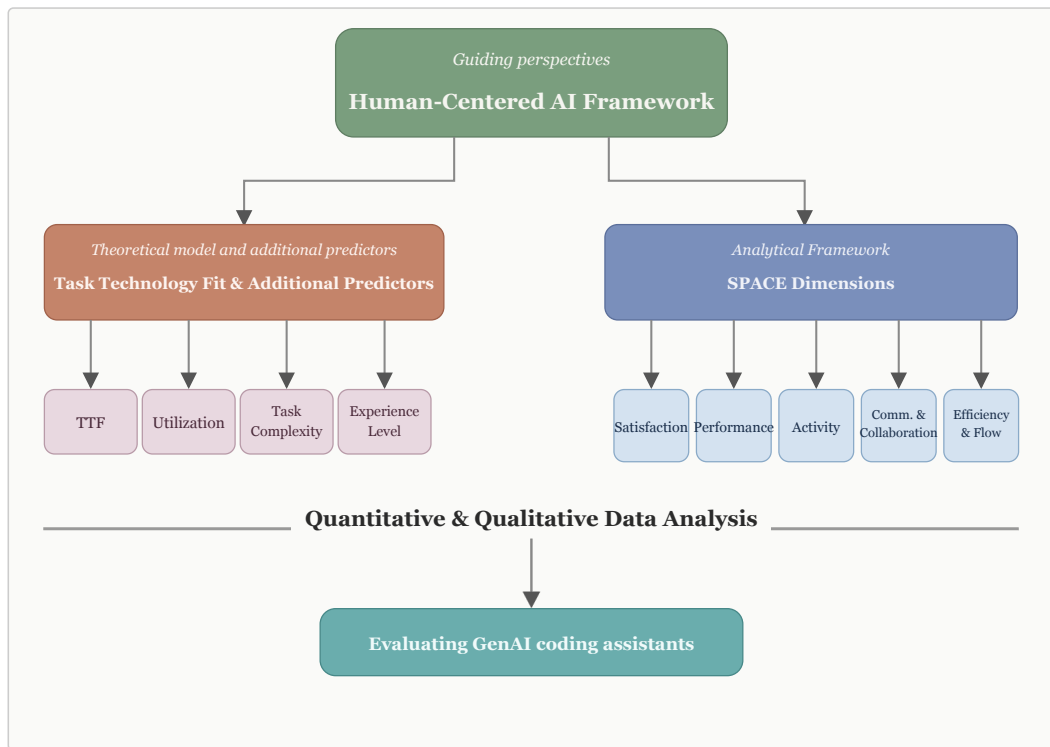


Figure 1: Integration of theoretical frameworks

The HCAI perspective provides an overarching lens, emphasizing that AI systems should augment human work while preserving human control, responsibility, and understanding.

The SPACE framework operationalizes this perspective by defining multiple dimensions of developers' perceived productivity, enabling a nuanced assessment.

TTF theory complements this by offering an explanatory foundation for understanding why productivity outcomes may differ depending on how well tool capabilities align with task demands.

Together, these frameworks establish the theoretical foundation for the study and motivate the conceptual model and expected relationships presented in the following section.

3.5 Conceptual Model and expected relationships

Figure 2 presents the conceptual model and the expected relationships between the main constructs. The model illustrates how perceived TTF is expected to be positively associated with developers' perceived productivity outcomes across the individual SPACE dimensions, reflecting the assumption that productivity benefits depend on how well tool capabilities align with task demands. Utilization is included as a contextual variable, reflecting that the technology must be used for effects to occur, but it is not expected to independently explain variation in perceived productivity outcomes once TTF is accounted for.

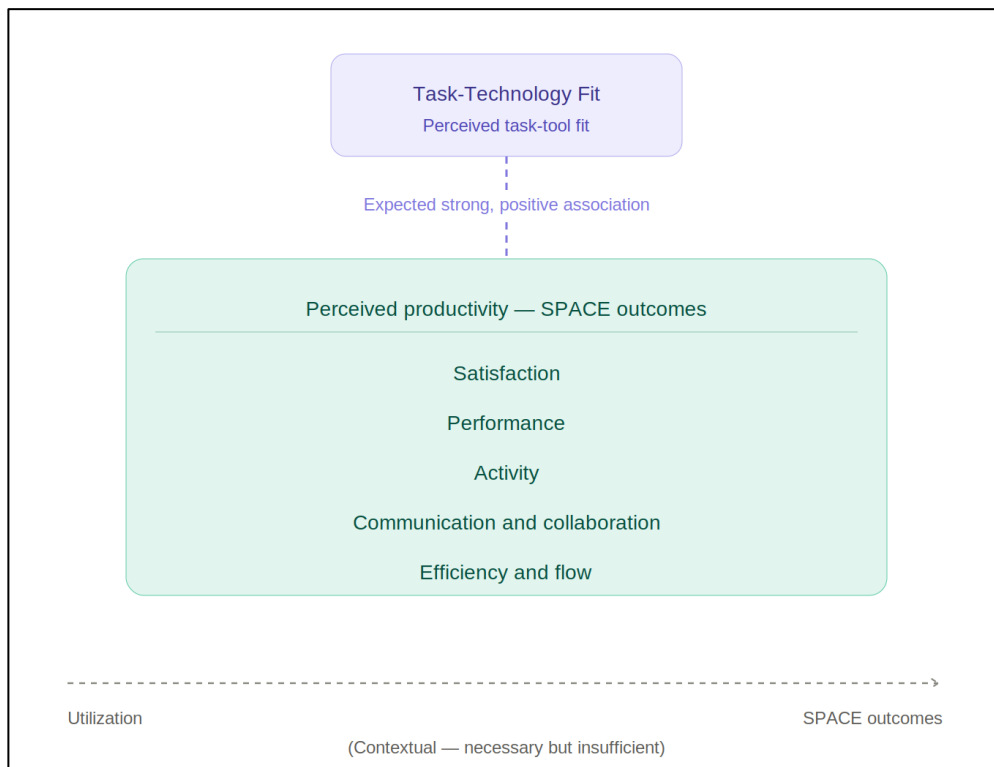


Figure 2: Conceptual Model and expected relationships between the main constructs

4 Methodology

4.1 Philosophical Tradition

This study adopts pragmatism as its overarching philosophical tradition. Pragmatism is appropriate because the study addresses a practical organizational problem and seeks to generate useful knowledge about how coding assistants are perceived and used in enterprise software development. Rather than committing exclusively to a single view of knowledge, pragmatism supports the use of different methods and forms of evidence when they are relevant to answering the research questions.

Within this pragmatic orientation, the quantitative component draws on post-positivist assumptions, in the sense that developers' perceptions of coding assistants, productivity, and TTF can be measured systematically through structured survey items and analysed statistically, even though such measurements are not assumed to be perfectly objective. At the same time, the qualitative component reflects a more interpretive orientation, as open-ended responses are used to understand how developers make sense of their experiences and how contextual factors shape their perceptions of these tools in practice. This philosophical position is consistent with the study's mixed-methods design.

The quantitative strand provides structured evidence of patterns and relationships across respondents, while the qualitative strand adds depth, nuance, and contextual understanding. Together, these approaches support a more comprehensive analysis of developer-perceived productivity than either approach would provide on its own.

In terms of reasoning, the study is primarily deductive, since the quantitative analysis examines relationships derived from the theoretical frameworks of SPACE and Task–Technology Fit. However, the qualitative thematic analysis also includes an inductive element, as recurring patterns are identified from respondents’ open-ended accounts rather than imposed entirely in advance. In this way, the study combines theory-driven explanation with openness to emerging insights from practice.

4.2 Methodological Approach

This study adopts a mixed-methods survey design to examine developers’ perceived productivity when using coding assistants in enterprise software development. The design is appropriate because the study seeks both to identify broad patterns in developers’ perceptions and to understand the contextual experiences underlying those patterns. Consistent with a HCAI perspective, the focus is not limited to technical tool performance, but extends to how developers experience, interpret, and evaluate these tools in practice.

More specifically, the study follows a convergent mixed-methods approach, in which quantitative and qualitative data are collected through the same survey instrument and analysed in parallel for complementary purposes. The quantitative component provides a structured assessment of perceived productivity across the five SPACE dimensions, while the qualitative component captures contextualized accounts of how developers experience the benefits, limitations, and conditions of coding assistant use in everyday work.

Within the quantitative analysis, the analytical strategy proceeds in two steps. First, the SPACE framework is used descriptively to capture a multidimensional view of perceived productivity. Second, explanatory variables are introduced, most notably perceived TTF, to examine statistically how perceived fit relates to productivity outcomes across the SPACE dimensions.

The survey was distributed to software professionals within the organization, resulting in 160 responses. Participation was voluntary and anonymous. The instrument combined Likert-scale items with open-ended questions, thereby supporting both statistical analysis and thematic exploration.

Overall, this methodological approach enables the study to generate both descriptive and explanatory insights into how coding assistants are perceived and used in enterprise software development.

4.3 Data Collection

4.3.1 Quantitative Data Collection

Quantitative data was collected through a survey that were distributed to software practitioners within the organization. Both surveys focus on developers’ use of coding assistants in software development and were designed to capture perceptions of developer productivity as well as related contextual factors.

The survey consisted primarily of five-point Likert-scale questions, complemented by open-ended questions to allow respondents to elaborate on their experiences. The survey also included items capturing contextual variables such as experience level, task complexity, and utilization of coding assistants. Detailed information of the survey is indicated in Appendix C.

The survey was designed based on the theoretical constructs described in Chapter 3. Developer productivity was captured through survey items aligned with the five SPACE dimensions. These items later supported both the descriptive SPACE analysis and the

explanatory regression analysis, although the two analyses relied on partly different operationalizations for different analytical purposes. The survey also included questions related to perceived TTF to assess how well coding assistants support developers in performing their tasks.

The use of the survey with partially overlapping constructs, enables both broad descriptive analysis of developer productivity and theory-driven explanatory analysis, strengthening the overall analytical depth of the study.

Alongside the survey data, additional quantitative data were collected on coding assistant adoption within the organization, including usage patterns for Amazon Q and AWS Kiro. Screenshots of these metrics are captured in Appendix E.

4.3.2 Qualitative Data Collection

In addition to the quantitative data, qualitative data was collected through open-ended questions included in the survey. This allowed the participants to elaborate on their experiences of using coding assistants in their daily software development work. The purpose of including open-ended questions was to capture insights that may not be fully reflected in the Likert-scale style of questions. This approach allowed participants to describe perceived benefits, challenges, and contextual factors that influence their use of coding assistants.

The qualitative data complements the quantitative measures by providing richer descriptions of developers' experiences, supporting a more comprehensive understanding of developer productivity from a HCAI perspective.

4.3.3 Operationalization of Constructs

The survey items were designed to operationalize the variables included in the study's conceptual model, based on the theoretical frameworks outlined in Chapter 3.

In this study, the SPACE framework was used in two related but analytically distinct ways. First, it was used descriptively to provide a broad multidimensional overview of perceived developer productivity. For this purpose, the descriptive SPACE analysis drew on a broader operationalization of the dimensions. Second, the SPACE dimensions were used as dependent variables in the regression analysis examining the explanatory role of perceived Task–Technology Fit. For this purpose, the original productivity items were retained to preserve conceptual separation between outcome variables, explanatory variables, and contextual predictors. As a result, the descriptive SPACE analysis and the regression analysis should not be interpreted as relying on identical dimension definitions.

4.3.4 Task–Technology Fit (TTF)

Perceived TTF was measured using a scale consisting of three items, capturing the extent to which coding assistants are perceived to support developers in performing their tasks. This operationalization reflects a generalized perception of fit, rather than the full multidimensional structure originally proposed in the TTF framework. The use of a reduced scale is a deliberate methodological choice. As TTF functions as an explanatory construct in this study rather than the primary object of analysis, a concise measure was prioritized to reduce survey fatigue and improve response rates.

4.3.5 SPACE Framework

Developer productivity was operationalized through the five SPACE dimensions: Satisfaction, Performance, Activity, Communication and Collaboration, and Efficiency and Flow. In this study, however, the SPACE framework was used for two related but analytically distinct purposes. For the descriptive SPACE analysis, the dimensions were operationalized more broadly, as shown in Table 1, in order to provide a multidimensional overview of perceived developer productivity. For the regression analysis, by contrast, the original productivity items were retained and used as dependent variables in order to preserve conceptual separation between the SPACE outcomes, perceived TTF, and the contextual predictors. As a result, the item counts and composition of the SPACE dimensions differ depending on analytical purpose, and the descriptive SPACE analysis and regression analysis should not be interpreted as relying on identical dimension definitions.

Accordingly, the item counts and composition of the SPACE dimensions vary depending on whether the analysis is descriptive or explanatory. Given the professional context and the time constraints of participants, the original survey instrument was intentionally kept concise to support response rates and completion quality.

Each dimension was measured using survey items rated on a five-point Likert scale (1=Strongly Disagree, 5=Strongly Agree), with higher scores indicating more positive outcomes. Table 1 provides a summary of the operationalization by dimension. The detailed survey questions for the various SPACE dimensions are illustrated in Appendix C.

The SPACE framework was originally conceived for measurement at the team and organizational level. This study adapts it to individual self-report survey responses, consistent with recent empirical work applying SPACE in developer tool research. The Communication & Collaboration dimension in particular needs careful interpretation, as some aspects of collaborative behaviour are less directly observable through individual perception alone.

SPACE Dimension	No. of Items	Scale	Example Item
Satisfaction	7	1–5 Likert	Help manage workload and energy
Performance	4	1–5 Likert	I achieve code standards with less manual effort
Activity	4	1–5 Likert / categorical	Coding assistants enable me to accomplish more tasks during sprints
Communication & Collaboration	2	1–5 Likert	Coding assistants help me comprehend colleagues' code without asking them directly
Efficiency & Flow	5	1–5 Likert / numeric	I have control over when and how I use coding assistants

Table 1: Broader operationalization used for SPACE analysis

Conceptual overlap and analytical separation

Several survey items could be conceptually related to multiple analytical frameworks, in particular, some items not originally designed as SPACE indicators, including items used to operationalize TTF, could plausibly be interpreted as relating to specific SPACE dimensions.

However, for analytical clarity, these conceptual overlaps were not reflected uniformly across analyses. The SPACE framework was used descriptively to summarize perceived productivity across its five dimensions, while TTF was treated as a distinct explanatory construct in the regression models. To avoid analytical circularity, items used to operationalize TTF were therefore not incorporated into the SPACE measures used in the regression analysis, and direct comparisons between SPACE outcomes and regression predictors were avoided. This separation reflects differences in analytical purpose rather than limitations in measurement.

This is indicated in Table 20, Appendix C.

4.3.6 Control Variables

In addition to the primary constructs, several contextual variables were included in the study. This was done to account for differences between participants that could influence perceived productivity outcomes independently of TTF. These variables were included to improve the internal validity of the regression analysis by reducing the risk that observed relationships were explained by background differences rather than by the focal constructs.

Task complexity was included to capture the perceived difficulty of the development tasks respondents typically perform, since more complex work may affect both the usefulness of coding assistants and perceived productivity outcomes.

Experience level was included to reflect respondents' professional background, as more experienced developers may differ in how they evaluate, adopt, or rely on coding assistants.

Utilization was included to measure the frequency of coding assistant use, since productivity effects presuppose some level of actual use.

Among these variables, utilization has a dual role in the analysis. In addition to being treated as a contextual control variable, it is also included as a moderating variable in the regression models to examine whether the relationship between perceived TTF and developer productivity differs depending on how frequently coding assistants are used. This is indicated in Table 21, Appendix C.

4.4 Definition of Task

Within the Task–Technology Fit framework, tasks are typically defined as the activities individuals perform to accomplish their work objectives. In software development these tasks can be very versatile, overlapping, and often interdependent. Developers frequently engage in a range of activities, including coding, debugging, documentation, and information retrieval, within a single workflow.

Therefore, within this study the word “task” is conceptualized at the level of general software development activities rather than narrowly defined micro-tasks. Respondents of the survey were therefore asked to evaluate coding assistants in relation to the development tasks they generally perform in their workflow.

This approach reflects the way GenAI tools are used in practice, where software practitioners interact with such tools, across multiple tasks, rather than in isolated contexts. While this generalization sacrifices task-level precision, it enables a more ecologically valid assessment of perceived TTF.

4.5 Participants and Distribution

The survey was distributed to software practitioners within a single large organization. The participant group included respondents with varying levels of professional experience and a range of technical roles related to software development, such as software design and development, integration and verification, and software architecture. Their work typically involved activities such as requirements analysis, coding, testing, verification, documentation, code review, and fault resolution. In total, 160 respondents completed the survey.

The survey was distributed through internal communication channels, including Teams and email, to reach a broad range of relevant participants within the organization. Participation was entirely voluntary, and informed consent was obtained before respondents completed the survey. To reduce respondent burden and support completion rates, the survey was designed to take approximately 8–10 minutes.

The variation in respondents' roles and experience levels supports the ecological relevance of the study, as it captures perceptions from different parts of the software development process. At the same time, the sample was limited to one organization and participation was voluntary, which means the findings should not be interpreted as statistically representative beyond the studied context.

4.6 Data Analysis

4.6.1 Quantitative Data Analysis

The quantitative data analysis was performed using data collected from the survey. The purpose of the analysis was to examine the relationship between perceived TTF and perceived developer productivity, as well as to understand how this relationship may vary depending on contextual factors. The analysis was conducted in several steps, including data preparation, reliability assessment, and regression analysis.

Preparation of Data

The survey responses were screened for missing data points, and any inconsistencies were removed and excluded from the analysis. All Likert scale items were recoded into numerical values on a 5-point scale ranging from 1 (strongly disagree) to 5 (strongly agree).

Task–Technology Fit

Perceived TTF was measured using three survey items, capturing the extent to which coding assistants support developers in performing their tasks. This operationalization reflects a generalized perception of fit rather than a task-specific evaluation, consistent with the study's focus on overall developer experience.

To ensure internal consistency, reliability analysis was conducted using Cronbach's alpha. Cronbach's alpha is a statistical measure used to verify the internal consistency and reliability of survey instruments by ensuring that a set of items accurately measures a single underlying construct (Tomaz et al., 2026, p. 10).

A threshold of $\alpha \geq 0.70$ was considered acceptable. If internal consistency was deemed acceptable, a composite TTF score was calculated as the mean of the included items.

Developer Productivity

Developer productivity was analysed through the five dimensions of the SPACE framework. For the regression analysis, the original productivity items were retained and combined into dimension-level composite scores. As described in Section 4.4, the

descriptive SPACE analysis used a broader operationalization for descriptive purposes, and the two analyses should therefore not be interpreted as relying on identical dimension definitions. Internal consistency was evaluated using Cronbach's alpha, and composite scores were calculated as the mean of the respective items used in the regression analysis.

Control and Contextual Variables

The regression models also included task complexity, experience level, and utilization of coding assistants as contextual control variables. These variables were incorporated to account for background differences between respondents that could influence perceived productivity outcomes independently of TTF. Utilization was additionally included as a moderating variable to test whether the relationship between TTF and developer productivity varied by frequency of use.

Descriptive Statistics

Descriptive statistics were used to summarize the data, including means, standard deviations, and distributions of the variables. This provided an overview of participants' perceptions of coding assistant usage and general patterns in the data.

Regression Analysis

To assess the relationship between perceived TTF and developer productivity, multiple linear regression analysis was performed. Separate regressions were conducted for each of the SPACE dimensions, acting as dependent variables. This allowed for a more nuanced analysis of how TTF relates to different aspects of developer productivity.

The general format of the regression model was:

SPACE dimension = $\beta_0 + \beta_1$ (TTFcentered) + β_2 (Utilizationcentered) + β_3 (TTF x Utilization) + β_4 (TaskComplexitycentered) + β_5 (Experiencecentered) + ϵ

Explanation of Model Components

TTF_centered (β_1): Effect of perceived TTF on developer productivity when utilization is at its average level.

Utilization_centered (β_2): Effect of coding assistant usage on developer productivity when TTF is at its average level.

TTF x Utilization (β_3): Interaction term assessing whether the effect of TTF on developer productivity depends on the level of utilization.

TaskComplexitycentered (β_4) and Experiencecentered (β_5): Control variables accounting for contextual differences between participants.

Error term (ϵ): Captures unexplained variance.

Centering of Variables

To reduce multicollinearity and improve the interpretability of the interaction term, the TTF and utilization variables were mean-centered before creating the interaction term. By centering these variables, a value of zero represents the average level, allowing the main effects to be interpreted at average levels of the interacting variable.

Purpose of the Interaction Term

Including the interaction term allows the model to capture whether the frequency of coding assistant usage influences how TTF affects developer productivity. A significant interaction effect indicates that the impact of TTF varies depending on the level of utilization.

Interpretation of Results

The models' total explanatory power is measured through the coefficient of determination (R^2), which represent the proportion of variance in the outcome variable which is explained by the predictor variables in the sample (Ozili, 2022, p. 2). Within perception based behavioural studies, the R^2 is expected to be lower than it would be in objective natural science studies. The reason for this is the many things that can affect human behaviour, data collection may be affected by participants' self-interest, group dynamics, feelings and many other factors. opposed to in natural science where the subject of the studies is usually objective and predictable (Ozili, 2022, p. 5). Furthermore, a lower R^2 in perception-based studies may indicate that the dependent variable is influenced by many factors not included in the model. However, this does not necessarily mean that the model is poor, but rather that it does not capture the full complexity of the phenomenon, which is often difficult in behavioural research (Moksony, 1999, pp. 2, 3).

In studies where the main goal is to examine and test a theory, opposed to making exact predictions, it is argued that the size of R^2 is of secondary importance (Moksony, 1999, pp. 2, 3). Within perception based behavioural science an $R^2 > 0,1$ is acceptable as long as the model includes statistically significant predictors (Ozili, 2022, p. 1).

To provide a more accurate representation of the model's fit and generalizability, the adjusted R^2 is also reported. The difference between the two measurements is that the standard R^2 reflects the proportion of variance explained in the specific sample, while the adjusted R^2 serves as an estimate of the explanatory power at the population level (Ozili, 2022, p. 2). A critical limitation of the standard R^2 is that its value tends to increase artificially whenever a new independent variable is added to the model, regardless of whether the variable actually improves the model's explanatory power. The adjusted R^2 addresses this issue by incorporating the model's degrees of freedom into the calculation.

As a result, the value increases only if the contribution of the added variable is sufficiently large to offset the loss of degrees of freedom. If a variable is included that does not significantly improve the model, the adjusted value will instead decrease. By reporting both the standard R^2 and the adjusted R^2 the risk of overestimating the model's explanatory power due to the number of included variables is reduced (Ozili, 2022, p. 4).

To ensure the validity of the model, the assumption of non-collinearity is assessed, meaning that the independent variables should not be excessively correlated with one another. This is examined using the Variance Inflation Factor (VIF) (Ali & Younas, 2021, p. 116; O'Brien, 2007, pp. 673, 674). A VIF value of 1 indicates the absence of correlation, whereas values above 1 reflect increasing levels of collinearity and VIF values exceeding 5 or 10 usually indicate serious cases of multicollinearity (Akinwande et al., 2015, pp. 755, 756).

The primary focus of the analysis is on the interpretation of the individual regression coefficients and their associated p-values (Moksony, 1999, p. 2; Ali & Younas, 2021, p. 116). A regression coefficient represents the estimated change in the dependent variable associated with a one-unit increase in the independent variable. To ensure that the observed relationships are not due to chance, a significance level of $p < 0.05$ is applied (Ali & Younas, 2021, pp. 116, 117).

Particular attention was given to the TTF variable, the utilization variable, and their interaction. The main effects represent their relationship with developer productivity when

the other variable is at its average level. The interaction term indicates whether the effect of TTF depends on how frequently coding assistants are used. A significant interaction suggests that the relationship varies across levels of utilization, while a non-significant interaction indicates a more stable effect.

4.6.2 Qualitative Data Analysis

The qualitative data consisted of free-text responses to open-ended questions included in the survey. These responses were analysed using thematic analysis to identify recurring patterns in how participants described the benefits, limitations, and practical conditions associated with coding assistant use in software development.

The analysis followed an iterative and primarily inductive process. First, all responses were read multiple times to gain familiarity with the material. Meaningful segments of text were then coded based on their substantive content, such as experiences of task support, correctness issues, workflow friction, context limitations, and learning-related concerns. In the next step, related codes were grouped into broader candidate themes representing recurring patterns across participants' responses.

The candidate themes were subsequently reviewed and refined to ensure internal consistency and clear distinction between themes. Theme definitions were then developed to capture the central meaning of each theme in relation to developers' experiences of using coding assistants. The qualitative analysis was conducted independently of the quantitative SPACE scoring, but the resulting themes were later related to the quantitative findings during the mixed-methods integration stage to explain and contextualize the observed patterns.

This approach is consistent with the HCAI perspective of the study, as it captures how developers themselves experience the usefulness, limitations, and consequences of coding assistants in their everyday work.

4.7 Integration of Findings

Both quantitative (Likert-scale) and qualitative (free-text) data were collected through the same survey. The two strands were first analysed separately, using SPACE-based analysis for the quantitative data and thematic analysis for the free-text responses. It was then integrated at the interpretation stage through a joint display in the form of a convergence coding matrix (Guetterman et al., 2015, p. 556). The purpose of which is to collect and analyse both types of data during the same time period to obtain a more comprehensive understanding of the phenomenon (Miller et al., 2013, p. 2137). The SPACE framework was used as the main structure for combining the results because it directly reflects the productivity concepts studied in this thesis.

The integration was conducted using a joint display in the form of a convergence coding matrix (Hands, 2022, p. 3). A joint display serves as a visual tool for integrating data and generating new insights that extend beyond what the individual results can contribute on their own (Guetterman et al., 2015, p. 555; Miller et al., 2013, p. 2143).

The integration process was conducted in three steps, qualitative analysis, qualitative thematic analysis and lastly the integration itself. First, the quantitative results were summarized at the dimensional level within the SPACE framework, including descriptive statistics and reliability tests. Subsequently, the qualitative themes were identified along with their corresponding definitions, subthemes, and representative quotations. The use of

quotations in the integration helps the reader understand the connection between the statistical findings and the participants' perspectives. Finally, the themes were linked to the SPACE dimensions by comparing the core meaning of each theme with the construct definitions of the respective dimensions as described by Hands (2022, pp. 12, 13). This process involves a deductive analysis in which qualitative data are directly compared with specific quantitative items.

As several qualitative themes represent overarching conditions, the matrix reports only the primary themes for each dimension. A theme was classified as primary if it constituted a direct mechanism or boundary condition for the dimension. Themes that were not identified as primary were treated as secondary influences and discussed narratively, consistent with a weaving approach to reporting (Miller et al., 2013, pp. 2142–2150). In cases where the qualitative data did not address a specific SPACE dimension, this was reported as partial silence. Identifying “silence,” or the absence of connections, is important to ensure that the integration is not forced but instead reflects the actual convergence of the data (Hands, 2022, pp. 13–14).

To support transparency, the categorization criteria (primary vs. secondary themes and “silence”) were applied consistently and documented clearly. The mapping was reviewed iteratively by re-examining the wording of the SPACE items against the theme definitions to ensure that the connections reflected the actual constructs rather than superficial similarities. This form of documented and reflective process is essential for maintaining the validity and transparency of the study (Reeping & Edwards, 2022, p. 473).

4.8 Reliability & Validity

Ensuring the reliability and validity of the study was important for establishing the credibility of the findings. Several measures were therefore taken to strengthen the quality of both the survey instrument and the overall research design.

4.8.1 Reliability

Internal consistency of the multi-item constructs was assessed using Cronbach's alpha. Reliability analysis was conducted for the TTF scale and for each SPACE dimension. Because the SPACE framework was used for both descriptive and explanatory purposes in this study, reliability assessments were interpreted in relation to the item composition used for the respective analysis. Accordingly, the descriptive and regression-based SPACE measures should not be understood as relying on identical dimension definitions. A threshold of $\alpha \geq 0.70$ was treated as acceptable for scale-level interpretation, while lower values were interpreted more cautiously considering the exploratory nature of some constructs and the limited number of items in certain dimensions. Where reliability was considered sufficient, composite scores were calculated by averaging the relevant items.

4.8.2 Construct Validity

Construct validity was addressed by designing survey items in alignment with the theoretical frameworks used in the study. The TTF items were intended to capture a generalized perception of how well coding assistants support developers' tasks, while the SPACE items were developed to reflect the five dimensions of developer productivity. At the same time, the study involved an adaptation of both frameworks. These adaptations improved feasibility, but they may also have narrowed the coverage of some constructs. Construct validity was further strengthened through qualitative thematic analysis, which

provided an additional basis for assessing whether the survey measures were consistent with developers' reported experiences.

4.8.3 Internal Validity

Internal validity was strengthened by including contextual control variables in the regression models, specifically task complexity, experience level, and utilization. These variables were included to reduce the risk that observed relationships were explained by background differences rather than by the focal constructs. In addition, mean-centering of variables used in the interaction term was applied to reduce multicollinearity and improve interpretability of the regression coefficients. However, as the study is cross-sectional and non-experimental, causal inferences cannot be made.

4.8.4 External Validity

The study was conducted within a real enterprise software development environment involving respondents with different roles and levels of experience. This supports the ecological relevance of the findings and increases their potential relevance for similar enterprise software development contexts. At the same time, the sample was limited to a single organization and based on voluntary participation, which limits broader generalization.

4.8.5 Common Method Bias

Harman's single-factor test was used to assess the potential for common method bias in the survey data. The test examines whether a single factor accounts for most of the covariance among the measurement items, which would indicate a possible common method bias problem. In this study, the first factor explained 43.9% of the total variance, which is below the commonly used 50% threshold. This suggests that common method bias was not a serious concern.

4.8.6 Limitations of Self-Report

The study relies on perception-based data, which may be influenced by respondent subjectivity, expectations, and other self-report biases. To mitigate this limitation, the study combined structured survey measures with qualitative free-text responses, allowing quantitative patterns to be interpreted considering participants' own descriptions of benefits, constraints, and usage conditions. This integration increases confidence that the findings reflect meaningful developer perceptions, even though they should not be interpreted as objective measures of productivity.

4.8.7 Summary

Overall, the study used multiple strategies to strengthen reliability and validity, including theory-based operationalization, reliability testing, contextual controls in regression analysis, mixed-methods analysis, and a check for common method bias. At the same time, the findings should be interpreted considering the study's cross-sectional design, single-organization sample, and reliance on self-reported data.

4.9 Ethical Considerations

This study was conducted with careful consideration of ethical principles relevant to empirical research in an enterprise software development context. As the study involves developers' perceptions of AI coding assistants, particular attention was given to protecting

participants, safeguarding organizational interests, and ensuring responsible handling of data.

Participation in the survey was voluntary, and respondents were informed about the purpose of the study, the nature of the questions, and how the data would be used. No personally identifiable information was collected, and responses were treated as anonymous, reducing the risk of individual identification. Participants were free to discontinue the survey at any time without consequence.

Given the enterprise setting, special care was taken to avoid the collection of sensitive organizational information, proprietary code, or confidential business data. Survey questions focused on perceptions and experiences rather than on specific systems, products, or internal processes. All collected data were handled in accordance with applicable data protection principles and were used solely for academic purposes.

The study also acknowledges broader ethical concerns associated with the use of GenAI in software development, such as reliability of AI-generated output, verification effort, skill development, and responsibility for code quality. While these issues are not directly caused by participation in the study, they form part of the context in which developers' experiences are examined. From a HCAI perspective, these concerns highlight the importance of maintaining human oversight, accountability, and control when AI tools are used in professional settings.

Overall, the ethical approach of this study emphasizes minimizing potential harm, respecting participant autonomy, and ensuring that both individual developers and the host organization are protected, while contributing knowledge about the responsible use of AI in enterprise software development.

5 Empirical Findings

5.1 SPACE- Quantitative Findings

The quantitative findings from the survey are organized according to the five dimensions of the SPACE framework. In this section, the SPACE dimensions are used descriptively to summarize developers perceived productivity when using coding assistants. As described in Chapter 4, this descriptive SPACE analysis draws on a broader operationalization of the dimensions than the regression analysis, and the results presented here should therefore not be interpreted as based on identical dimension definitions. Descriptive statistics, including means, standard deviations, and distributions, are used to summarize the observed patterns. Where relevant, the results are also presented by tool and contextual variables to support comparison and interpretation in relation to the research questions.

5.1.1 Cronbach's alpha reliability analysis

As an initial step in the quantitative analysis, Cronbach's alpha (α) was calculated for each SPACE dimension to assess internal consistency reliability, as shown in Table 2. The number of items in each dimension is reported as k , and alpha was calculated using complete cases for each scale. Following common guidelines, values of approximately $\alpha \geq 0.70$ were interpreted as acceptable for scale-level reporting, while values in the 0.60–0.70 range were treated as exploratory.

Dimension	Items (k)	N (complete)	Cronbach's α	Changes
Satisfaction	7	160	0.821	Good, original set was kept
Performance	4	160	0.768	Acceptable, original set was kept
Activity (Improved)	3	160	0.644	Replaced tenure with behavioral breadth
Communication & Collaboration	2	160	0.588	2-item scale; reliability is capped
Efficiency & Flow (Improved)	4	160	0.758	Removed misaligned "Revise code" item

Table 2: Reliability using Cronbach's α

Overall, the results indicate that Satisfaction showed good internal consistency, while Performance and the revised Efficiency & Flow scale reached acceptable reliability. The revised Activity scale showed lower but still exploratory reliability, and Communication & Collaboration produced the weakest result. Because the latter dimension was based on only two items, its reliability should be interpreted with caution. Taken together, the results suggest that most dimensions were sufficiently reliable for descriptive interpretation, while some scales required more cautious treatment due to limited item coverage or weaker internal consistency.

5.1.2 SPACE dimension averages

Table 3 presents the mean scores, standard deviations, and respondent counts for each SPACE dimension. Together, these descriptive statistics provide an overall view of how developers perceived the impact of coding assistants across the five dimensions and establish a baseline for subsequent subgroup and regression analyses.

Dimension	Average (1–5)	Std Dev	N (respondents)
Satisfaction	3.91	0.59	148
Performance	3.47	0.69	160
Activity	3.67	0.61	152
Communication & Collaboration	3.35	0.79	160
Efficiency & Flow	3.77	0.58	160

Table 3: SPACE dimension averages, standard deviation and N values

Figure 3 compares the original SPACE scores with revised scores based on reliability-improved item groupings, using the same 1–5 scale. These revised scores were used to support descriptive interpretation in the SPACE analysis, whereas the regression analysis retained the original item-based productivity measures described in Chapter 4. The overall pattern remains largely unchanged, indicating that the descriptive conclusions are robust to the revised operationalization. The revised scoring produces a slight increase in Efficiency & Flow, consistent with the improved internal consistency of that scale, while Activity is somewhat lower under the revised definition, reflecting a narrower and more behaviorally focused measure. The remaining dimensions show only minimal differences between the two scoring approaches.

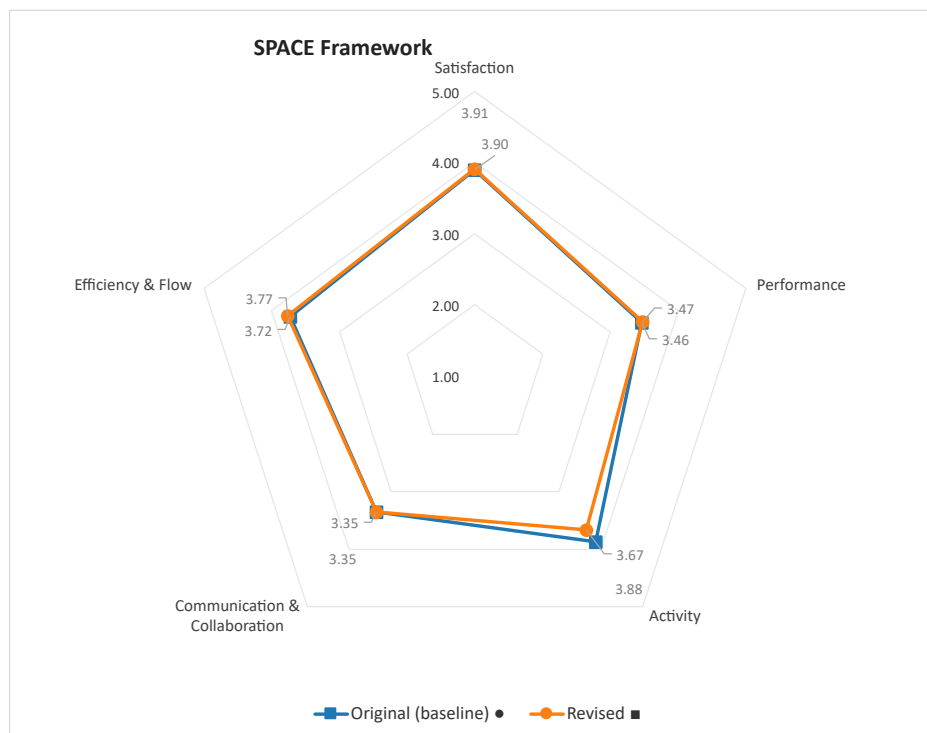


Figure 3: SPACE dimension scores

Overall, respondents reported the most positive outcomes for Satisfaction, followed by Efficiency & Flow and Activity. Performance was more moderate, while Communication & Collaboration had the lowest mean score and the highest standard deviation. This suggests that coding assistants were perceived as more consistently supportive for individual experience and workflow than for collaboration-related aspects of productivity.

5.1.3 SPACE: Satisfaction

Figure 4 presents the item-level mean scores and standard deviations for the Satisfaction dimension. The graph shows a consistently positive overall pattern across all items. Respondents generally rated the coding assistants favorably on fit, usefulness, workflow integration, and overall satisfaction. The chart also suggests that the assistants are seen as more effective at easing repetitive workload and supporting day-to-day work than at reducing the deeper mental effort involved in software development tasks. The spread around some items indicates that experiences were more mixed for cognitive-effort-related aspects, while views on overall usefulness were more consistently positive.

The distribution of Satisfaction scores by tool indicates broadly similar central tendency and spread for Amazon Q and AWS Kiro, suggesting no substantial difference in perceived satisfaction between the two assistants. The detailed plot is provided in Appendix F, Figure 20.

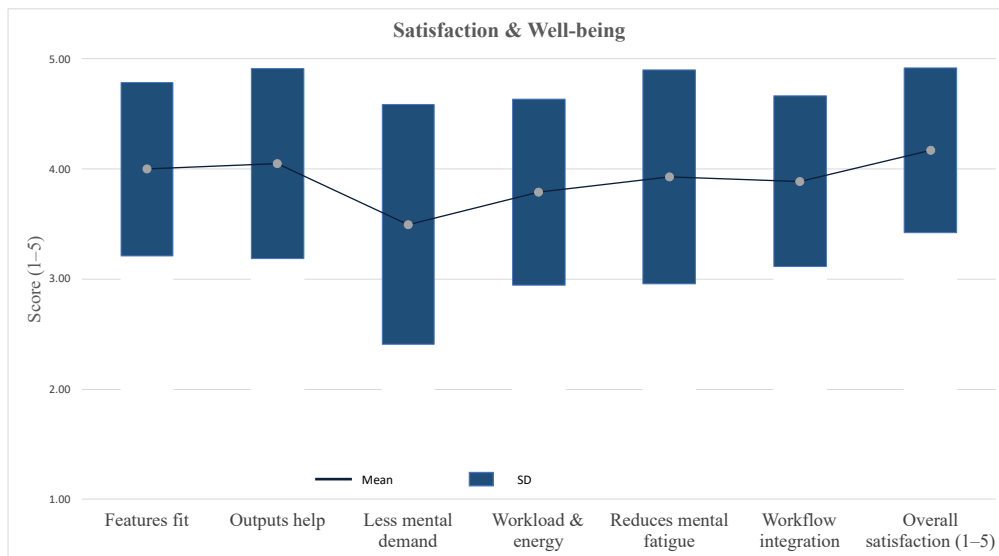


Figure 4: Mean and standard deviation of Satisfaction scores

5.1.4 SPACE: Performance

Figure 5 indicates that Performance was rated moderately positively, indicating that respondents generally perceived coding assistants as supportive for outcome quality and task completion, but less strongly than for satisfaction or flow. This suggests that the tools were seen as particularly helpful for well-defined tasks, while perceived performance gains remained more conditional and varied across contexts.

Appendix F, Figure 21 illustrates the performance comparison between Amazon Q and AWS Kiro. The data suggests that respondents viewed both tools as moderately positive on performance, with Kiro showing a small advantage.

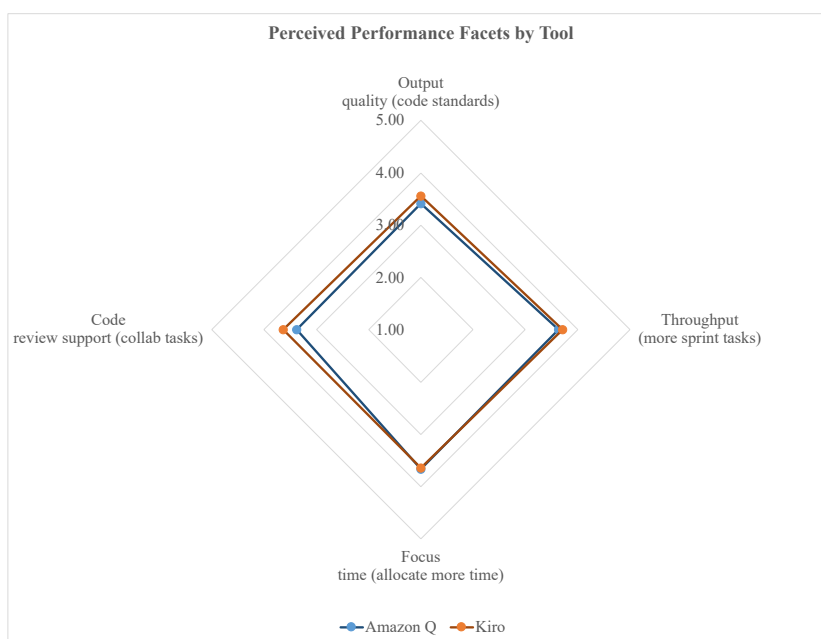


Figure 5: Perceived Performance by Tool

5.1.5 SPACE: Activity

Figure 6 illustrates the frequency with which coding assistants are used for different tasks. The graph shows that coding assistants are used most often for understanding code flow and bug fixing/troubleshooting, making these the most common activity areas overall. Code reviews, feature development, and test case implementation also appear frequently, but at a lower level. In contrast, systemization and other tasks are the least common, suggesting that respondents mainly rely on coding assistants for practical day-to-day development support rather than niche activities.

Figure 22, Appendix F represents the distribution of task categories for which respondents reported using coding assistants, grouped by tool. Overall, AWS Kiro users reported a higher number of task selections than Amazon Q users, suggesting reported use across a somewhat broader range of development activities. AWS Kiro also showed relatively higher reported use in areas such as feature development, code review, documentation, and test implementation as compared to Amazon Q.

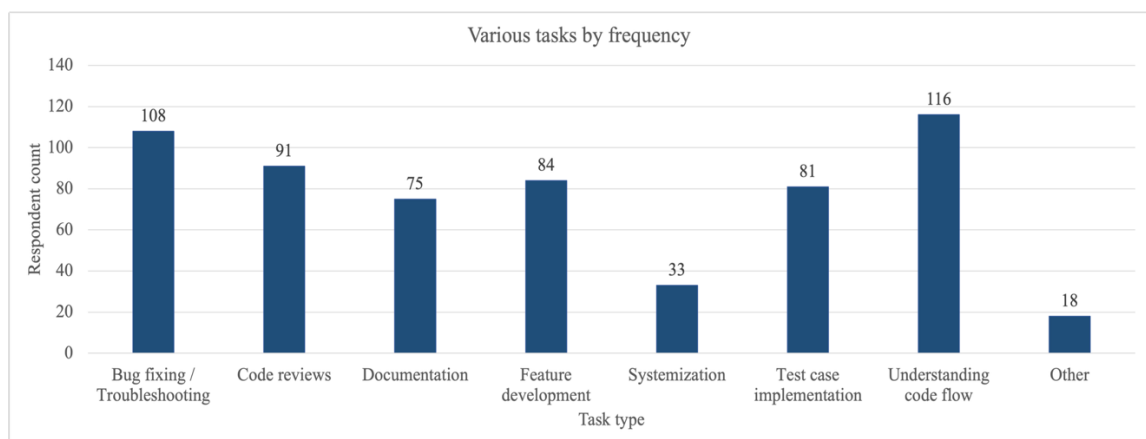


Figure 6: Various software development tasks by frequency

Chi-square tests, which assess whether two categorical variables are associated, were conducted to examine whether the distribution of reported task categories differed by tool and by usage frequency. The association between task category and tool (Appendix F, Table 21) was not statistically significant, indicating that Amazon Q and AWS Kiro users reported broadly similar distributions of task selections. Likewise, the association between task category and usage frequency (Appendix F, Table 22) was not statistically significant, suggesting that more frequent use was not concentrated in any specific task category within this subset.

5.1.6 SPACE: Communication & Collaboration

Communication and Collaboration were the lowest-rated SPACE dimensions and showed the greatest variation among respondents. When broken down by professional role (Figure 7), Software Architects/System Engineers reported the highest mean score, followed by Other roles and Software Designers/Developers, while Testers/Verification Engineers reported the lowest mean. These differences are modest, but they suggest that coding assistants may be perceived as somewhat more supportive in roles involving architecture, systems understanding, and code interpretation.

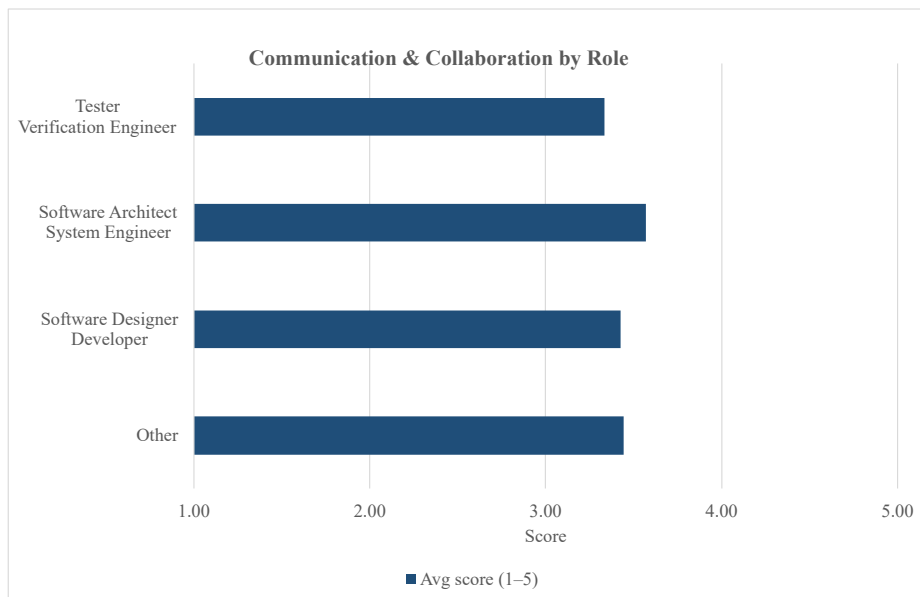


Figure 7: Communication & Collaboration by Role

When compared by tool (Appendix F, Figure 23), the overall Communication and Collaboration score was moderately positive, with only small differences between AWS Kiro and Amazon Q Developer. This indicates broadly similar perceived collaboration-related support across the two assistants. However, this dimension should be interpreted with caution, as it is based on only two items and therefore functions more as a pair of related indicators than a robust multi-item scale.

5.1.7 SPACE: Efficiency & Flow

Efficiency and Flow was rated highly, suggesting that coding assistants generally helped developers work faster and more smoothly. As shown in Figure 8 and Table 4, this reflects time savings, support with routine/repetitive tasks, and reduced rework.

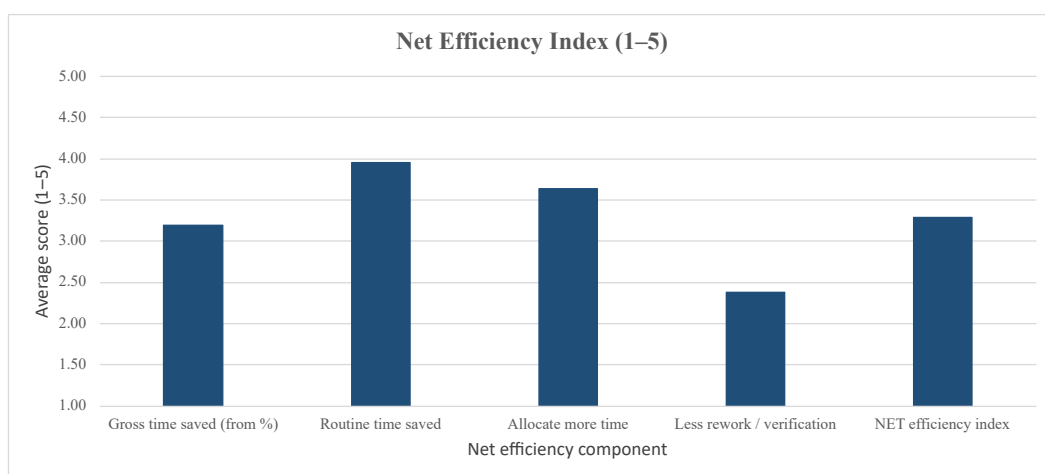


Figure 8: Net efficiency with using coding assistants (in general)

Efficiency parameter	Function
Gross time saved	A scored item reflecting % of work time saved. Higher = more perceived time saved
Routine time saved	A scored item reflecting agreement that the assistant helps save time on routine/repetitive work.
Allocate more time	A scored item reflecting whether the assistant frees time for other responsibilities.
Less rework/verification	Higher = less rework/verification needed (better)
Net efficiency index	Average of Time saved score + Routine time saved + Allocate more time + Rework/verification.

Table 4: Efficiency parameters and their functions

Figure 9 indicates that perceived time savings were lower among developers with more than 11 years of experience, whereas respondents with fewer years of experience reported comparatively greater efficiency gains from coding assistants.

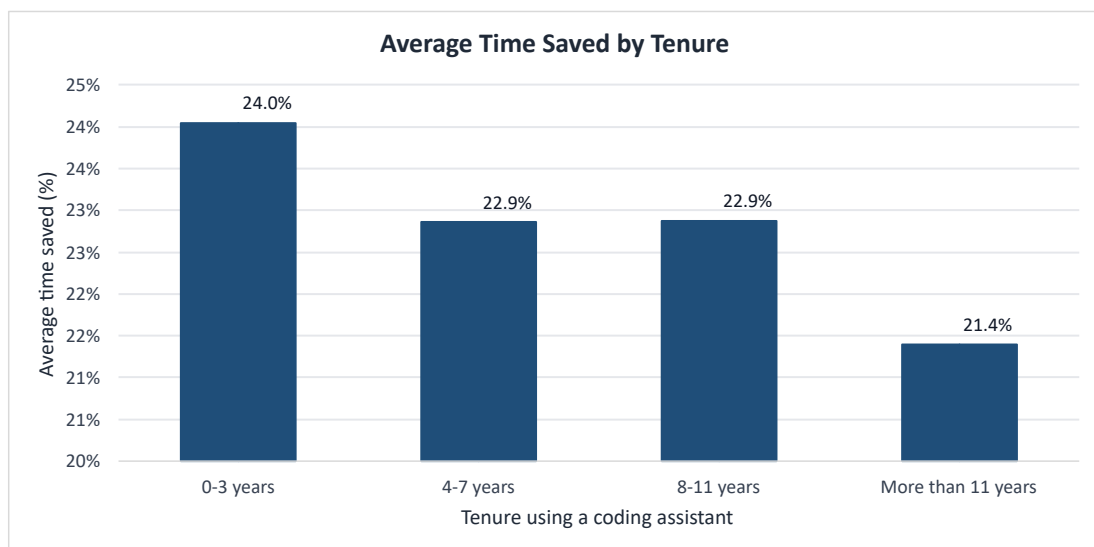


Figure 9: Average time saved based on tenure of developers

Appendix F, Figure 24 illustrates the efficiency comparison between Amazon Q developer and AWS Kiro.

5.2 SPACE- Thematic Analysis of qualitative data

A thematic analysis of the qualitative responses from the survey, is indicated in Table 5.

Theme	Description	Subthemes	Keywords	Example quotations
Context, prompting & integration constraints	Limitations in providing sufficient context and steering the assistant alongside requests for deeper tool/workflow integration	Context & prompting difficulty Integration & tooling requests	context, prompt, inputs, repo, integrate, Jira, Gerrit, GitLab	Limited context size for amazon Q I would like to be able to easier add much more context, from different repos
Reliability & correctness risks	Concerns about incorrect or inconsistent outputs, false assumptions, hallucinations, and consequences such as bug introduction and reduced maintainability	Incorrect/ inconsistent answers Code quality & maintainability	incorrect, hallucinate, assumption, bug, quality, risk, review	Sometimes it tends to hallucinate into believing it fixed the problem because there is "some" output. It's becoming difficult to review a large amount of AI generated code from other team members and myself.
Operational friction	Interruptions and instability that disrupt work (crashes, slowness, errors) and limitations in maintaining context across sessions	Interruptions/ stability & speed	crash, interrupt, stable, speed, slow, error, recover, memory	Memory between sessions is not the best. AWS Kiro is not stable enough, sometimes several times per day.
Perceived productivity & task support	Reported time savings and practical assistance across development tasks, including summarisation/explanations/documentation and support for testing/coverage	Productivity (positive impact) Documentation, Testing / coverage / build help	save, faster, efficient, overview, document, diagram, test, coverage, compile, build	Rewriting 30 lines in 18 different files to use a new function that replaced the old one, without any error, passing all tests I copy pasted the call flow of some variables in the code
Human factors	The effort required to validate AI output and the resulting cognitive load, alongside concerns about over-reliance and weaker long-term learning/skill development.	Cognitive load Over-reliance / learning loss	review, mentally exhausting, alertness, rely, trust, knowledge, learning	Constantly verifying whether solutions produced by a coding assistant are correct can be mentally exhausting and keeps developers in a continuous state of alertness. Due to our lack of in-depth understanding, we retain less than when we research things on our own.

Table 5: Thematic Analysis of qualitative data from the survey

The thematic analysis shows that developers generally perceive coding assistants as useful for a broad range of development tasks, especially code comprehension, debugging, documentation, and testing support. At the same time, the findings highlight recurring constraints related to limited context handling, weak workflow integration, correctness and

reliability risks, operational instability, and increased cognitive burden from verification. Together, these themes suggest that coding assistants provide meaningful support in everyday work, but that their realized value depends on fit with task demands and the surrounding organizational context.

5.2.1 Multiple linear regression analysis

The following regression analysis builds on the original item-based productivity measures described in Chapter 4 rather than the broader descriptive SPACE operationalization used in Section 5.1. Multiple linear regression models were estimated separately for each outcome. All predictors were mean-centred to reduce multicollinearity and to facilitate interpretation of the interaction term. The same predictor set was used across outcomes: TTF, Utilization, their interaction (TTF $\times$ Utilization), Task Complexity, and Experience.

The general model specification was: $DV = \beta_0 + \beta_1(TTF_{centered}) + \beta_2(Utilization_{centered}) + \beta_3(TTF \times Utilization) + \beta_4(TaskComplexity_{centered}) + \beta_5(Experience_{centered}) + \epsilon$, where each SPACE dimension was analysed as the dependent variable in turn. Across outcomes, model fit was modest to moderate ($R^2 \approx 0.23$ – 0.32) and all models were statistically significant overall ($p < 0.001$). Detailed diagnostic statistics and full model-fit information are reported in Appendix F.

5.2.2 Satisfaction

A multiple linear regression model was estimated to examine how TTF, utilisation, task complexity, and experience level, as well as the interaction between TTF and utilisation, were associated with satisfaction. The model explained a modest share of the variance in satisfaction ($R^2 = 0,230$; Adj. $R^2 = 0,205$) and was statistically significant overall ($F = 8,983$, $df = 150$, $n = 156$).

TTF showed a positive and statistically significant association with satisfaction ($\beta = 0,450$, $p < 0,001$; $\beta^* = 0,454$), whereas utilisation was not statistically significant ($\beta = 0,101$, $p = 0,107$). The interaction between TTF and utilisation was also not significant ($\beta = 0,018$, $p = 0,794$), indicating that the relationship between TTF and satisfaction did not vary meaningfully across levels of utilisation in this model. Task complexity ($\beta = -0,035$, $p = 0,610$) and experience level ($\beta = -0,052$, $p = 0,305$) were not statistically significant predictors. Model assumptions were assessed and found acceptable; diagnostic statistics and plots are reported in Appendix F.

Model summary

Metric	Value
R^2	0,230
Adj R-squared	0,205
F-statistic	8,983
df (residual)	150,000
Std. error of y	0,654
n (rows used)	156,000

Table 6: Model summary for satisfaction

Multiple regression table

Predictor	β (Coefficient)	Std. Error	t	p-value (two-tailed)	β^* (Std.)	VIF
Intercept (β_0)	3,868	0,054	71,891	<0,001**		
TTF centered (β_1)	0,450	0,076	5,914	<0,001**	0,454	1,148
Utilization centered (β_2)	0,101	0,062	1,622	0,107	0,132	1,294
TTF $\times$ Utilization (β_3)	0,018	0,070	0,262	0,794	0,019	1,058
Task complexity centered (β_4)	-0,035	0,068	-0,511	0,610	-0,041	1,251
Experience level centered (β_5)	-0,052	0,051	-1,030	0,305	-0,076	1,055

Table 7: Multiple regression table for Satisfaction

5.2.3 Performance

A multiple linear regression model was estimated to examine how TTF, utilisation, task complexity, and experience level, as well as the interaction between TTF and utilisation, were associated with performance. The model explained a moderate share of the variance in performance ($R^2 = 0,312$; Adj. $R^2 = 0,290$) and was statistically significant overall ($F = 13,969$, $df = 154$, $n = 160$).

TTF showed a positive and statistically significant association with performance ($\beta = 0,379$, $p < 0,001$; $\beta^* = 0,316$). Task complexity was also positively associated with performance ($\beta = 0,266$, $p = 0,001$; $\beta^* = 0,260$). Utilisation was not statistically significant ($\beta = 0,134$, $p = 0,070$), and the interaction between TTF and utilisation was not significant ($\beta = -0,024$, $p = 0,734$). Experience level was not statistically significant ($\beta = 0,045$, $p = 0,458$). Model assumptions were assessed and found acceptable; diagnostic statistics and plots are reported in Appendix F.

Model summary

Metric	Value
R^2	0,312
Adj R^2	0,290
F-statistic	13,969
df (residual)	154,000
Std. error of y	0,773
n (rows used)	160,000

Table 8: Model summary for Performance

Multiple linear regression

Predictor	β (Coefficient)	Std. Error	t	p-value (two- tailed)	β^* (Std.)	VIF
Intercept (β_0)	3,475	0,063	55,130	<0,001**		
TTF centered (β_1)	0,379	0,088	4,330	<0,001**	0,316	1,193
Utilization centered (β_2)	0,134	0,073	1,822	0,070	0,144	1,392
TTF $\times$ Utilization (β_3)	-0,024	0,070	-0,340	0,734	-0,024	1,122
Task complexity centered (β_4)	0,266	0,078	3,407	0,001**	0,260	1,300
Experience level centered (β_5)	0,045	0,060	0,745	0,458	0,051	1,053

Table 9: Multiple linear regression for performance

5.2.4 Activity

A multiple linear regression model was estimated to examine how TTF, utilisation, task complexity, and experience level, as well as the interaction between TTF and utilisation, were associated with activity. The model explained a moderate share of the variance in activity ($R^2 = 0,320$; Adj. $R^2 = 0,297$) and was statistically significant overall ($F = 14,463$, $df = 154$, $n = 160$).

TTF showed a positive and statistically significant association with activity ($\beta = 0,466$, $p < 0,001$; $\beta^* = 0,472$). Utilisation was not statistically significant ($\beta = 0,082$, $p = 0,174$), and the interaction between TTF and utilisation was not significant ($\beta = -0,057$, $p = 0,318$), suggesting no evidence of moderation of the TTF-activity relationship by utilisation. Task complexity was not statistically significant ($\beta = 0,076$, $p = 0,239$). Experience level showed a small negative association with activity and was statistically significant ($\beta = -0,100$, $p = 0,044$; $\beta^* = -0,138$). Model assumptions were assessed and found acceptable; diagnostic statistics and plots are reported in Appendix F.

Model summary

Metric	Value
R-squared	0,320
Adj R-squared	0,297
F-statistic	14,463
df (residual)	154,000
Std. error of y	0,634
n (rows used)	160,000

Table 10: Model summary for activity

Multiple linear regression

Predictor	β (Coefficient)	Std. Error	t	p-value (two- tailed)	β^* (Std.)	VIF
Intercept (β_0)	3,656	0,052	70,699	<0,001**		
TTF centered (β_1)	0,466	0,072	6,494	<0,001**	0,472	1,193
Utilization centered (β_2)	0,082	0,060	1,367	0,174	0,107	1,392
TTF $\times$ Utilization (β_3)	-0,057	0,057	-1,002	0,318	-0,071	1,122
Task complexity centered (β_4)	0,076	0,064	1,181	0,239	0,090	1,300
Experience level centered (β_5)	-0,100	0,049	-2,030	0,044*	-0,138	1,053

Table 11: Multiple linear regression for Activity

5.2.5 Communication and collaboration

A multiple linear regression model was estimated to examine how TTF, utilisation, task complexity, and experience level, as well as the interaction between TTF and utilisation, were associated with communication and collaboration. The model explained a modest share of the variance in communication and collaboration ($R^2 = 0,249$; Adj. $R^2 = 0,225$) and was statistically significant overall ($F = 10,239$, $df = 154$, $n = 160$).

TTF showed a positive and statistically significant association with communication and collaboration ($\beta = 0,324$, $p < 0,001$; $\beta^* = 0,283$). Task complexity was also positively associated with communication and collaboration ($\beta = 0,223$, $p = 0,005$; $\beta^* = 0,228$). Utilisation was not statistically significant ($\beta = 0,121$, $p = 0,098$), and the interaction between TTF and utilisation was not significant ($\beta = 0,001$, $p = 0,984$). Experience level was not statistically significant ($\beta = 0,054$, $p = 0,372$). Model assumptions were assessed and found acceptable; diagnostic statistics and plots are reported in Appendix F.

Model summary

Metric	Value
R-squared	0,249
Adj R-squared	0,225
F-statistic	10,239
df (residual)	154,000
Std. error of y	0,771
n (rows used)	160,000

Table 12: Model summary for communication and collaboration

Multiple linear regression

Predictor	β (Coefficient)	Std. Error	t	p-value (two- tailed)	β^* (Std.)	VIF
Intercept (β_0)	3,613	0,063	57,489	<0,001**		
TTF centered (β_1)	0,324	0,087	3,713	<0,001**	0,283	1,193
Utilization centered (β_2)	0,121	0,073	1,662	0,098	0,137	1,392
TTF $\times$ Utilization (β_3)	0,001	0,070	0,020	0,984	0,001	1,122
Task complexity centered (β_4)	0,223	0,078	2,859	0,005*	0,228	1,300
Experience level centered (β_5)	0,054	0,060	0,896	0,372	0,064	1,053

Table 13: Multiple linear regression for communication and collaboration

5.2.6 Efficiency and flow

A multiple linear regression model was estimated to examine how TTF, utilisation, task complexity, and experience level, as well as the interaction between TTF and utilisation, were associated with efficiency and flow. The model explained a modest share of the variance in efficiency and flow ($R^2 = 0,275$; Adj. $R^2 = 0,252$) and was statistically significant overall ($F = 11,710$, $df = 154$, $n = 160$).

TTF showed a positive and statistically significant association with efficiency and flow ($\beta = 0,318$, $p < 0,001$; $\beta^* = 0,406$). Task complexity was also positively associated with efficiency and flow ($\beta = 0,125$, $p = 0,018$; $\beta^* = 0,187$). Utilisation was not statistically significant ($\beta = 0,051$, $p = 0,302$), and the interaction between TTF and utilisation was not significant ($\beta = 0,014$, $p = 0,762$). Experience level was not statistically significant ($\beta = -0,025$, $p = 0,533$). Model assumptions were assessed and found acceptable; diagnostic statistics and plots are reported in Appendix F.

Model summary

Metric	Value
R^2	0,275
Adj R-squared	0,252
F-statistic	11,710
df (residual)	154,000
Std. error of y	0,518
n (rows used)	160,000

Table 14: Model summary for efficiency and flow

Multiple linear regression

Predictor	β (Coefficient)	Std. Error	t	p-value (two- tailed)	β^* (Std.)	VIF
Intercept (β_0)	3,808	0,042	90,192	<0,001**		
TTF centered (β_1)	0,318	0,059	5,418	<0,001**	0,406	1,193
Utilization centered (β_2)	0,051	0,049	1,035	0,302	0,084	1,392
TTF $\times$ Utilization (β_3)	0,014	0,047	0,304	0,762	0,022	1,122
Task complexity centered (β_4)	0,125	0,052	2,397	0,018*	0,187	1,300
Experience level centered (β_5)	-0,025	0,040	-0,624	0,533	-0,044	1,053

Table 15: Multiple linear regression for efficiency and flow

5.2.7 Summary

Overall, the results indicate that TTF is the most consistent predictor across the SPACE dimensions, showing a positive and statistically significant association with satisfaction, productivity, activity, communication and collaboration, and efficiency and flow. In contrast, the interaction term (TTF $\times$ utilisation) was not statistically significant in any model, providing no evidence that utilisation moderates the relationship between TTF and the outcomes. Task complexity was positively associated with productivity, communication and collaboration, and efficiency and flow, whereas experience level was only statistically significant for activity (negative association). Utilisation was generally not statistically significant in the main models.

To assess robustness, sensitivity analyses were conducted by re-estimating each regression after excluding observations with extreme residuals (standardised residual > 3). Across outcomes, the substantive conclusions were unchanged: TTF remained statistically significant and positive, and the interaction term remained non-significant. The corresponding robustness tables are reported in Appendix G. For the satisfaction model, one influential observation was examined in more detail; removing this observation did not alter the conclusion regarding TTF, although utilisation became statistically significant in the reduced sample.

5.3 Mixed-methods integration

This section integrates the quantitative SPACE results with the qualitative thematic analysis (free-text responses) collected within the same survey. The two strands were analysed separately and then interpreted jointly, using the SPACE dimensions as the organising frame. The purpose of the integration is to identify where findings converge, complement each other, diverge, or remain silent. The detailed integration procedure and linking rules are described in Table 16. Here, the joint display summarises the primary qualitative themes linked to each SPACE dimension and the resulting integrated interpretation.

SPACE dimension	Primary qualitative themes	Integration outcome	Integrated interpretation
Satisfaction (M=3.91; highest) Consistently positive across items (~3.8–4.2).	Perceived productivity & task support; Operational friction; Human factors.	Convergence + complementarity	High satisfaction aligns with frequent reports of practical task support and time savings. The qualitative themes also explain why satisfaction is not uniformly high for everyone, instability, latency and context limitations can reduce perceived usefulness. Sustained verification effort can increase perceived workload and mental demand, which may help explain why satisfaction is high overall but still varies between respondents.
Performance (M=3.47; moderate) Positive but weaker than Satisfaction and Efficiency & Flow.	Perceived productivity & task support; Reliability & correctness risks; Context, prompting & integration constraints.	Complementarity (boundary conditions)	Performance seems to improve the most when the task is well defined (for example refactoring, writing documentation/explanations, generating test cases, or troubleshooting). However, these gains are often limited by correctness problems and by the difficulty of giving the assistant enough context, which can reduce the net productivity for more complex work.
Activity (M=3.67; relatively high) Broad use across tasks; tool comparison shows similar task-type patterns, with higher volume for AWS Kiro	Perceived productivity & task support; Context, prompting & integration constraints.	Convergence + complementarity	The quantitative results show that developers use the assistants across many parts of the SDLC. The free-text answers help explain why, since people describe support with understanding code, debugging, documentation, and testing. At the same time, missing workflow integration and tool instability are described as practical reasons why usage may not scale to larger or more context-heavy tasks.
Communication and collaboration (M=3.35; lowest; highest variability) Two-item indicator; interpret cautiously.	Reliability & correctness risks; Context, prompting & integration constraints.	Partial coverage (complementary / some silence)	The free-text answers focus more on how the assistants support individual work than on explicit team collaboration. Still, two factors may help explain why Communication & Collaboration is the lowest and most variable dimension: AI-generated code can increase review effort and coordination costs, and limited integration with shared tools (such as issue trackers and code review systems) can make collaborative workflows harder to support. Since collaboration is not discussed as much in the qualitative data, this should be treated as a limitation.
Efficiency and flow Flow (M=3.77; second highest) Positive overall; revised scale slightly higher after removing misaligned “revise code” item.	Perceived productivity & task support; Reliability & correctness risks; Operational friction.	Complementarity (explains variance)	Overall, Efficiency & Flow scores are positive. Free-text responses show that the net effect depends on how much verification is needed, how correct the output is, and how stable the tool is. If the assistant produces wrong or low-quality code, it creates rework, and having to constantly check the output can increase the cognitive load. This helps explain why “less mental demand” is not rated as strongly, and why the responses vary even though the average is high. In addition, limited context and weak integration can interrupt flow when tasks require a lot of prompting or information from multiple code repositories.

Table 16: Detailed integration procedure

5.3.1 Integrated findings by SPACE dimension

Satisfaction

Satisfaction was the highest-rated dimension, indicating that respondents generally value the coding assistants and perceive them as supportive in their daily work. This converges with the qualitative theme Perceived productivity & task support, where respondents describe concrete benefits such as faster refactoring, explanations, and support in troubleshooting. At the same time, the theme of Operational friction and Context, prompting & integration constraints provide complementary explanations for why satisfaction is not uniformly high. One explanation is that instability and limited session/context handling can reduce the perceived usefulness in more complex or longer-running tasks. Finally, the theme of Human factors (verification effort and continuous alertness) help interpret why “less mental demand” is the lowest rated satisfaction-related item, despite overall positive satisfaction.

Performance

Performance shows a moderately positive pattern, suggesting that coding assistants tend to support participants in increasing their work output. The qualitative findings clarify that performance gains are often realized in well-defined tasks (for example creating tests, rewriting code patterns, generating explanations or diagrams, and helping interpret unfamiliar formats such as crash dumps). However, Reliability & correctness risks introduce an important boundary condition; incorrect or inconsistent outputs can reduce net performance, when users must debug AI-introduced issues or invest time validating solutions. In addition, the theme of Context, prompting & integration constraints suggests that when users cannot provide enough context, the assistant is less effective for larger or cross-repository tasks, which can weaken perceived performance.

Activity

Activity is relatively high, and the task breakdown shows that coding assistants are used across a broad range of development activities. This aligns with the theme “Perceived productivity & task support”, which highlights their use for code comprehension, debugging, documentation and explanation, as well as testing and build support. The theme “Context, prompting & integration constraints” helps explain why this activity does not always scale further: respondents describe a need for easier ways to provide extensive context and better integration with common workflow tools, such as issue tracking and code review systems. At the same time, instability, latency, and weak session memory are identified as practical barriers that make some users less likely to use the assistants consistently across tasks, even when overall activity remains high.

Communication and Collaboration

Communication & Collaboration is the lowest-rated dimension and shows the highest variability. Given that it is based on only two items, it should mainly be read as two indicators rather than a full scale. The qualitative themes include less direct evidence about collaboration outcomes (relative to individual task support), which should be treated as partial silence in the integration. The free-text answers still point to two plausible reasons for the lower collaboration scores. AI-generated code can increase review effort and coordination when it is incorrect or produces large changes. In addition, limited integration with shared tools (issue tracking and code review) can reduce how well the coding assistants support team-based workflows.

Efficiency and Flow

Efficiency & Flow is the second-highest dimension, indicating that respondents often experience time savings and smoother work when using coding assistants. This converges with the theme of Perceived productivity & task support, where respondents describe faster completion of routine or repetitive work and support across the SDLC. At the same time, the themes of Reliability & Correctness risks and Human factors help explain why the experience is not uniformly positive. Incorrect outputs can create rework, and continuous validation can be mentally demanding. Operational friction (instability and limited continuity between sessions) can also interrupt flow. Overall, this suggests that net efficiency depends on task scope, output quality, and how much context the user can provide.

Summary of integrated findings

Overall, the findings suggest that developers generally perceive coding assistants positively and experience them as useful in everyday development work. This is reflected in the high scores for Satisfaction and Efficiency & Flow. The strongest reported benefits relate to well-defined tasks, such as refactoring, troubleshooting, writing documentation, and generating test cases. At the same time, the free-text responses highlight three recurring limitations: correctness and maintainability issues that increase rework and review effort, operational friction such as instability, latency, and weak session continuity, and limited context and workflow integration, which reduce usefulness for complex or cross-repository tasks. Taken together, these limitations help explain why Performance is more moderate and why outcomes for Communication & Collaboration are more mixed.

6 Discussion

This chapter interprets the empirical findings in relation to the research questions and theoretical framework, focusing on developers perceived productivity across the SPACE dimensions, the explanatory role of TTF, and the organizational conditions shaping effective use. The discussion adopts a HCAI perspective, emphasizing that value emerges when automation enhances capability while preserving human control over quality, security, and decision-making.

6.1 Overview and answers to research questions

This section provides an overview of the findings by addressing each research question in turn. The aim is to synthesize the key results at a high level and clarify how they help answer the research questions. More in-depth interpretations of perceived productivity across the SPACE dimensions and the role of TTF are presented in Sections 6.2 and 6.3.

RQ1: How do software developers perceive the impact of AI coding assistants across the five SPACE dimensions of productivity?

The findings indicate that software developers generally perceived AI coding assistants as having a positive impact on their work. However, this impact is not uniform across the five SPACE dimensions. Perceived effects are strongest for Satisfaction and Efficiency & Flow, followed by Activity, while Performance is more moderate and Communication & Collaboration is the weakest and most variable dimension.

This pattern suggests that, in this sample, developers primarily perceived coding assistants as supporting their individual, day-to-day work, e.g. by reducing friction in routine tasks and helping work progress more smoothly. In contrast, perceived effects are less consistent

for dimensions that depend more strongly on outcome quality (Performance) and team-level coordination (Communication & Collaboration). In other words, perceived benefits are most pronounced in dimensions related to personal experience and workflow, while effects on outcome-oriented and team-dependent dimensions vary more across respondents.

These dimension-specific patterns are examined in detail in Section 6.2 using the SPACE framework, which highlights how perceived productivity outcomes differ across dimensions rather than forming a single, uniform measure of ‘improved productivity’.

Finding: Within the context of this study, the value of AI coding assistants is realized most clearly in developers’ individual workflow and experience, while benefits for quality and collaboration appear more dependent on organizational and contextual conditions.

RQ2: To what extent does Task–Technology Fit (TTF) predict developer-perceived productivity across the SPACE dimensions?

The results indicate that perceived Task–Technology Fit was positively associated with developer-perceived productivity across all five SPACE dimensions. TTF is positively and statistically significantly associated with Satisfaction ($\beta = 0.450$, $p < 0.001$), Performance ($\beta = 0.379$, $p < 0.001$), Activity ($\beta = 0.466$, $p < 0.001$), Communication & Collaboration ($\beta = 0.324$, $p < 0.001$), and Efficiency & Flow ($\beta = 0.318$, $p < 0.001$).

In contrast, neither utilization nor the interaction between utilization and TTF was statistically significant in any of the models. This suggests that, in this sample, perceived alignment between tool capabilities and task demands was more consistently associated with perceived productivity than frequency of use. In other words, in this sample, better perceived productivity outcomes were more consistently associated with how well the coding assistant fit developers’ tasks than with how often it was used.

The explanatory role of perceived TTF and its implications for understanding productivity differences are examined in more depth in Section 6.3.

Finding: Perceived productivity is more strongly associated with how well the coding assistant fits developers’ tasks than with how often it is used.

RQ3: What organizational factors influence how software developers use AI coding assistants and realize value from them in their daily work?

The findings suggest that, in this sample, the adoption and effective use of AI coding assistants were influenced by organizational and contextual conditions that go beyond tool availability. In particular, the findings suggest that effective use was more likely when the surrounding socio-technical environment enabled assistants to be used in a context-rich, well-integrated, and operationally reliable manner.

The results further suggest that organizational practices related to verification, governance, and learning influenced how developers in this sample experienced and worked with coding assistants. Rather than directly determining perceived productivity outcomes, these organizational factors appear to shape how developers experience task-technology fit in

their everyday work. A more detailed discussion of these organizational factors is provided in Section 6.4.

Finding: The value of AI coding assistants appears to depend in part on whether the organization enables their effective use by ensuring proper governance and ways of working.

6.2 Interpreting perceived productivity through the SPACE dimensions

This section interprets the findings through the five SPACE dimensions. The pattern across dimensions is not uniform; respondents reported the strongest outcomes for Satisfaction and Efficiency & Flow, more moderate outcomes for Activity and Performance, and the weakest and most variable outcomes for Communication & Collaboration. A core point of the SPACE framework is that productivity is multidimensional and that gains in one area can coexist with costs or stagnation in others; therefore, each dimension is discussed on its own terms rather than treated as a single “overall productivity” outcome (Forsgren et al., 2021).

From a cross-dimensional perspective, this pattern suggests that, in this sample, coding assistants were perceived as providing stronger support for individual workflow and task execution than for outcome-oriented or team-dependent aspects of productivity. In particular, dimensions such as Satisfaction and Efficiency & Flow appeared to benefit more directly from assistive support in day-to-day development work, whereas Communication & Collaboration appeared to depend more strongly on shared context, coordination, and workflow integration. This distinction helps frame the dimension-specific discussions that follow.

6.2.1 Satisfaction and well-being

Satisfaction was the highest-rated dimension, suggesting that many developers in this sample perceived coding assistants as helpful and supportive in day-to-day work rather than as disruptive tools. The qualitative responses suggest that satisfaction was closely tied to practical task support, for example, help with refactoring, explanations, summarization of complex logic, and troubleshooting. These experiences suggest that satisfaction was linked not only to speed, but also to reduced friction in cognitively demanding work (Mendes et al., 2024).

At the same time, satisfaction-related items were not uniformly strong, particularly those related to reduced mental demand. Respondents described continuous verification and the need to stay alert as mentally demanding. From a Human-Centered AI perspective, this is consistent with the idea that the value of automation depends on whether developers can maintain control and confidence while using the tool. When confidence in correctness is reduced, the perceived benefit may be partially offset by the burden of validation (Shneiderman, 2022).

6.2.2 Performance

Performance was rated positively but more moderately than satisfaction and flow, suggesting that developers in this sample did not automatically equate AI assistance with

improved work outcomes. In SPACE terms, performance reflects outcome quality and impact, not simply producing something faster (Forsgren et al., 2021).

The qualitative data suggests that perceived performance gains were most credible for well-defined and relatively local tasks (e.g. refactoring known patterns, test scaffolding, documentation/explanations, and targeted troubleshooting). However, performance benefits appeared to weaken when tasks become context-heavy or constrained by architectural or organizational conventions. In those cases, respondents reported hallucinations, incorrect assumptions, and outputs that required debugging, rework, or careful review. This aligns with the broader trade off argument in GenAI productivity literature, that speed or output can increase while effort shifts into validation and review, limiting net performance improvements (Afroz et al., 2026).

6.2.3 Activity

Activity was relatively high, suggesting relatively broad reported use across tasks and frequent incorporation of coding assistants into work. The task breakdown shows concentration around comprehension and troubleshooting, with additional use for documentation, testing, and refactoring. This is consistent with established software engineering research showing that a large share of developers' time is spent on program comprehension and information seeking rather than writing new code (Meyer et al., 2021, p. 864; Minelli et al., 2015, pp. 30–32).

However, Activity should not be interpreted as a direct indicator of value. The qualitative findings suggest that broad usage was constrained by practical issues such as instability, latency, and difficulty providing sufficient context for complex work. This highlights that Activity reflects use across tasks, while outcomes are captured elsewhere in the SPACE framework (Forsgren et al., 2021).

6.2.4 Communication and collaboration

Communication & Collaboration was the lowest and most variable dimension. This suggests that, in this sample, coding assistants were perceived as less consistently supportive for team coordination than for individual task execution. This result should be interpreted cautiously, since the construct is measured with only two items.

Most qualitative comments center on individual work, while collaboration-related effects are mentioned far less often. Where collaboration-related effects are discussed, two mechanisms appear plausible. First, AI-generated code can increase review and coordination effort when outputs are incorrect or involve larger sets of code changes. Second, limited integration with shared workflow tools (such as issue trackers and code review systems) may reduce the assistant's contribution to shared context and traceability. These points align with the idea that AI assistance can strengthen individual productivity experiences without necessarily improving, and sometimes even increasing, team-level coordination demands (Afroz et al., 2026).

6.2.5 Efficiency and Flow

Efficiency & Flow was the second-highest dimension, suggesting that many respondents in this sample perceived time savings and smoother progress when using assistants, particularly for routine or repetitive work. Flow in this context is not simply speed; it reflects continuous progress with minimal disruption (Forsgren et al., 2021).

The qualitative themes suggest that flow was sensitive to two main factors: operational friction (e.g., crashes, latency, weak session continuity) and output reliability. When suggestions were usable and the tool behaved predictably, respondents described fewer interruptions and smoother progress. Uncertain output quality or unstable tool behavior could disrupt flow. Any initial time savings may then be offset by additional verification and review effort (Afroz et al., 2026).

6.3 Task–Technology Fit as an explanatory mechanism

Building on the answer to RQ2, this section moves beyond the empirical results to examine why perceived Task-Technology Fit was associated with SPACE outcomes, while utilization was not, by relating the findings to core assumptions and limitations of the TTF theory.

Across all regression models, perceived TTF was the most consistent explanatory factor for developer perceived productivity outcomes, while utilization and the interaction between TTF and utilization were not statistically significant. This pattern is not unexpected but instead aligns closely with the original logic of Task–Technology Fit. Goodhue and Thompson (1995) argue that information technology contributes to performance only when two conditions are met: the technology must be used, and it must provide functionality that fits the tasks being performed. Utilization is therefore a necessary precondition for impact, but it is not sufficient to explain performance differences once fit is accounted for.

As discussed in the literature review, TTF has primarily been shown to explain variation in perceived performance outcomes rather than technology use, and a good fit does not necessarily translate into higher utilization (Marikyan & Papagiannidis, 2023). From this perspective, the non-significant utilization effects observed in this study are theoretically coherent. This suggests that frequency of use alone is a weak indicator of productivity gains when the alignment between tool capabilities and task requirements varies across users and contexts. Developers may therefore use coding assistants frequently without seeing clear productivity benefits if the tools do not sufficiently support their work.

The qualitative findings help clarify what “fit” represents in the context of enterprise software development supported by GenAI. In line with Dishaw and Strong’s (1998) argument that TTF can be applied at the level of a portfolio of tasks rather than isolated activities, fit in this study appears to be shaped by several recurring conditions. These include the assistant’s ability to access and apply sufficient task relevant context, its reliability and correctness relative to enterprise quality standards, and the degree of integration with existing workflows such as code review and issue tracking systems. Operational factors, including instability, latency, and weak session continuity, further influence whether the assistant is perceived as supportive or disruptive in everyday work. Together, these factors describe concrete mechanisms through which developers evaluate whether a coding assistant “fits” their work, consistent with the core TTF definition.

A further interpretation of the non-significant utilization effects relates to the characteristics of the study sample and how experienced users extract value from development tools. The participants are professional software developers working in an enterprise environment, and it is plausible that many already possess the technical competence needed to use such tools effectively with limited prior usage. From a TTF perspective, this suggests that productivity gains may arise from selective, targeted use of the assistant for tasks where fit is high, rather than from extensive or continuous use across all activities. In this sense,

using the tool more often does not necessarily result in additional benefits beyond those already achieved for well-defined tasks.

This interpretation also helps explain why increased utilization may not be universally beneficial and could, under certain conditions, introduce additional costs. The qualitative themes highlight sustained verification effort, cognitive burden, and coordination challenges associated with AI generated output. If developers already use coding assistants at a level sufficient to capture their main benefits, further increases in use may amplify these costs rather than improve outcomes. This aligns with concerns in the literature on “spurious productivity,” where apparent gains from faster or more extensive AI use are offset by downstream rework, validation, and review effort (Afroz et al., 2026). Seen through this lens, utilization can act as a multiplier of both benefits and drawbacks, depending on the degree of TTF.

Finally, TTF also provides a useful lens for interpreting why SPACE dimensions move differently rather than uniformly. Where coding assistants fit common, well specified, and locally scoped tasks, such as comprehension, troubleshooting, refactoring patterns, documentation, or test scaffolding, developers are more likely to report gains in Efficiency & Flow and Satisfaction. In contrast, tasks that are heavily context-dependent, cross-repository, or tightly constrained by organizational conventions may show weaker fit, shifting effort toward verification and coordination. This helps explain why Communication & Collaboration outcomes are weaker and more variable than other dimensions, since collaborative work depends strongly on shared context, traceability, and workflow integration, which current assistants only partially support.

Overall, the findings indicate that perceived TTF plays a key role in explaining variation in productivity outcomes across the SPACE dimensions. Rather than suggesting that productivity gains increase simply with greater use, the results point to a more conditional pattern. Coding assistants are perceived as beneficial when they align with developers’ tasks and existing ways of working, without creating excessive verification or coordination effort. This reinforces the relevance of TTF theory for interpreting different outcomes of GenAI supported work and complements the SPACE framework by explaining why productivity impacts differ across dimensions, users, and contexts.

6.4 Organizational factors for effective use and realized value

The thematic and integrated findings suggest that realizing value from coding assistants is not simply a matter of tool availability. Effective use depends on whether the surrounding socio-technical environment enables coding assistants to be used in a trustworthy, context-rich, and well-integrated way. In practice, this means that perceived productivity outcomes are shaped by both tool capabilities and the organizational conditions that influence how well the assistant can be embedded into everyday development work.

More specifically, organizational conditions influence how perceived task-technology fit is reflected in reported productivity outcomes. Constraints related to context availability, workflow integration, and reliability affect whether a coding assistant can be applied effectively beyond well-defined, local tasks. When these conditions are misaligned, developers reported increased verification effort, higher cognitive burden, and reduced confidence in outputs, which may limit perceived productivity benefits. These conditions

therefore appear to act as boundary factors that influence whether perceived TTF is associated with positive productivity experiences in developers' everyday work.

Context, prompting & integration constraints

First, "Context, prompting & integration constraints" highlights the importance of enabling developers to provide and retrieve task-relevant context efficiently. Developers reported difficulties in providing sufficient task-relevant context to the assistant, especially when working in large codebases, across multiple repositories, or within workflows where tool integration is limited. This suggests that even a technically capable coding assistant may provide limited value when it lacks access to, or cannot effectively use, the contextual information required for the task. Requests for cross-repository context, stronger indexing, and integration with engineering systems (such as issue tracking and code review tools) suggest that perceived task-technology fit was influenced not only by the assistant itself, but also by how well it was integrated into the organization's existing development toolchain. Without smooth context loading and workflow integration, assistants may remain helpful for local tasks but less effective for complex or end-to-end work.

Reliability & correctness risks

Second, "Reliability & correctness risks" highlights that output quality and correctness influence effective use. Respondents mention hallucinations, incorrect assumptions, inconsistent answers, and the risk of introducing bugs or maintainability issues. These concerns increase the need for review and verification, which can reduce the apparent productivity gains. When developers must repeatedly verify or correct outputs, the overall value of the tool may become less predictable. Trust, in this sense, is not merely a psychological factor but also appears to be a practical condition for sustained value. The findings therefore suggest that organizations may benefit from treating code quality and security verification as an integral part of AI-assisted development rather than as an afterthought. Governance mechanisms (such as secure usage guidelines, code review norms, and potentially automated checks) can help prevent short-term speed from creating long-term quality and maintainability costs. From an HCAI lens, these controls are not barriers to productivity but may be understood as prerequisites for keeping human oversight and accountability intact.

Operational friction

Third, "Operational friction (stability, latency, session continuity)" suggests that the reliability of the tool experience itself was an important factor in reported productivity outcomes. Instability, latency, poor session continuity, and weak memory across sessions were reported to interrupt workflow and reduce the tool's contribution to flow. This suggests that perceived productivity was influenced not only by what a tool could do in principle, but also by how smoothly and reliably it operated in everyday work. This is particularly relevant in enterprise environments where developers return to problems over multiple days; weak session continuity was reported to increase repeated prompting and reduce perceived efficiency. Investments in client stability, predictable performance, and support processes may therefore contribute to higher perceived productivity even without changes to model capability.

Human factors

Finally, "Human factors: cognitive burden & learning concerns" suggests that organizations may benefit from supporting developers in developing effective and safe usage practices. Developers describe cognitive burden associated with continuously checking AI-generated output, and some express concern that over-reliance may weaken

learning, understanding, and long-term skill development. These concerns are especially relevant from a HCAI perspective because they relate directly to the tension between automation and retained human agency. The findings suggest a need for training that goes beyond prompting tactics and instead targets judgment, when to trust suggestions, how to validate outputs, and how to use assistants to deepen understanding rather than replace it. Clear norms around responsible use and review expectations can also help manage cognitive load and reduce the risk of deskilling, particularly as AI tools take on a broader role in development work.

Summary

Taken together, these factors suggest that realizing value from coding assistants depends in part on whether the organization enables them to be used in a way that is context-rich, well-integrated, operationally stable, and supported by governance and learning-oriented practices. In other words, organizational conditions do not merely influence use; they also appear to shape how perceived task–technology fit is reflected in perceived productivity outcomes in daily work.

These observations are consistent with TTF research, which emphasizes that performance impacts depend on alignment between task demands and technological capabilities rather than on use alone (Goodhue & Thompson, 1995), and with HCAI perspectives highlighting the role of trust, oversight, and meaningful human control in AI-supported work (Shneiderman, 2022).

6.5 Interpreting similarities and differences between coding assistants

Although this study does not aim to systematically compare individual coding assistants, the data allow for a cautious interpretation of similarities and differences in how Amazon Q Developer and AWS Kiro are perceived and used in practice.

The findings suggest that Amazon Q Developer and AWS Kiro are associated with broadly similar perceived productivity outcomes in the studied enterprise context. Across the quantitative comparisons, both tools received generally positive evaluations, and no major divergence was observed across the SPACE dimensions. Satisfaction, for example, was similarly positive for both tools, while Communication & Collaboration differed only marginally, with AWS Kiro scoring slightly higher. Overall, the results do not indicate that either tool produces a fundamentally different productivity profile.

Some differences did emerge in reported usage. AWS Kiro users selected a higher total number of development tasks, suggesting broader reported use across the software development lifecycle. Amazon Q appeared somewhat more concentrated around understanding code flow and bug fixing, while AWS Kiro showed relatively broader use in feature development, code review, documentation, and test implementation. However, the chi-square comparison of task distributions by tool was not statistically significant, indicating that the overall task pattern remained broadly similar. The observed difference therefore appears to concern usage breadth rather than a qualitatively different use structure.

One interpretation is that contemporary enterprise coding assistants may be similar enough in their core value that developers do not strongly differentiate between them in terms of productivity outcomes. Shared capabilities such as code explanation, generation, and debugging support, together with common organizational constraints related to verification, integration, and stability, may therefore matter more than tool-specific features alone.

Another interpretation is that differences in design and interaction style may shape how developers use the tools, even if these differences do not translate into clear differences in perceived productivity outcomes. It is also possible that the observed differences reflect variation in who adopts which tool, such as differences in team roles or task portfolios, which cannot be fully disentangled in a cross-sectional survey.

Taken together, the findings suggest that perceived productivity outcomes are shaped less by the choice between Amazon Q and AWS Kiro and more by how well a given tool fits developers' tasks and organizational contexts. For these two contemporary enterprise coding assistants, task–technology fit and contextual conditions appear more central than tool identity alone.

6.6 Implications for theory and practice

The findings of this study have implications for both theory and practice. From a theoretical perspective, the results reinforce the argument that developer productivity should be understood as a multidimensional construct rather than as a single output-oriented measure. By applying the SPACE framework, the study shows that coding assistants are associated with stronger perceived effects in some dimensions, particularly Satisfaction and Efficiency & Flow, than in others, especially Communication & Collaboration. This supports prior calls to move beyond narrow productivity proxies such as coding speed, output volume, or usage frequency when evaluating AI-supported software development. The study also suggests that TTF provides important explanatory value in this context. Across all regression models, perceived TTF was positively associated with the examined SPACE dimensions, whereas utilization alone was not statistically significant. This indicates that productivity outcomes are more closely related to how well coding assistants fit developers' task demands and work conditions than to how frequently the tools are used. In this respect, the study extends prior work by showing that an explanatory perspective centered on fit can complement multidimensional productivity frameworks such as SPACE.

Taken together, the study suggests that combining SPACE and TTF is theoretically useful for evaluating AI coding assistants in enterprise software development. SPACE helps capture what kinds of productivity outcomes developers perceive, while TTF helps explain why those outcomes vary across tasks, users, and organizational contexts. This contributes to a more nuanced and human-centered understanding of AI-supported development work. From a practical perspective, the findings suggest that organizations should not evaluate coding assistants primarily through adoption rates or usage volume. Instead, they should focus on whether the tools fit the actual tasks developers perform and the conditions under which those tasks are carried out. In practice, this means improving access to relevant context, supporting stronger integration with existing development workflows, and reducing friction in how developers interact with the tools.

The findings also indicate that governance and verification practices are central to realizing sustainable value. Because correctness issues, hallucinations, and verification burden can offset apparent efficiency gains, organizations need review norms, quality assurance mechanisms, and security practices that support responsible use. In addition, training should go beyond basic prompting skills and help developers build judgment about when to trust outputs, how to validate them effectively, and how to use coding assistants in ways that preserve learning, control, and professional agency.

7 Conclusion

7.1 Conclusions

This thesis examined the perceived effectiveness of GenAI coding assistants in enterprise software development, using the SPACE framework to capture productivity as a multidimensional construct and TTF as an explanatory lens. Focusing on Amazon Q Developer and AWS Kiro, the study investigated how developers perceive the impact of these tools on their daily work and under what conditions perceived productivity benefits emerge.

The findings show that developers generally perceive AI coding assistants as supportive tools, but that perceived productivity effects vary across dimensions. The strongest perceived impacts are observed for Satisfaction and Efficiency & Flow, followed by Activity, while Performance is more moderate and Communication & Collaboration is the weakest and most variable dimension. This pattern indicates that coding assistants are primarily experienced as tools that support individual workflow and task execution, rather than consistently strengthening outcome oriented or team dependent aspects of productivity.

Across the analyses, perceived task–technology fit emerges as a central factor explaining variation in developer perceived productivity across all SPACE dimensions. Alignment between tool capabilities and task demands is more strongly associated with positive outcomes than frequency of use alone, indicating that productivity gains depend on how well coding assistants fit developers’ tasks and working conditions rather than on how often they are used.

Finally, the results indicate that effective use of AI coding assistants is shaped by organizational and contextual conditions such as workflow integration, reliability, operational stability etc. that extend beyond tool availability. Taken together, the findings show that coding assistants are perceived by developers as supportive tools in enterprise software development, particularly for well-defined and commonly occurring tasks, but that these perceived productivity benefits depend on strong task–tool fit and organizational conditions that enable productive and reliable use.

7.2 Contributions

This thesis contributes to the growing body of research on Generative AI coding assistants in software development in four main ways: theoretical, empirical, methodological, and practical. Although prior studies have examined the productivity effects of coding assistants, much of the literature has relied on narrow productivity proxies, such as coding speed or output volume, or on controlled tasks and experimental settings rather than real enterprise environments (Forsgren et al., 2021; Yu, 2025).

First, this thesis makes a theoretical contribution by integrating the SPACE framework with TTF in the context of AI coding assistants. Existing research has increasingly used SPACE to capture the multidimensional nature of developer productivity, while more recent studies have drawn on TTF-related reasoning to explain differences in performance when using digital technologies. However, these perspectives have rarely been combined into a single explanatory model for coding assistants. By using SPACE as a multidimensional outcome framework and TTF as an explanatory lens, this thesis provides a more complete account

of both what productivity outcomes developers perceive and why those outcomes vary across users and tasks. In this way, the study extends prior SPACE-based research, which has largely been descriptive, and responds to calls for broader, human-centered approaches to productivity evaluation beyond activity-based metrics alone (Forsgren et al., 2021; Chen et al., 2026).

This theoretical integration also refines prior discussions of adoption success by showing that coding assistants should not be evaluated primarily through usage volume alone, but through the degree of alignment between tool capabilities, task characteristics, and enterprise context. The study therefore offers a more nuanced explanation of heterogeneous outcomes than research that focuses mainly on adoption rates or general productivity gains (Goodhue & Thompson, 1995; Yu, 2025; Chen et al., 2026).

Second, this thesis makes an empirical contribution by providing evidence from a real enterprise software development setting involving developers using Amazon Q Developer and AWS Kiro. Much prior research on AI coding assistants has focused on GitHub Copilot, student populations, benchmark tasks, or experimental settings detached from the complexity of industrial software engineering. In contrast, this study examines developer perceptions in a large enterprise context where work is shaped by mature codebases, organizational constraints, review practices, governance requirements, and tool integration challenges. The findings therefore add context-specific evidence to a research area where enterprise-based studies remain limited.

Third, this thesis makes a methodological contribution by applying a mixed-methods design that integrates quantitative SPACE-based analysis with qualitative thematic analysis. Prior research has highlighted that the impact of AI coding assistants can be difficult to interpret when only one type of data is used, since apparent efficiency gains may conceal downstream costs such as increased verification effort, cognitive burden, or review overhead. By combining descriptive statistics, regression analysis, and open-ended qualitative responses, this study captures both the perceived benefits and the constraints of coding assistant use. In doing so, it demonstrates how multidimensional survey measures and qualitative evidence can be combined to study AI-assisted software development in a more ecologically valid way (Forsgren et al., 2021; Chen et al., 2026; Weisz et al., 2025).

Fourth, this thesis makes a practical contribution by identifying the organizational conditions that shape whether coding assistants deliver value in enterprise settings. The qualitative findings show that usefulness is strongly influenced by access to relevant context, integration with workflows and repositories, output reliability, operational stability, and the cognitive effort required for verification. These insights move beyond a simple “AI improves productivity” narrative by showing that organizational support, governance, workflow design, and developer support are central to realizing value from coding assistants. The study therefore contributes practical knowledge for organizations seeking to deploy such tools responsibly and effectively, especially in environments where software quality, maintainability, and human oversight remain critical (Yu, 2025; Weisz et al., 2025).

Taken together, these contributions position the thesis as an extension of current research from narrow and often tool-specific productivity discussions toward a more multidimensional, explanatory, and enterprise-grounded understanding of AI coding assistants. Overall, the study shows that the value of coding assistants is best understood not simply in terms of faster code generation, but in terms of how well they fit real development tasks, how they shape multiple dimensions of developer-perceived

productivity, and under what organizational conditions they can augment rather than disrupt human work (Forsgren et al., 2021; Chen et al., 2026).

7.3 Limitations

This thesis has several limitations that should be considered when interpreting the results. First, the study is based on a single enterprise organization and a voluntary survey sample, which limits external validity. Usage patterns, governance constraints, and team practices may differ across organizations, so the findings should be interpreted as context dependent.

Second, adoption of GenAI tools is not uniform across software development teams in the organization. Differences in knowledge, trust, experience, mindset, and organizational culture may influence how the tools are used and, in turn, affect the overall assessment of their effectiveness.

Third, the research design is cross-sectional and captures experiences at a single point in time while both the tools and the organizational rollout were still evolving. The study therefore cannot support causal claims or assess longer-term effects such as sustained productivity changes, skill development, or technical debt accumulation.

Fourth, both SPACE and TTF were operationalized in simplified form to reduce respondent burden. Adapting SPACE to individual-level survey items and using a shortened TTF scale may underrepresent some aspects of the constructs, particularly Communication & Collaboration, where measurement was based on few items and the qualitative data provided less direct evidence.

Fifth, the study does not include objective data such as repository, CI/CD, or code review metrics, nor does it track quality or security over time. The findings should therefore be understood as developers' perceptions of impact rather than as a complete measure of actual organizational productivity.

Sixth, the organization uses several types of GenAI tools, including coding assistants, search tools, chatbots, and in-house agentic AI tools. This makes it difficult to evaluate the overall effectiveness of GenAI in the organization. The thesis therefore focuses specifically on coding assistants, namely Amazon Q Developer and AWS Kiro, rather than the full GenAI tool landscape.

Finally, empirical research on GenAI tools in software development is still limited and the field is evolving rapidly. Earlier studies may not reflect the current state of the technology, and the small number of available studies also limited the size of the literature review.

7.4 Future Research

This study highlights that the value of GenAI coding assistants in enterprise software development cannot be assessed solely in terms of adoption or frequency of use. While perceived TTF consistently predicted outcomes across all SPACE dimensions, utilisation alone did not. Future research should therefore focus on clarifying when coding assistants generate net productivity value and when they primarily redistribute effort toward verification, review, and coordination. Addressing this distinction requires research designs that move beyond cross-sectional perceptions and examine outcomes over time, across tasks, and within organizational contexts.

Longitudinal productivity outcomes and skill development

Longitudinal research is needed to examine whether perceived productivity gains persist as tools and users mature, and how short-term improvements relate to longer-term outcomes such as code quality, maintainability, skill development, and technical debt. Such studies could clarify whether early gains in efficiency and flow translate into sustainable productivity improvements or instead contribute to weaker learning, changed onboarding practices, or increased downstream effort over time.

Integrating perceived and objective productivity measures

Future research should integrate SPACE-aligned survey data with behavioural and outcome data from development infrastructures such as version control systems, CI/CD pipelines, issue tracking tools, and code review platforms. Combining perceived productivity with objective indicators would enable more robust evaluation of how coding assistants affect different stages of the development lifecycle and help distinguish genuine productivity gains from redistributed effort.

Task-level and context-level task–technology fit

Further studies should assess TTF at a more granular, task-specific level to identify where coding assistants are well suited and where they are not. In particular, future research should examine how context availability, prompting effort, and enterprise constraints shape outcomes across different development tasks, such as comprehension, debugging, feature implementation, and testing. This line of research could refine TTF as an analytical construct by linking perceived fit more explicitly to concrete task characteristics and organizational conditions.

Code review load, coordination costs, and collaboration outcomes

As Communication & Collaboration emerged as the weakest and most variable SPACE dimension in this study, future research should investigate how coding assistants affect coordination costs, code review load, knowledge sharing, and team workflows. This includes examining whether coding assistants support shared understanding and collaboration or introduce additional review and alignment effort that offsets individual productivity gains.

Governance, security, and accountability in enterprise use

Future research should examine how governance, security, and accountability mechanisms shape the effective use of coding assistants in enterprise settings. In particular, it should investigate which practices can reduce risks such as hallucinations, maintainability problems, and verification burden without diminishing productivity. It should also explore how these mechanisms influence developer trust, autonomy, and alignment with HCAI principles.

Reactive and agentic coding assistants in enterprise settings

As development tools evolve from reactive assistants to more agentic systems, future research should compare these interaction paradigms in terms of productivity, reliability, cognitive burden, traceability, and support for collaboration. Important questions include how different levels of autonomy affect accountability, how interfaces influence developers' mental models and confidence, and how agentic workflows redistribute work across roles such as developers, reviewers, architects, and testers.

Overall, future research should aim to produce more robust and practically useful evidence about when coding assistants create net value in enterprise software development and when

they mainly shift effort toward verification and coordination. Combining longitudinal designs, task-level analyses of fit, mixed-methods integration, and comparisons between reactive and agentic tools could help refine SPACE and TTF into more effective frameworks for evaluating rapidly evolving AI-supported development work.

References

- Afroz, S., Feng, Z., Menezes, T., Kimura, K., Trinkenreich, B., Steinmacher, I., & Sarma, A. (2026). The fast and spurious: Developer productivity with GenAI. In *Companion Proceedings of the 34th ACM Symposium on the Foundations of Software Engineering (FSE '26)*. ACM.
<https://doi.org/10.48550/arXiv.2510.24265>
- Akinwande, O., Dikko, H. G., & Agboola, S. (2015). Variance inflation factor: As a condition for the inclusion of suppressor variable(s) in regression analysis. *Open Journal of Statistics*, 5(7), 754–767. <https://doi.org/10.4236/ojs.2015.57075>
- Ali, P., & Younas, A. (2021). Research made simple: Understanding and interpreting regression analysis. *Evidence-Based Nursing*, 24(4), 116–118.
<https://doi.org/10.1136/ebnurs-2021-103425>
- Ammenwerth, E., Iller, C., & Mahler, C. (2006). IT-adoption and the interaction of task, technology and individuals: A fit framework and a case study. *BMC Medical Informatics and Decision Making*, 6, Article 3. <https://doi.org/10.1186/1472-6947-6-3>
- Banh, L., Holldack, F., & Strobel, G. (2025). Github Copiloting the future: How generative AI transforms software engineering. *Information and Software Technology*, 183, Article 107751. <https://doi.org/10.1016/j.infsof.2025.107751>
- Bazine, I.-d., & Salahddine, M. A. (2025). Understanding the task-technology fit (TTF) model: Genesis and conceptual developments. *International Journal of Latest Technology in Engineering, Management & Applied Science*, 14(8).
<https://doi.org/10.51583/IJLTEMAS.2025.1408000030>
- Becker, J., Rush, N., Barnes, E., & Rein, D. (2025). Measuring the impact of early-2025 AI on experienced open-source developer productivity (arXiv:2507.09089). arXiv. <https://doi.org/10.48550/arXiv.2507.09089>
- Borg, M., Hewett, D., Hagatulah, N., Couderc, N., Söderberg, E., Graham, D., Kini, U., & Farley, D. (2025). Echoes of AI: Investigating the downstream effects of AI assistants on software maintainability <https://doi.org/10.48550/arXiv.2507.00788>
- Chang, T.-S., & Huang, L.-G. (2026). The impact of generative artificial intelligence on software development performance: A study based on the theory of technology dominance. *Information & Management*, 63(3), 104306.
<https://doi.org/10.1016/j.im.2026.104306>
- Chen, V., He, J., Williams, B., Valentino, J., & Talwalkar, A. (2026). Beyond the Commit: Developer Perspectives on Productivity with AI Coding Assistants. In *2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE-SEIP '26)*. ACM, New York, NY, USA.
<https://doi.org/10.48550/arXiv.2602.03593>
- Corso, V., Mariani, L., Micucci, D., & Riganelli, O. (2024). Assessing AI-based code assistants in method generation tasks. In *Proceedings of the 2024 IEEE/ACM*

- 46th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion '24) (pp. 380–381). ACM.
<https://doi.org/10.1145/3639478.3643122>
- Coutinho, M., Marques, L., Santos, A., Dahia, M., França, C., & Santos, R. S. (2024). The role of generative AI in software development productivity: A pilot case study. In Proceedings of the 1st ACM International Conference on AI-Powered Software (AIware '24). ACM. <https://doi.org/10.1145/3664646.3664773>
- Cursor IDE Blog. (2025, July 17). Kiro IDE review 2025: AWS's revolutionary spec-driven AI coding assistant. <https://www.cursor-ide.com/blog/kiro-ide-review>
- Dang, T. T., Jani, J. M., Ling, H. C., Tran, L. D., Tran, Q., Nguyen, Q., Tran, H., & Pham, T. C. (2026). Are LLMs ready for competitive programming? An empirical study. In Proceedings of the 2026 14th International Conference on Software and Information Engineering (ICSIE 2026). ACM.
<https://doi.org/10.1145/3789595.3789613>
- Demir, K. A., Muppalla, T. N. V., & Liu, B. (2025). Generative AI efficiency and effectiveness in software project documentation review process. In Proceedings of the 2nd International Conference on Artificial Intelligence, Computer, Data Sciences, and Applications (ACDSA 2025). IEEE.
<https://doi.org/10.1109/ACDSA65407.2025.11165833>
- Dishaw, M. T., & Strong, D. M. (1998). Supporting software maintenance with software engineering tools: A computed task–technology fit analysis. *Journal of Systems and Software*, 44(2), 107–120. [https://doi.org/10.1016/S0164-1212\(98\)10048-1](https://doi.org/10.1016/S0164-1212(98)10048-1)
- Dishaw, M. T., & Strong, D. M. (1998). Assessing software maintenance tool utilization using task–technology fit and fitness-for-use models. *Journal of Software Maintenance: Research and Practice*, 10(3), 151–175.
[https://doi.org/10.1002/\(SICI\)1096-908X\(199805/06\)10:3<151::AID-SMR165>3.0.CO;2-%23](https://doi.org/10.1002/(SICI)1096-908X(199805/06)10:3<151::AID-SMR165>3.0.CO;2-%23)
- Fischer, M., & Lanquillon, C. (2024). Evaluation of generative AI-assisted software design and engineering: A user-centered approach. In H. Degen & S. Ntoa (Eds.), *Lecture Notes in Computer Science* (Vol. 14734, pp. 31–47). Springer.
https://doi.org/10.1007/978-3-031-60606-9_3
- Flamourtzoglou, N., Vrochidou, E., & Papakostas, G. A. (2025). AI Github Copilots for machine learning pipelines: A comparative study on code quality and model performance. In Proceedings of the 3rd International Conference on Intelligent Technologies (3ICT 2025). IEEE.
<https://doi.org/10.1109/3ict68299.2025.11442232>
- Forsgren, N., Storey, M.-A., Maddila, C., Zimmermann, T., Houck, B., & Butler, J. (2021). The SPACE of developer productivity. *Queue*, 19(1), 20–48.
<https://doi.org/10.1145/3453928>
- Goodhue, D. L., & Thompson, R. L. (1995). Task-technology fit and individual performance. *MIS Quarterly*, 19(2), 213–236. <https://doi.org/10.2307/249689>

- Goodhue, D. L. (1998). Development and measurement validity of a task–technology fit instrument for user evaluations of information systems. *Decision Sciences*, 29(1), 105–138. <https://doi.org/10.1111/j.1540-5915.1998.tb01346.x>
- Guenes, P., Tomaz, R., Kalinowski, M., Baldassarre, M. T., & Storey, M.-A. (2024). Impostor phenomenon in software engineers. In *Proceedings of the 46th International Conference on Software Engineering: Software Engineering in Society (ICSE-SEIS 2024)* (pp. 96–106). ACM. <https://doi.org/10.48550/arXiv.2312.03966>
- Guetterman, T. C., Fetters, M. D., & Creswell, J. W. (2015). Integrating quantitative and qualitative results in health science mixed methods research through joint displays. *The Annals of Family Medicine*, 13(6), 554–561. <https://doi.org/10.1370/afm.1865>
- Hands, A. S. (2022). Integrating quantitative and qualitative data in mixed methods research: An illustration. *Canadian Journal of Information and Library Science*, 45(1), 1–20. <https://doi.org/10.5206/cjilsresib.v45i1.10645>
- Hassan, A. E., Oliva, G. A., Lin, D., Chen, B., & Jiang, Z. M. (2026). Towards AI-native software engineering (SE 3.0): A vision and a challenge roadmap. *ACM Transactions on Software Engineering and Methodology*, 35(4), Article 4. <https://doi.org/10.1145/3807901>
- Houck, B., Lowdermilk, T., Beyer, C., Clarke, S., & Hanrahan, B. (2025). The SPACE of AI: Real-World lessons on AI’s impact on developers <https://doi.org/10.48550/arXiv.2508.00178>
- Kaul, S., Nhu, K., Eissayou, J., Eser, I., & Borup, V. (2024). SPACEX: Exploring metrics with the SPACE model for developer productivity. ACM, New York, NY, USA. <https://doi.org/10.48550/arXiv.2511.20955>
- Klemmer, J. H., Horstmann, S. A., Patnaik, N., Ludden, C., Burton, C., Jr., Powers, C., Massacci, F., Rahman, A., Votipka, D., Richter Lipford, H., Rashid, A., Naiakshina, A., & Fahl, S. (2024). Using AI assistants in software development: A qualitative study on security practices and concerns. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS '24)*. ACM. <https://doi.org/10.1145/3658644.3690283>
- Kumar, A., Khare, V., Sharma, D., Kumar, S., Saini, V., Yadav, A., Jain, S., Rana, A., Verma, P., Meena, V., & Edubilli, A. (2025). *Intuition to evidence: Measuring AI's true impact on developer productivity*. arXiv. <https://doi.org/10.48550/arXiv.2509.19708>
- Lau, S., & Guo, P. J. (2025). The design space of LLM-based AI coding assistants: An analysis of 90 systems in academia and industry. In *Proceedings of the 2025 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC 2025)*. IEEE. <https://conf.researchr.org/details/vlhcc-2025/vlhcc-2025-research-papers/32/The-Design-Space-of-LLM-Based-AI-Coding-Assistants-A>
- Li, H., Zhang, H., & Hassan, A. E. (2025). The rise of AI teammates in software engineering (SE) 3.0: How autonomous coding agents are reshaping software

- engineering. In Proceedings of Conference Acronym 'XX. ACM, New York, NY, USA. <https://doi.org/10.48550/arXiv.2507.15003>
- Lin, W.-S. (2012). Perceived fit and satisfaction on web learning performance: IS continuance intention and task-technology fit perspectives. *International Journal of Human-Computer Studies*, 70(7), 498–507. <https://doi.org/10.1016/j.ijhcs.2012.01.006>
- Liu, C., Lin, Y., Chang, J., Liu, J., Qi, B., Jiang, B., Huang, Z., & Dong, J. S. (2026). EditFlow: Benchmarking and optimizing code edit recommendation systems via reconstruction of developer flows. *Proceedings of the ACM on Programming Languages*, 10(OOPSLA1), Article 141. <https://doi.org/10.1145/3798249>
- Lyu, M., Ray, B. B., Roychoudhury, A., Tan, S. H., & Thongtanunam, P. (2025). Automatic programming: Large language models and beyond. *ACM Transactions on Software Engineering and Methodology*, 34(5), Article 117. <https://doi.org/10.48550/arXiv.2405.02213>
- Marcus, A. (2026, January 11). *The paradigm shift from reactive to proactive AI in software development: A comparative analysis of AI IDEs*. DEV Community. <https://dev.to/AWSKirodotdev/the-paradigm-shift-from-reactive-to-proactive-ai-in-software-development-a-comparative-analysis-of-ai-ides-123>
- Marikyan, D., & Papagiannidis, S. (2023). Task-technology fit: A review. In S. Papagiannidis (Ed.), *TheoryHub Book*. <http://open.ncl.ac.uk>
- Mendes, W., Souza, S., & de Souza, C. R. B. (2024). "You're on a bicycle with a little motor": Benefits and challenges of using AI code assistants. In *Proceedings of the 2024 IEEE/ACM 17th International Conference on Cooperative and Human Aspects of Software Engineering (CHASE '24)*. ACM. <https://doi.org/10.1145/3641822.3641882>
- Meyer, A. N., Barr, E. T., Bird, C., & Zimmermann, T. (2021). Today was a good day: The daily life of software developers. *IEEE Transactions on Software Engineering*, 47(5), 863–880. <https://doi.org/10.1109/TSE.2019.2904957>
- Miller, W. L., Crabtree, B. F., Harrison, M. I., & Fennell, M. L. (2013). Integrating mixed methods in health services and delivery system research. *Health Services Research*, 48(6 Pt 2), 2125–2133. <https://doi.org/10.1111/1475-6773.12123>
- Minelli, R., Mocci, A., & Lanza, M. (2015). I know what you did last summer: An investigation of how developers spend their time. In *Proceedings of the 2015 IEEE 23rd International Conference on Program Comprehension (ICPC 2015)* (pp. 25–35). IEEE. <https://doi.org/10.1109/ICPC.2015.12>
- Mohamed, A., Assi, M., & Guizani, M. (2025, July 3). *The impact of LLM-assistants on software developer productivity: A systematic literature review* [Preprint]. arXiv. <https://doi.org/10.48550/arXiv.2507.03156>
- Moksony, F. (1999). Small is beautiful: The use and interpretation of R^2 in social research. *Szociológiai Szemle*, Special issue, 130–138.

- Ng, K. K. B., Fauzi, L., Leow, L., & Ng, J. (2024). Harnessing the potential of Gen-AI coding assistants in public sector software development. Government Technology Agency (GovTech) Singapore. <https://doi.org/10.48550/arXiv.2409.17434>
- Nghiem, K., Nguyen, A. M., & Bui, N. D. Q. (2024). Envisioning the next-generation AI coding assistants: Insights & proposals. In Proceedings of the 2024 IEEE/ACM 1st International Workshop on Integrated Development Environments (IDE '24) (pp. 7–10). Association for Computing Machinery. <https://doi.org/10.48550/arXiv.2403.14592>
- O'Brien, R. M. (2007). A caution regarding rules of thumb for variance inflation factors. *Quality & Quantity*, 41, 673–690. <https://doi.org/10.1007/s11135-006-9018-6>
- Ozili, P. K. (2022, January). The acceptable R-square in empirical modelling for social science research. *SSRN Electronic Journal*. <https://doi.org/10.2139/ssrn.4128165>
- Özer, Mehmet & Dündar, Salih. (2026). Evaluating LLM Performance on CWE Identification and CVSS Estimation. 684-688. 10.1109/IISec69317.2026.11418496
- Passmore, J., Daly, J., & Tee, D. (2025). If the hat fits: A review of task technology fit theory (TTF) through users' experience of engaging with an AI coaching agent. *Organisationsberatung, Supervision, Coaching*, 33, Article 1. <https://doi.org/10.1007/s11613-025-00981-8>
- Pearce, H., Ahmad, B., Tan, B., Dolan-Gavitt, B., & Karri, R. (2022). Asleep at the keyboard? Assessing the security of Github Copilot's code contributions. In *Proceedings of the 2022 IEEE Symposium on Security and Privacy (SP 2022)*. IEEE. <https://doi.org/10.1109/SP46214.2022.9833571>
- Pezzè, M., Abrahão, S., Penzenstadler, B., Poshyvanyk, D., Roychoudhury, A., & Yue, T. (2025). A 2030 roadmap for software engineering. *ACM Transactions on Software Engineering and Methodology*, 34(5). <https://doi.org/10.1145/3731559>
- Pirzado, F. A., Ahmed, A., Mendoza-Urdiales, R. A., & Terashima-Marin, H. (2024). Navigating the pitfalls: Analyzing the behavior of LLMs as a coding assistant for computer science students—A systematic review of the literature. *IEEE Access*, 12, 116248–116275. <https://doi.org/10.1109/ACCESS.2024.3443621>
- Reeping, D., & Edwards, C. (2022). Integrating using a crossover analysis with formative joint displays in mixed methods research. *Journal of Mixed Methods Research*, 16(4), 458–477. <https://doi.org/10.1177/15586898211047660>
- Roy, P. (2025). *Towards just-in-time, inclusive clone refactoring* [Master's thesis, University of Saskatchewan].
- Sabahi, N. (2025, August 16). *AWS Kiro AI IDE vs Cursor AI IDE*. New Math Data. <https://newmathdata.com/2025/08/16/AWS-Kiro-ai-ide-vs-cursor-ai-ide/>
- Sabouri, S., Eibl, P., Zhou, X., Ziyadi, M., Medvidovic, N., Lindemann, L., & Chattopadhyay, S. (2025). Trust dynamics in AI-assisted development: Definitions, factors, and implications. In Proceedings of the 2025 IEEE/ACM

- 47th International Conference on Software Engineering (ICSE) (pp. 736–736). IEEE Computer Society. <https://ieeexplore.ieee.org/document/11029928>
- Salah, D. (2025). Invisible risks, visible code: A vision for understanding ethical debt in AI-based coding. In *Proceedings of the 2025 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM 2025)*. ACM/IEEE. <https://conf.researchr.org/details/esem-2025/esem-2025-vision-and-emerging-results-track-/18/Invisible-Risks-Visible-Code-A-Visi>
- Sankhe, P., Patil, N., Ghorpade, M., Prasad, P., & Linkesh, M. (2024). Empirical analysis of AI-assisted code generation tools: Impact on code quality, security and developer productivity. *International Journal for Multidisciplinary Research*. <https://www.ijfmr.com>
- Sauvola, J., Tarkoma, S., Klemettinen, M., Riekk, J., & Doermann, D. (2024). Future of software development with generative AI. *Automated Software Engineering*, 31, Article 26. <https://doi.org/10.1007/s10515-024-00426-z>
- Sharma, R. K. (2025). Amazon AWS Kiro: Transforming software development through AI-powered autonomous coding. *International Journal of Science and Research*, 14(11), 866–900.
- Shihab, M. I. H., Hundhausen, C., Tariq, A., Haque, S., Qiao, Y., & Mulanda, B. W. (2025). The effects of Github Copilot on computing students' programming effectiveness, efficiency, and processes in brownfield coding tasks. In *Proceedings of the 2025 ACM Conference on International Computing Education Research Vol. 1 (ICER 2025)*. ACM. <https://doi.org/10.1145/3702652.3744219>
- Shneiderman, B. (2022). *Human-centered AI*. Oxford University Press.
- Stalnaker, T., Wintersgill, N., Chaparro, O., Heymann, L. A., Di Penta, M., German, D. M., & Poshyvanyk, D. (2026). Developer perspectives on licensing and copyright issues arising from generative AI for software development. *ACM Transactions on Software Engineering and Methodology*, 35(2). <https://doi.org/10.1145/3743133>
- Swaminathan, N., & Singh, D. (2025, July). *Introducing AWS Kiro*. AWS Kiro. <https://AWSKiro.dev/blog/introducing-AWSKiro/>
- Thangarajah, K., Chen, B., Chang, S., & Hassan, A. E. (2025). *Context-aware CodeLLM eviction for AI-assisted coding*. arXiv. <https://doi.org/10.48550/arXiv.2506.18796>
- Tomaz, R., Guenes, P., Araújo, A. A., Baldassarre, M. T., & Kalinowski, M. (2026, February 13). *Impacts of generative AI on agile teams' productivity: A multi-case longitudinal study*. arXiv. <https://arxiv.org/abs/2602.13766>
- Vongjaturapat, S. (2018). Application of the task-technology fit model to structure and evaluation of the adoption of smartphones for online library systems. *Science & Technology Asia*, 23(1), 39–56. <https://doi.org/10.14456/scitechasia.2018.6>

- Weber, T., Brandmaier, M., Schmidt, A., & Mayer, S. (2024). Significant productivity gains through programming with large language models. *Proceedings of the ACM on Human-Computer Interaction*, 8(EICS), Article 233. <https://doi.org/10.1145/3661145>
- Weisz, J. D., Kumar, S. V., Muller, M., Browne, K. B., Goldberg, A., Heintze, K. E., & Bajpai, S. (2025). Examining the use and impact of an AI code assistant on developer productivity and experience in the enterprise. In *Extended Abstracts of the 2025 CHI Conference on Human Factors in Computing Systems (CHI EA '25)*. ACM. <https://doi.org/10.1145/3706599.3706670>
- Welter, A., Schneider, N., Dick, T., Weis, K., Tinnes, C., Wyrich, M., & Apel, S. (2024). *An empirical study of knowledge transfer in AI pair programming* [Unpublished manuscript]. Saarland University; Siemens AG.
- Xie, C., Jiao, M., Gu, X., & Shen, B. (2025). Empowering AI to generate better AI code: Guided generation of deep learning projects with LLMs. In *Proceedings of the 2025 IEEE 49th Annual Computers, Software, and Applications Conference (COMPSAC 2025)* (pp. 1–12). IEEE. <https://doi.org/10.1109/COMPSAC65507.2025.00175>
- Xia, C. S., Deng, Y., Dunn, S., & Zhang, L. (2025). The design space of LLM-based AI coding assistants: An analysis of 90 systems in academia and industry. *Proceedings of the ACM on Software Engineering*, 2(FSE). <https://doi.org/10.1145/3715754>
- Yilmaz, K. (2013). Comparison of quantitative and qualitative research traditions: Epistemological, theoretical, and methodological differences. *European Journal of Education*, 48(2), 311–325. <https://doi.org/10.1111/ejed.12014>
- Yu, L. (2025). Paradigm shift on coding productivity using GenAI. In *Proceedings of the 29th International Conference on Evaluation and Assessment in Software Engineering (EASE 2025)* (pp. 1–6). ACM. <https://doi.org/10.48550/arXiv.2504.18404>

Appendix A: Report on the use of Generative AI in Master Thesis

Purpose of generative AI use

The following AI tools were used in the thesis work:

Microsoft 365 Github Copilot (Licensed version used within the organisation)

- Used to search for information within the organization
- Fine tuning survey questions
- Analysis of the quantitative and qualitative data from the surveys
- Language editing and planning

Claude AI

- Information search

Extent of generative AI use

All the work in this thesis is original. However, Generative AI was used for language editing, data analysis and planning.

Impact of generative AI use

The use of generative AI supported language editing and planning, contributing to an overall improvement in thesis quality. In addition, its application in data analysis enabled cross-verification of manual calculations, reducing the risk of computational errors, and improving efficiency.

Personal contribution

Our role in the thesis was both practical and analytical. We contributed jointly to all major parts of the study, including the formulation of the research problem, the design of the study, the literature review, data collection, data analysis, interpretation of findings, and the writing and revision of the final thesis. In particular, the thesis draws on our combined work on both the SPACE framework and Task–Technology Fit (TTF), including the related literature review and analytical interpretation.

We ensured that our own critical reflections and analyses were foregrounded by actively engaging in the interpretation of both the theoretical material and the empirical findings throughout the research process. This included discussing alternative explanations, critically evaluating the relevance and limitations of the selected frameworks, interpreting the survey results in relation to the research questions, and jointly shaping the arguments presented in the thesis. Our contribution therefore extended beyond practical coordination and writing support to include active involvement in the intellectual and analytical development of the study.

Ethical considerations of generative AI use

We sought to ensure an ethical use of generative AI throughout the thesis by treating it as a support tool rather than a substitute for our own academic judgment. Only AI tools approved by the organization was used. AI was used selectively for tasks such as language support, structuring ideas, and assisting with parts of the data analysis process, but all outputs were critically reviewed, revised, and validated by us before being included in the

thesis. In this way, responsibility for the content, interpretation, and conclusions remained entirely with us, thereby supporting academic integrity.

To protect data, we were careful not to input organizational information, proprietary code, or other sensitive material into generative AI systems that are not approved for use. The use of AI was limited to tasks and content that did not compromise privacy, confidentiality, or data security. This was particularly important given that the study was conducted in an enterprise context and involved survey-based empirical material.

Copyright and originality were also considered throughout the process. References, citations, and source attributions were checked manually to reduce the risk of plagiarism, misattribution, or the unintentional inclusion of copyrighted material. Overall, we aimed to use generative AI in a transparent, limited, and responsible manner that was consistent with academic integrity, data protection, and copyright requirements.

Reflection on non-use of generative AI (if applicable)

Not applicable.

Appendix B: Description of the work division between the authors

General Description of Collaboration

The thesis was conducted as a collaborative effort, with both authors contributing to the planning, execution, analysis, and writing of the study. Both authors jointly participated in formulating the research problem, designing the study, interpreting the findings, and revising the final manuscript. Meetings with the supervisor at Linnaeus University and representatives from the organization were attended jointly. The authors also held frequent meetings throughout the thesis process and maintained continuous communication through Microsoft Teams and face-to-face discussions at the organization. All major decisions concerning the study were made jointly, and both authors reviewed and approved the final version of the thesis.

Division of Work Across the Thesis Work

All aspects of the thesis were discussed and developed jointly by both authors. The literature review and data analysis were divided between the authors, with Louise assuming primary responsibility for Task–Technology Fit (TTF) and Reena assuming primary responsibility for the SPACE framework. The literature and analytical findings were subsequently discussed jointly and integrated into the final theoretical foundation of the study.

Individual Contributions

Louise Bergman

Contributed to all parts of the thesis in collaboration with Reena. I had primary responsibility over the multiple linear regression analysis and the TTF framework regarding the literature review. Overall, the thesis was created in a team effort where we both took responsibility over the work and the outcome of the thesis.

Reena Thomas

Contributed to all parts of the thesis in collaboration with Louise. Had primary responsibility for the SPACE framework, including the associated literature review and the analysis of SPACE-based data. Also coordinated the distribution of the survey across relevant teams within the organization and conducted follow-ups to ensure enough responses.

All remaining aspects of the thesis, such as planning, structuring the document, and writing, were carried out jointly in close collaboration with Louise and with guidance from supervisors at both the organization and the university.

Statement of Agreement

Louise and Reena confirm that the above description accurately reflects the division of work and their respective contributions to this thesis. Both authors have reviewed and formally approved the content of this appendix.

Appendix C: Survey Questions

Table 17 presents the broader descriptive operationalization of the SPACE framework dimensions based on survey items collected from the respondents. The item groupings shown here were used for the descriptive SPACE analysis and should not be interpreted as identical to the original item-based operationalization used in the regression analysis.

SPACE Dimension	#	Survey Question	Notes
Satisfaction	1	How well do coding assistant features align with your typical tasks?	Perceived fit of features to daily work needs (overall experience).
	1.1	The coding assistant features are suitable for my daily tasks	Agreement item: features suit daily tasks (fit/usability).
	1.2	Coding assistants outputs help me complete my work	Agreement item: outputs help complete work (usefulness).
	1.3	Using the coding assistant makes my tasks less mentally demanding	Agreement item: reduces cognitive load/mental effort.
	2	How do coding assistants impact your daily work experience?	Well-being aspect: whether the assistant helps manage workload/energy.
	2.1	Helps manage workload and energy	Workload/energy management (well-being).
	2.2	Reduces mental fatigue from repetitive tasks	Reduction of mental fatigue from repetitive tasks.
	2.3	Positive integration into workflow	How smoothly the assistant integrates into the existing workflow.
	3	How would you rate your overall satisfaction with the Coding Assistant?	Single overall satisfaction rating
Performance	4	Approximately what percentage of your work time is saved by using the coding assistant?	Perceived productivity gain from time saved.
	5	How have coding assistants affected your ability to complete daily tasks?	Throughput impact: ability to complete more work in a sprint/day.
	5.1	Coding assistants enable me to accomplish more tasks during sprints	Agreement item: accomplish more sprint tasks (throughput).
	5.2	Using coding assistants allows me to allocate more time to other responsibilities	Agreement item: frees time for other responsibilities (capacity).
Activity	6	How long have you been using a coding assistant?	Adoption tenure; how long respondent has used assistants.
	7	How often do you use a coding assistant?	Usage frequency/intensity of assistant use.
	8	What is the usual complexity of tasks you assign to coding assistants?	Typical task difficulty when using the assistant.
	9	Which tasks do you rely on coding assistants for?	Task breadth (multi-select); used to derive breadth/engagement.
Communication & Collaboration	10	How do coding assistants influence collaboration within your team?	Perceived impact on collaboration: understanding others' code and coordination load.
	10.1	Coding assistants help me comprehend colleagues' code without needing to ask them directly	Understanding colleagues' code without asking (reduces clarification needs).
	10.2	Using coding assistants leads to fewer technical discussions among team members	Perceived change in technical discussions/coordination (may be positive or negative).
Efficiency & Flow	11	How does using coding assistants influence code quality?	Rework/verification burden (reverse-scored); can disrupt flow if frequent.
	11.1	Achieve code standards with less manual effort	Effort needed to meet code standards/quality requirements.
	11.2	Often revise AI-generated code before final commit	Frequency of revising AI code before commit (rework/verification effort).
	12	How do coding assistants affect your productivity and ability to stay focused?	Flow and efficiency: interruptions, control, and routine-work time savings.
	12.1	I am able to quickly resume work after interruptions caused by coding assistants	Ability to resume quickly after interruptions (flow continuity).
	12.2	I have control over when and how I use coding assistants	Sense of control/autonomy over when/how to use assistant.
	12.3	Coding assistants help me save time on repetitive or routine tasks	Time saved on routine/repetitive tasks (efficiency).

Table 17: Descriptive operationalization of the SPACE dimensions

Table 18 presents the original item-based operationalization of the SPACE framework dimensions used in the regression analysis. The item groupings shown here were used as dependent-variable measures in the explanatory models and should not be interpreted as identical to the broader descriptive operationalization used in the SPACE analysis.

Space dimension	#	Survey question	Notes
Satisfaction	1	How does the coding assistant impact your daily work experience?	Overall daily work experience.
	1.1	Helps manage workload and energy	Workload and energy support.
	1.2	Reduces mental fatigue from repetitive tasks	Reduced mental fatigue.
	1.3	Positive integration into workflow	Workflow integration.
Performance	2	How does using coding assistants influence code quality?	Overall code quality support
	2.1	I achieve code standards with less manual effort	Support for code quality.
Activity	3	How have coding assistants affected your ability to complete daily tasks?	Overall task completion support.
	3.1	Coding assistants enable me to accomplish more tasks during sprints	Higher task throughput.
	3.2	Using coding assistants allows me to allocate more time to other responsibilities	Frees time for other tasks.
Communication and collaboration	4	How do coding assistants influence collaboration within your team?	Overall collaboration support.
	4.1	Coding assistants help me comprehend colleagues' code without needing to ask them directly	Support for code understanding.
Efficiency and flow	5	How do coding assistants affect your productivity and ability to stay focused?	Overall flow and productivity support.
	5.1	I am able to quickly resume work after interruptions caused by coding assistants	Recovery after interruption.
	5.2	I have control over when and how I use coding assistants	User control in use.
	5.3	Coding assistants help me save time on repetitive or routine tasks	Time savings in routine work.

Table 18: Original item based operationalization of the SPACE dimensions

Table 19 presents the survey items used to operationalize perceived Task–Technology Fit (TTF) in this study. These items were used to construct the explanatory TTF measure in the regression analysis and were treated as distinct from the SPACE productivity measures.

#	Survey question	Notes
1	How well do coding assistant features align with your typical tasks?	Overall perceived task fit.
1.1	The coding assistant's features are suitable for my daily tasks.	Suitability for daily work.
1.2	The coding assistant's outputs help me complete my work.	Perceived task support.
1.3	Using the coding assistant makes my tasks less mentally demanding.	Perceived reduction in mental effort.

Table 19: Item based opalization of TTF construct

Demographic and Contextual Variables

The following items are not mapped to SPACE dimensions. They serve as moderator or subgroup variables in comparative analyses.

Survey Question	Notes
What is your current professional role?	Respondent role for subgroup comparisons
How many years have you worked in your current field?	Tenure in profession; used for segmentation/experience comparisons.
Which coding assistant do you use most frequently?	Primary tool used (e.g., Amazon Q, Kiro); used for tool comparisons.
If the coding assistant were a team member, which role best describes its contribution?	Metaphor-based perception of the assistant's role (qualitative categorization).
What are the potential drawbacks of using Coding Assistants?	Open-ended risks/drawbacks; used for thematic/qualitative analysis.
Can you share a recent task where the Coding Assistant made your work easier or more efficient?	Open-ended example; used for qualitative evidence and themes.
What capabilities would you like to see added to the Coding Assistant?	Open-ended feature requests; used for qualitative roadmap insights.
What challenges have you faced while using Coding Assistants?	Open-ended challenges; used for thematic/qualitative analysis.

Table 20: Demographic and contextual questions

Appendix D: Screenshots

Survey Screenshots

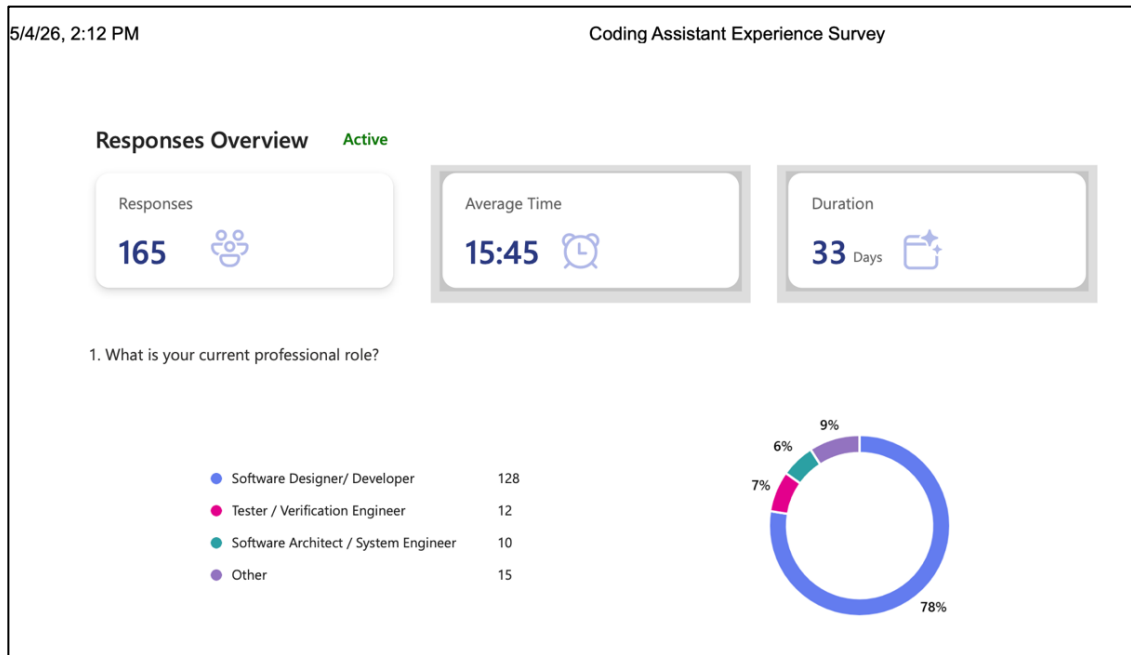


Figure 10: Survey response overview

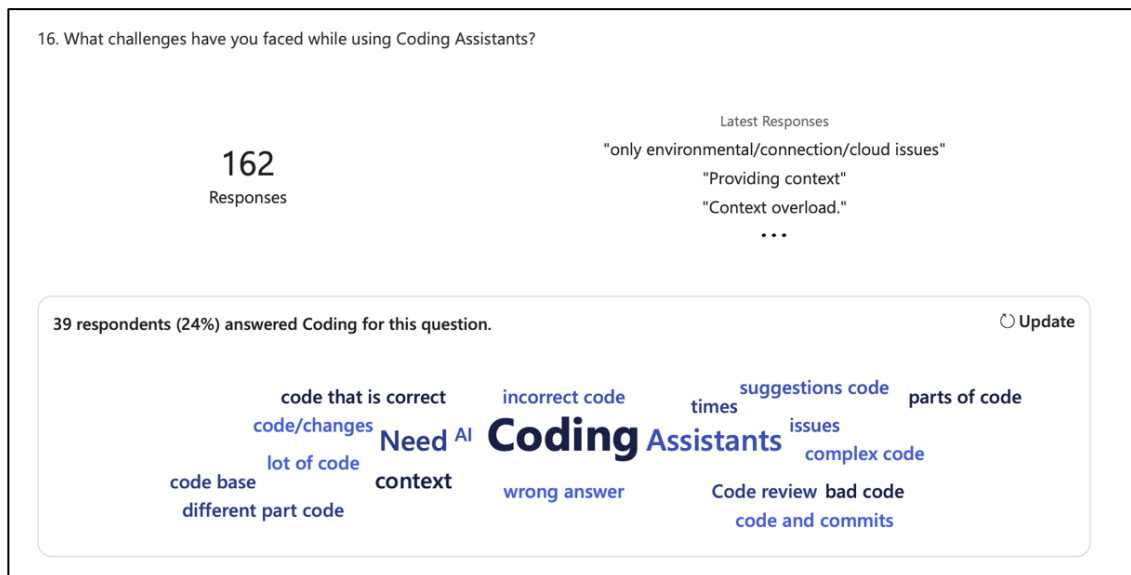


Figure 11: Survey Q16 on the challenges faced while using coding assistants

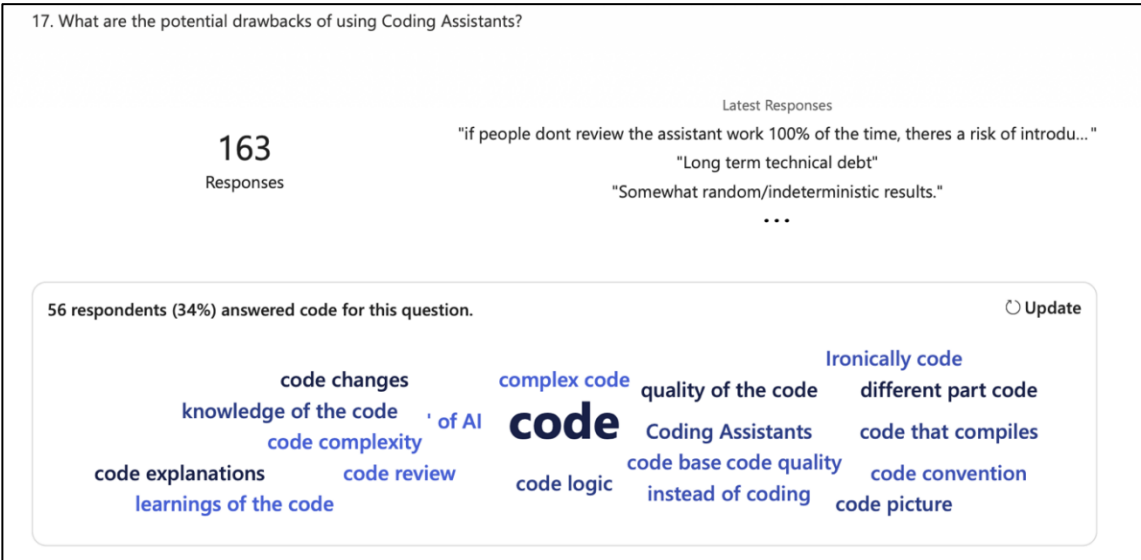


Figure 12: Survey Q17 on the potential drawbacks of using coding assistants

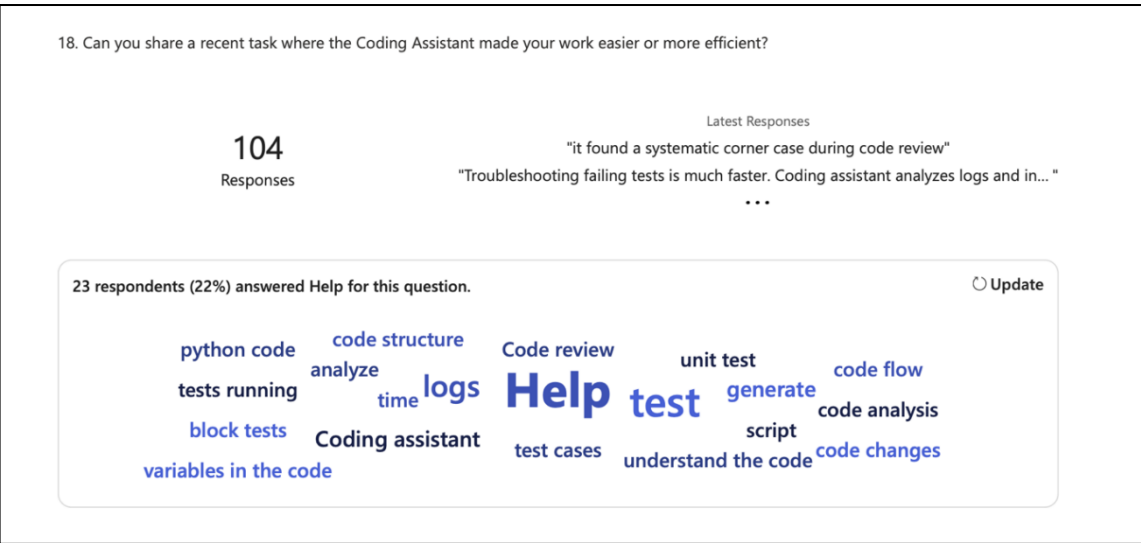


Figure 13: Survey Q18 on recent task where coding assistant made work more efficient

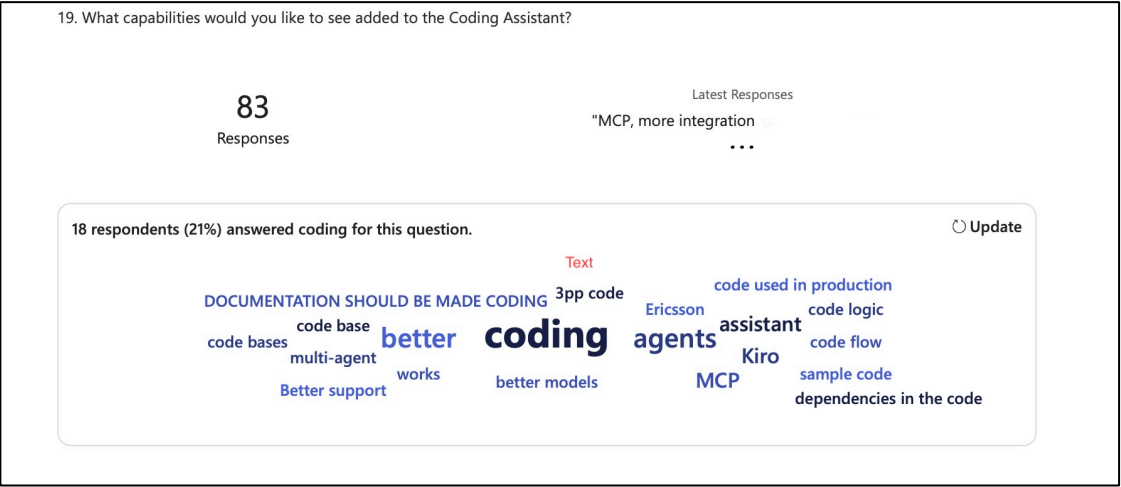


Figure 14: Survey Q19 on capabilities that would be good to have

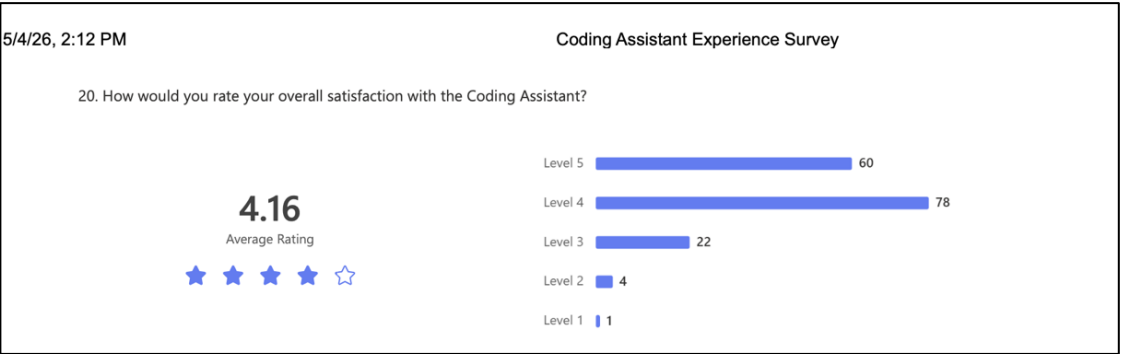


Figure 15: Survey Q20 on overall satisfaction using Coding Assistants

Appendix E: Coding Assistants Usage Patterns

Usage Patterns (Actual data from the organization)

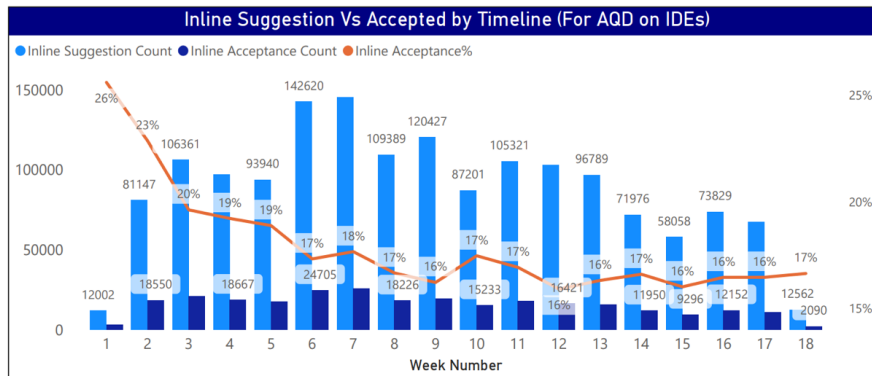


Figure 16: Inline suggestions vs acceptance for Amazon Q Developer

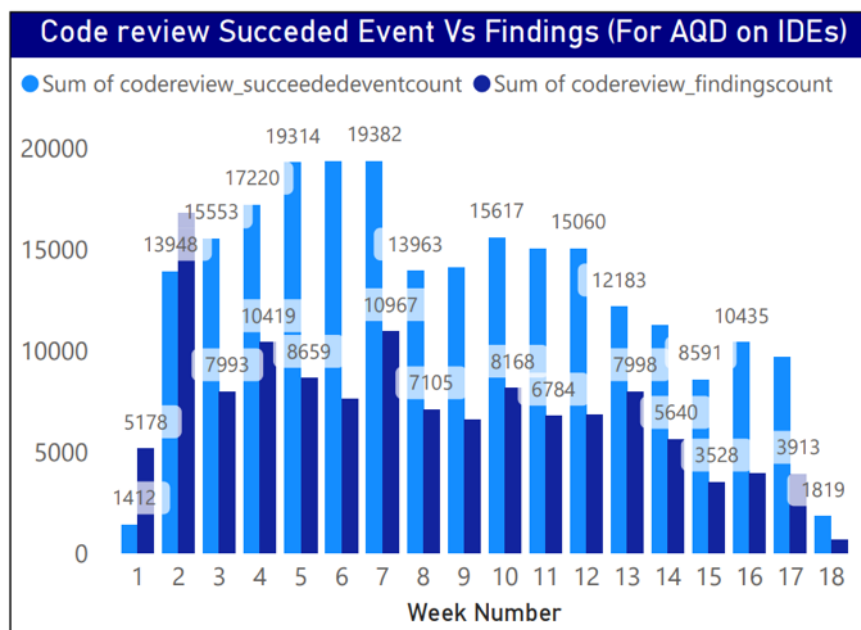


Figure 17: Number of code review events that succeeded for Amazon Q Developer

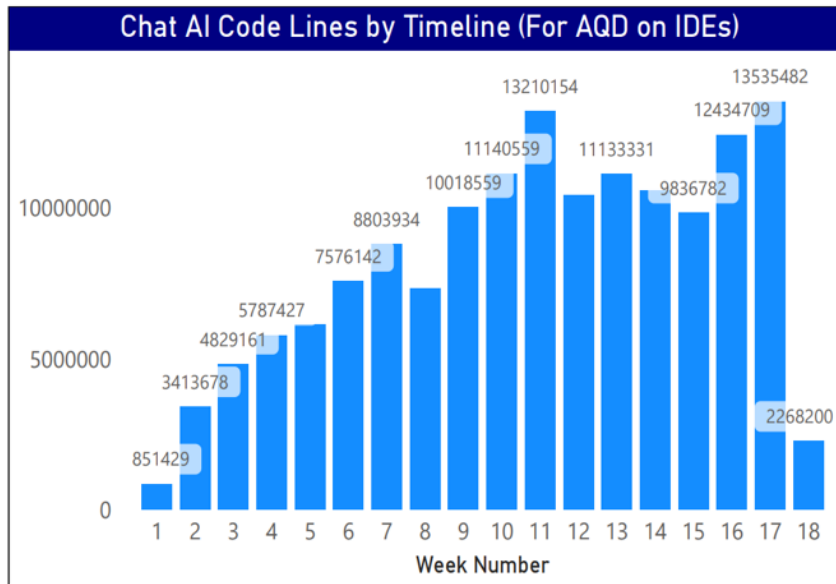


Figure 18: Chat AI Code lines over a period for Amazon Q Developer

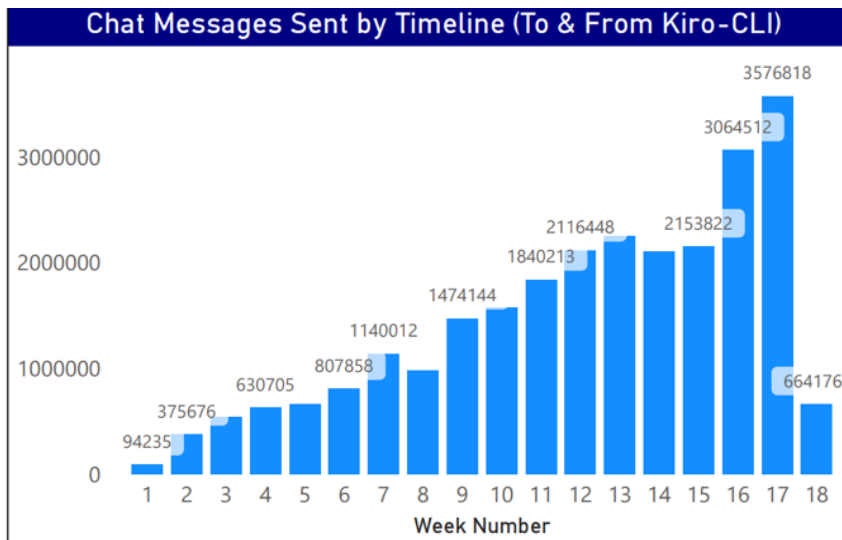


Figure 19: Chat Messages Sent over a period using Kiro CLI

Appendix F: Coding Assistants Comparisons

Satisfaction Spread comparison across tools



Figure 20: Satisfaction spread between Amazon Q and Kiro

Performance comparison across tools

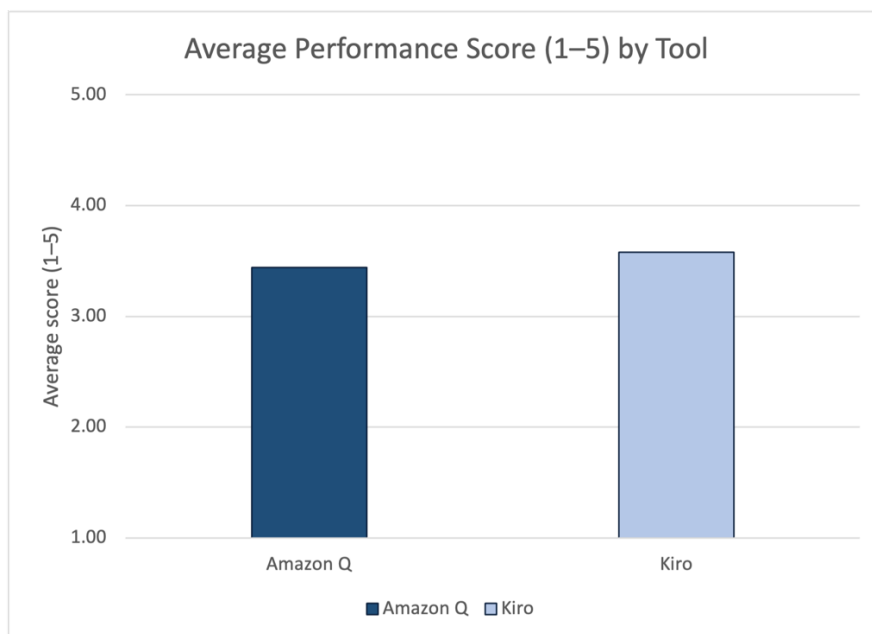


Figure 21: Performance comparison between Amazon Q and AWS Kiro

Task Activity comparison across tools

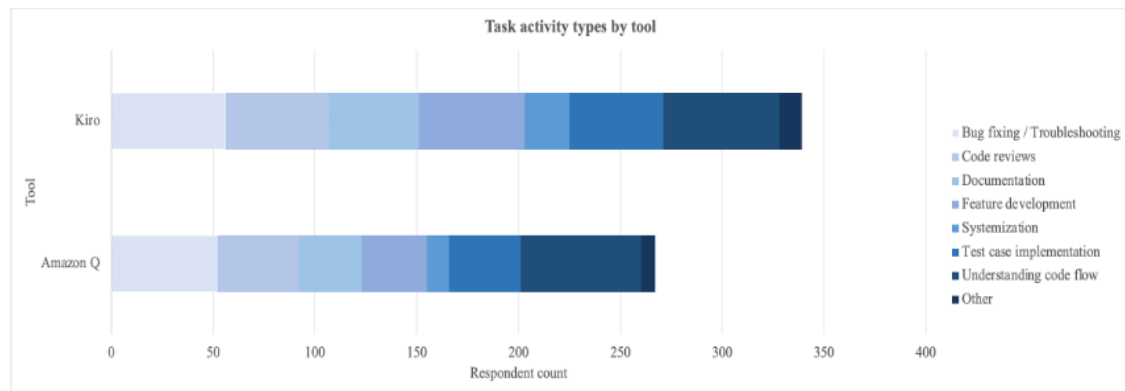


Figure 22: Task activity by tool

Tool	Bug fixing / Troubleshooting	Code reviews	Documentation	Feature development	Systemization	Test case implementation	Understanding code flow	Other	Total
Amazon Q	52	40	31	32	11	35	59	7	267
Kiro	56	51	44	52	22	46	57	11	339
Total	108	91	75	84	33	81	116	18	606

Chi-square test (Task × Tool) p-value=0.527

Table 21: Activity types by tool

Task	Rarely	A few times a week	Daily	Multiple times a day	Total
Bug fixing / Troubleshooting	2	22	22	62	108
Code reviews	0	12	23	56	91
Documentation	1	13	14	47	75
Feature development	0	10	16	58	84
Systemization	1	1	3	28	33
Test case implementation	0	12	16	53	81
Understanding code flow	1	25	25	65	116
Other	0	5	3	10	18
Total	5	100	122	379	606
Chi-square test (Task × Frequency) p-value	0.307				

Table 22: Heatmap-Task x Use frequency (respondent counts)

Communication and Collaboration comparison across tools

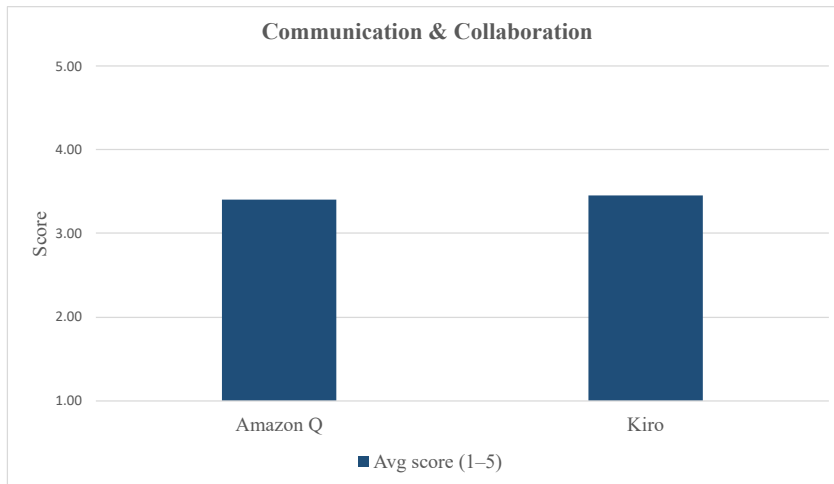


Figure 23: Communication & Collaboration by tool

Efficiency comparison across tools

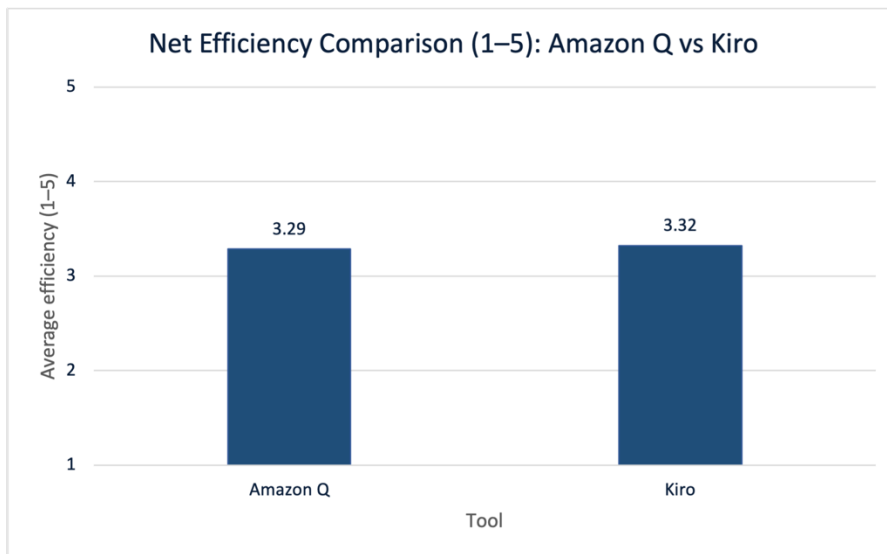


Figure 24: Efficiency comparison of Amazon Q and AWS Kiro

Appendix G: Multiple linear regression diagnostic tables

Satisfaction

✓ Diagnostics summary	
Valid N	156
R ²	0,230429
Adj. R ²	0,204776
F	8,982744
df (resid)	150
Sig. F	1,74E-07
Skewness (resid)	-1,27375
Kurtosis (excess)	4,50296
Jarque-Bera	173,9813
JB p-value	1,66E-38
Corr(resid , fitted)	-0,03184
Breusch-Pagan (approx)	64,2344

Table 23: Diagnostic summary

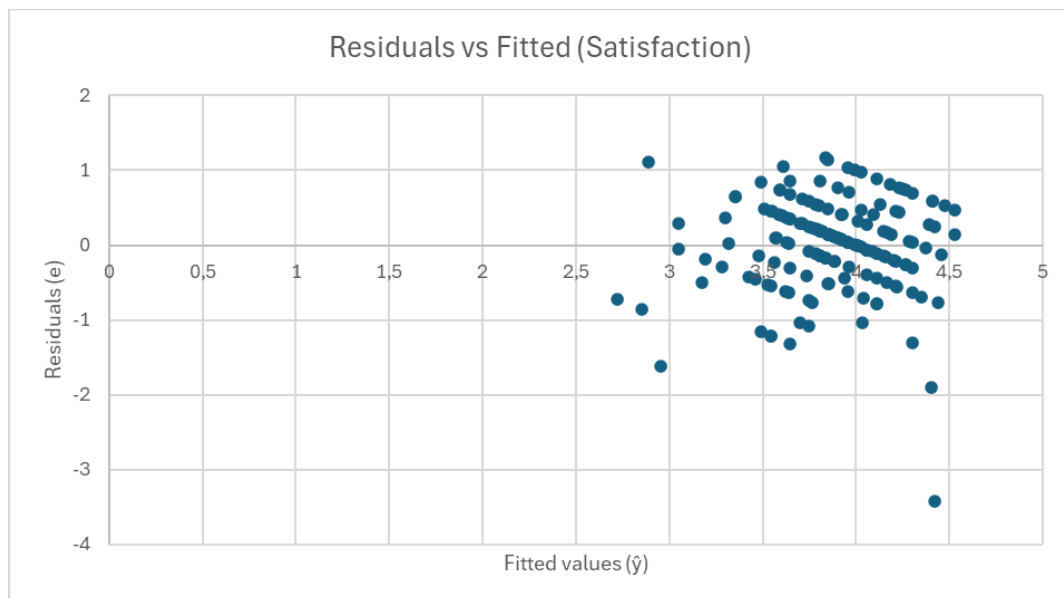


Figure 25: Residuals vs fitted

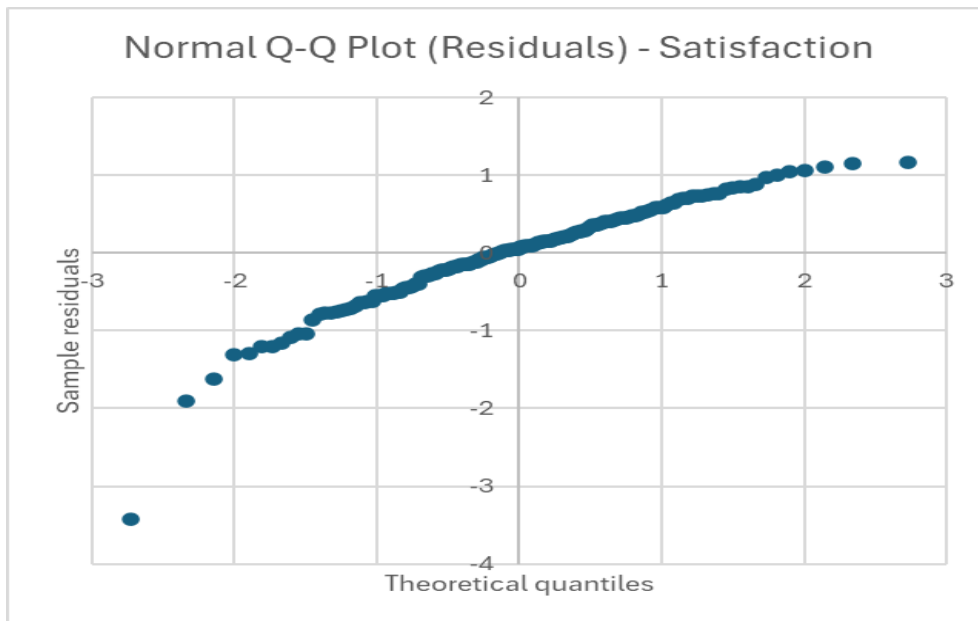


Figure 26: Q-Q plot for Satisfaction

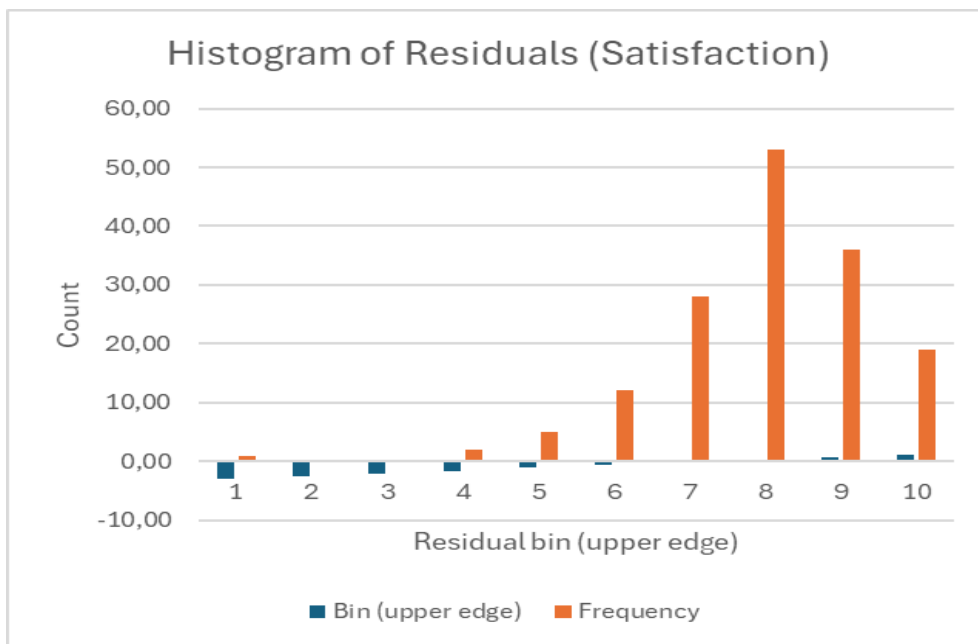


Figure 27: Histogram of residuals for Satisfaction

Performance

✓ Diagnostics summary	
Valid N	160
R ²	0,312021
Adj. R ²	0,289684
F	13,96881
df (resid)	154
Sig. F	2,95E-11
Skewness (resid)	-0,57819
Kurtosis (excess)	0,992746
Jarque–Bera	15,48504
JB p-value	0,000434
Corr(resid , fitted)	0,067761
Breusch–Pagan (approx)	92,08169

Table 24: Diagnostic summary

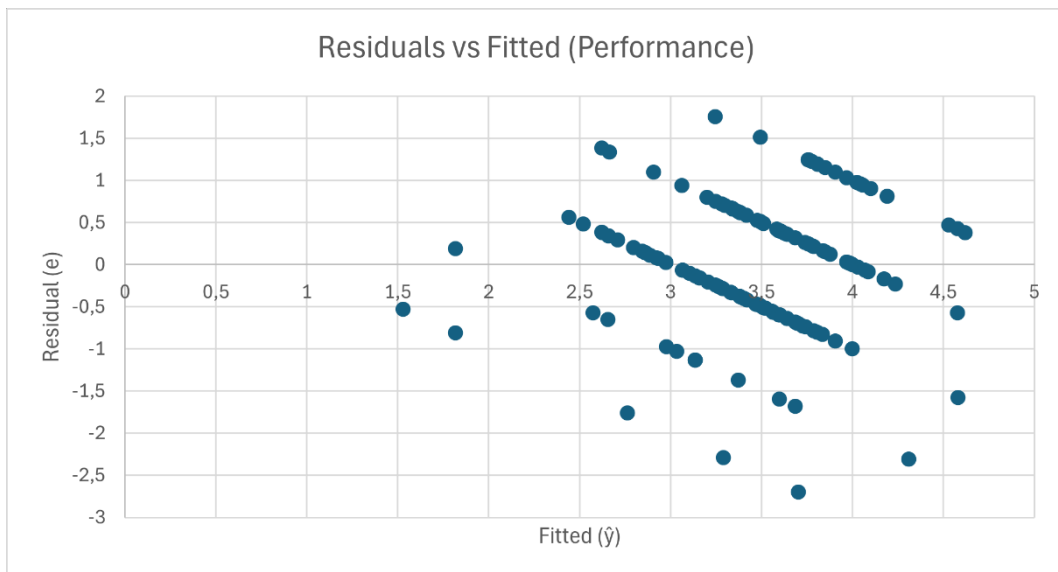


Figure 28: Residuals vs fitted for Performance

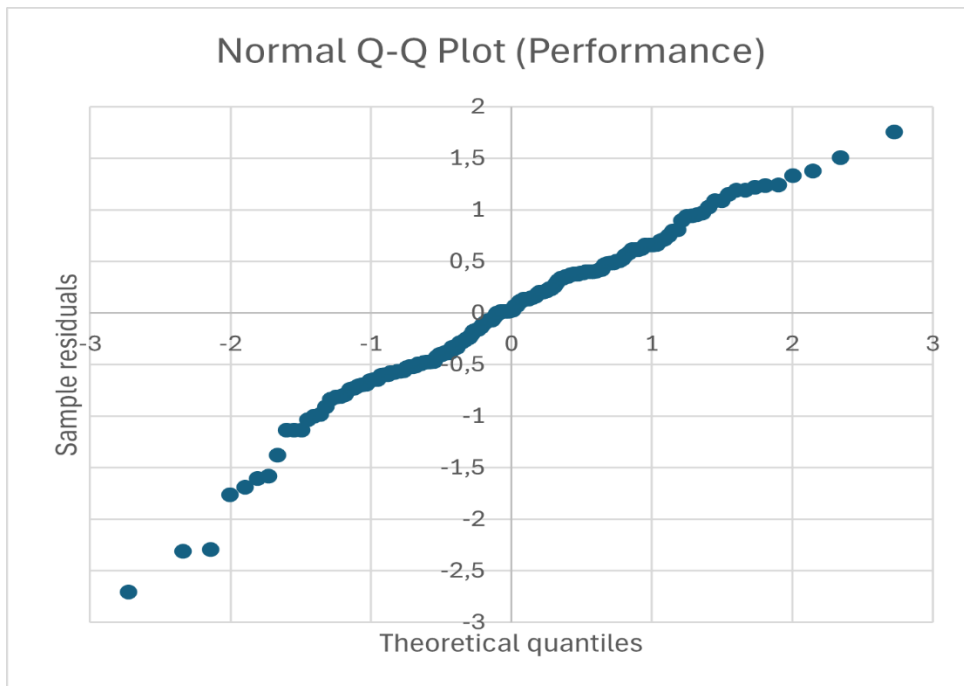


Figure 29: Q-Q plot for Performance

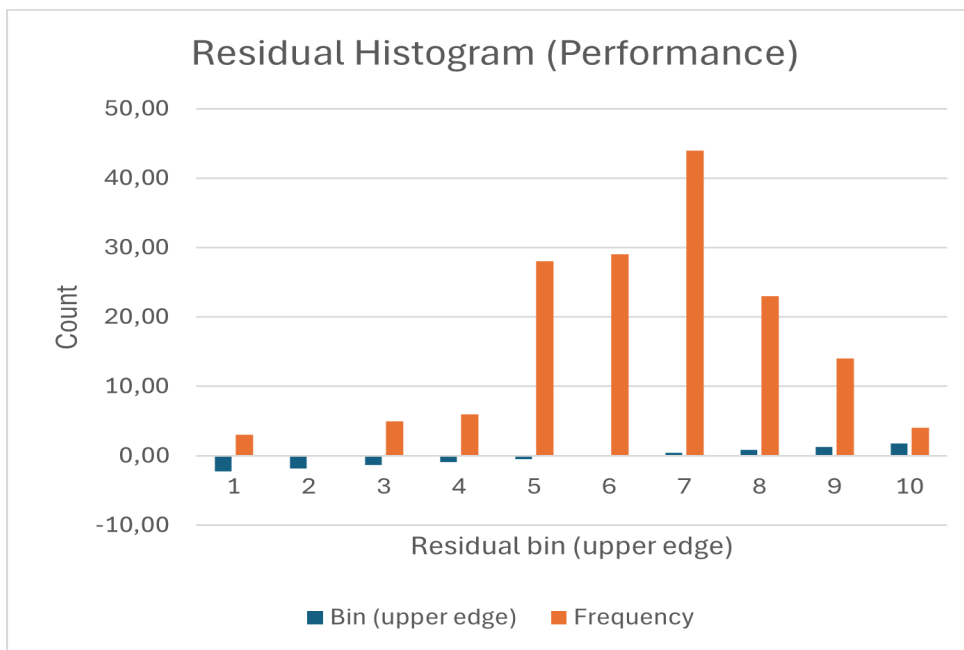


Figure 30: Residuals histogram for Performance

Activity

✓ Diagnostics summary	
Valid N	160
R ²	0,319538
Adj. R ²	0,297445
F	14,46333
df (resid)	154
Sig. F	1,31E-11
Skewness (resid)	-0,61515
Kurtosis (excess)	0,684279
Jarque–Bera	13,21253
JB p-value	0,001352
Corr(resid , fitted)	-0,08121
Breusch–Pagan (approx)	61,98056

Table 25: Diagnostic summary

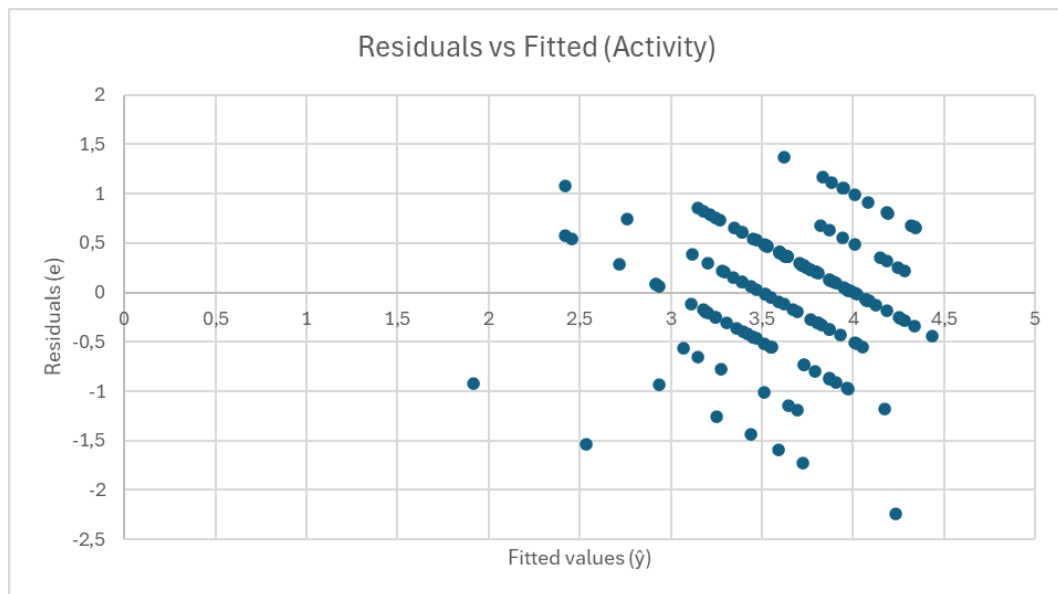


Figure 31: Residuals vs fitted for Activity

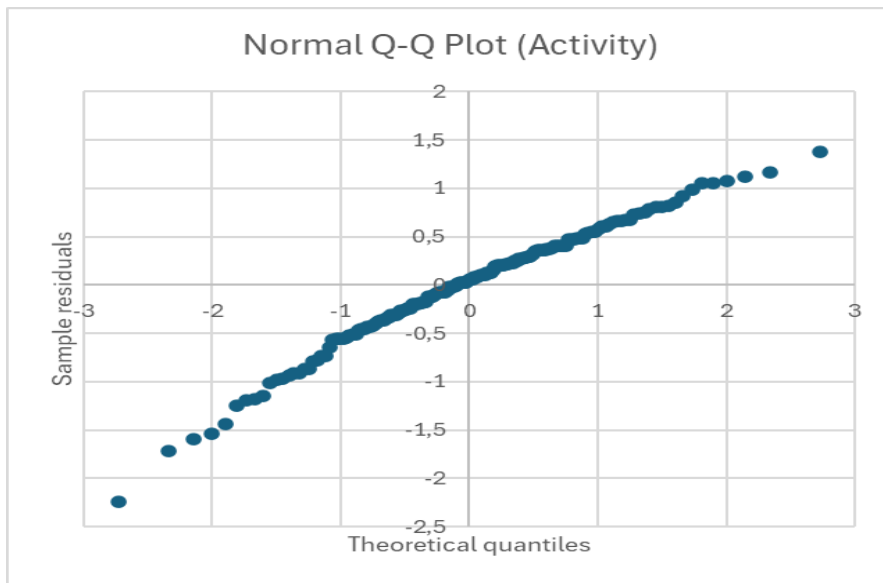


Figure 32: Q-Q Plot for Activity

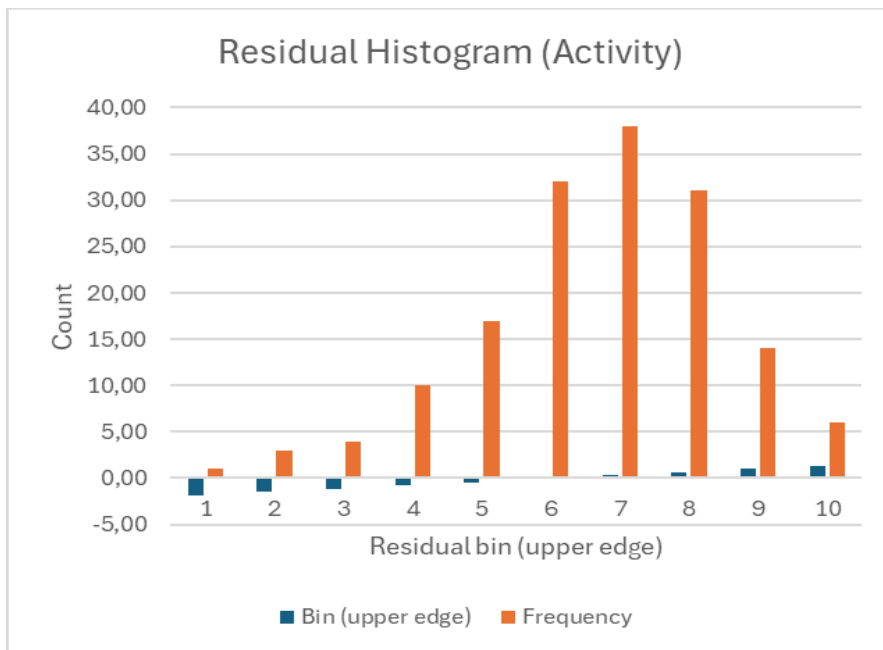


Figure 33: Residuals Histogram for Activity

Communication and collaboration

✓ Diagnostics summary	
Valid N	160
R ²	0,24949
Adj. R ²	0,225123
F	10,23878
df (resid)	154
Sig. F	1,74E-08
Skewness (resid)	-0,67165
Kurtosis (excess)	0,719858
Jarque–Bera	15,48423
JB p-value	0,000434
Corr(resid , fitted)	-0,1416
Breusch–Pagan (approx)	91,54341

Table 26: Diagnostic summary

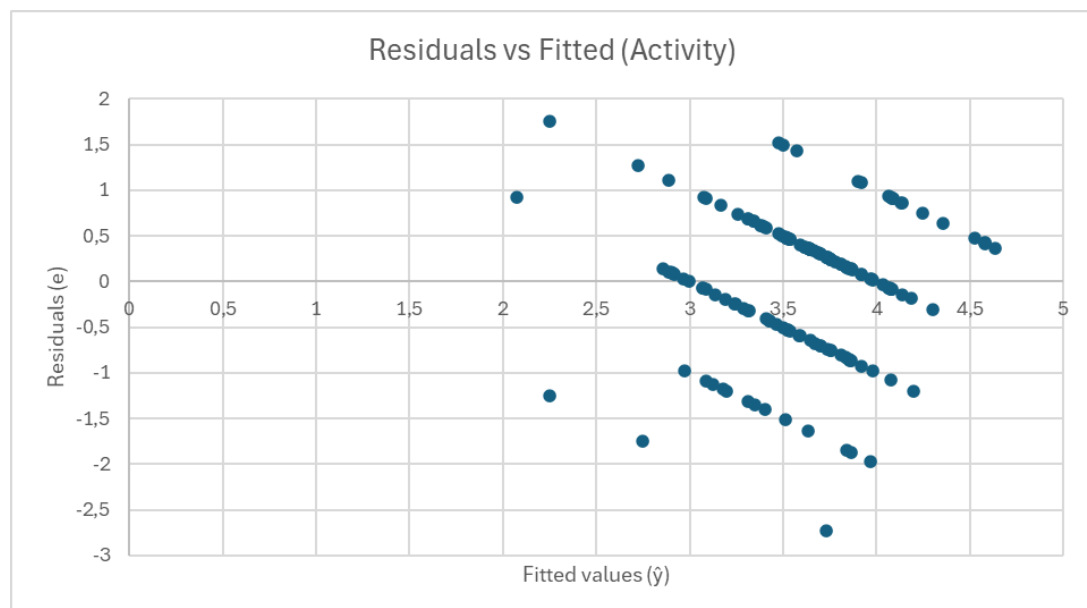


Figure 34: Residuals vs fitted for Activity

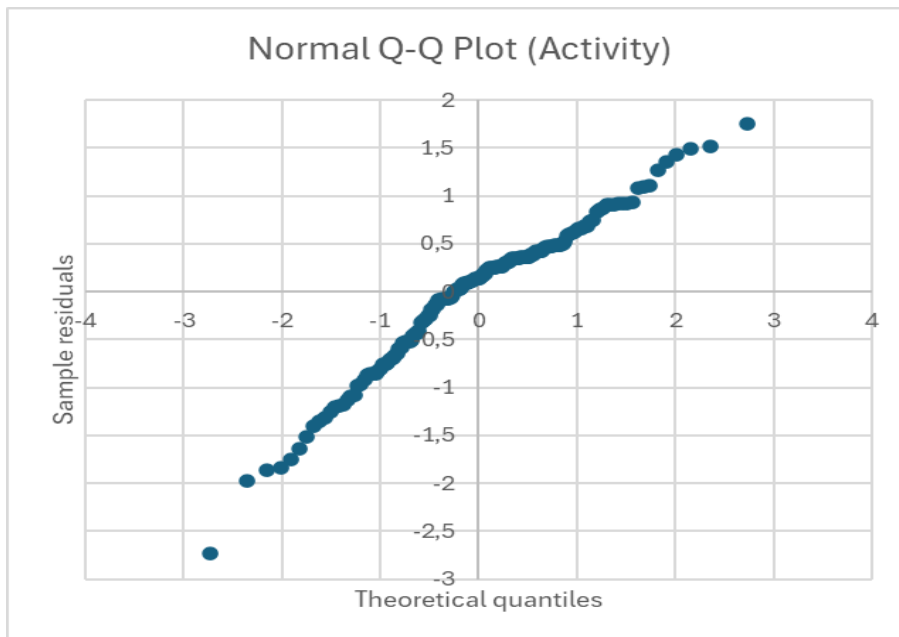


Figure 35: Q-Q Plot for Activity

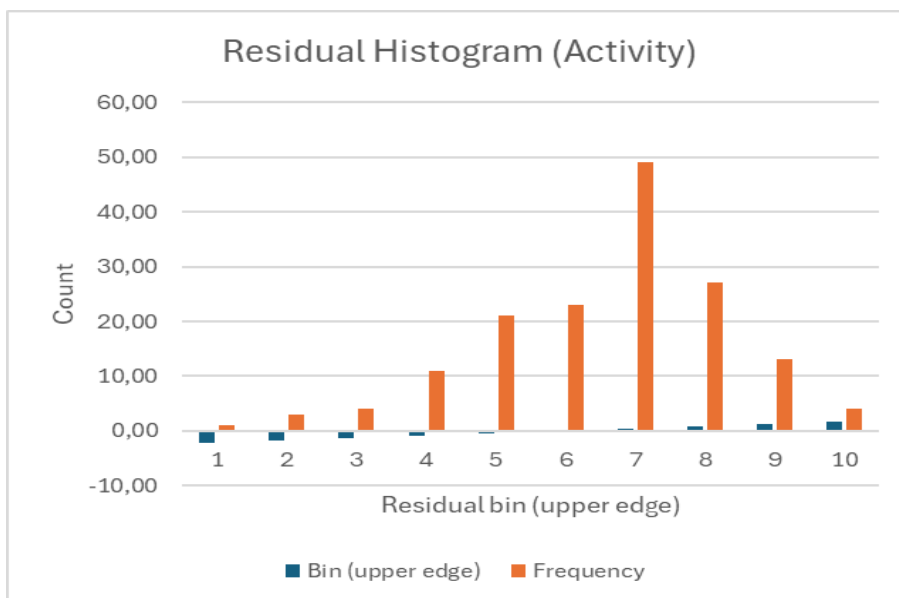


Figure 36: Residuals histogram for Activity

Efficiency and flow

✓ Diagnostics summary	
Valid N	160
R ²	0,275462
Adj. R ²	0,251938
F	11,70987
df (resid)	154
Sig. F	1,33E-09
Skewness (resid)	-0,59259
Kurtosis (excess)	2,250662
Jarque–Bera	43,1342
JB p-value	4,3E-10
Corr(resid , fitted)	0,095616
Breusch–Pagan (approx.)	41,3067

Table 27: Diagnostic summary

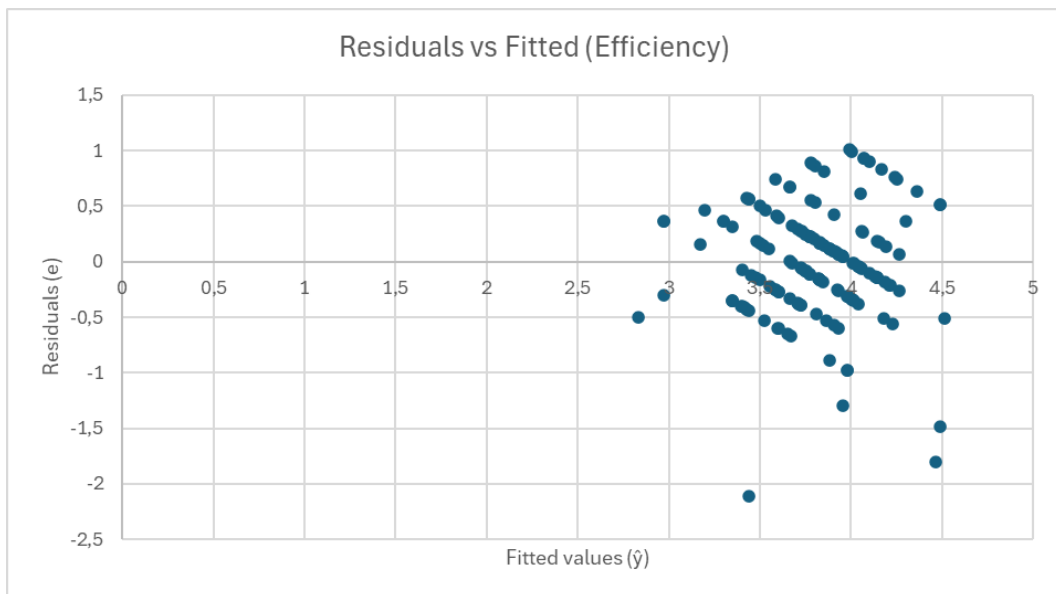


Figure 37: Residuals vs fitted for Efficiency

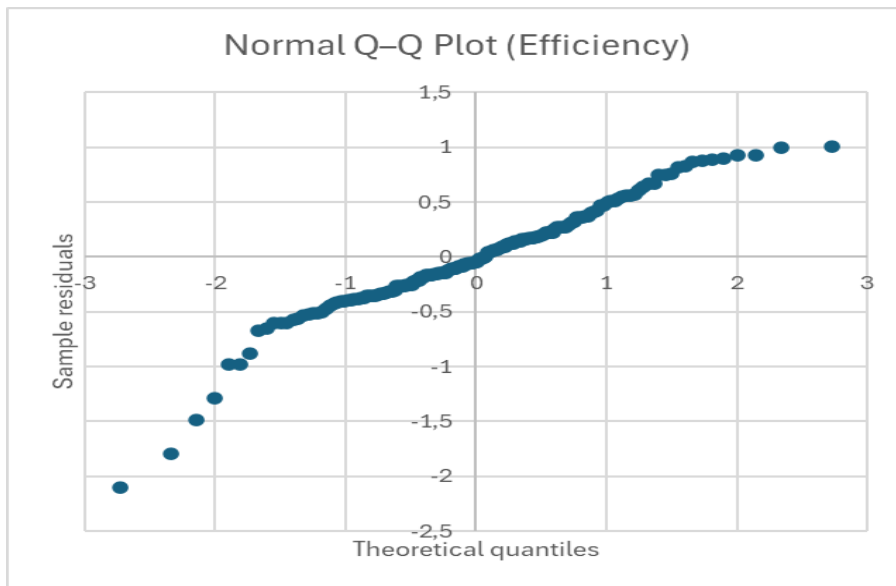


Figure 38: *Q-Q plot for Efficiency*

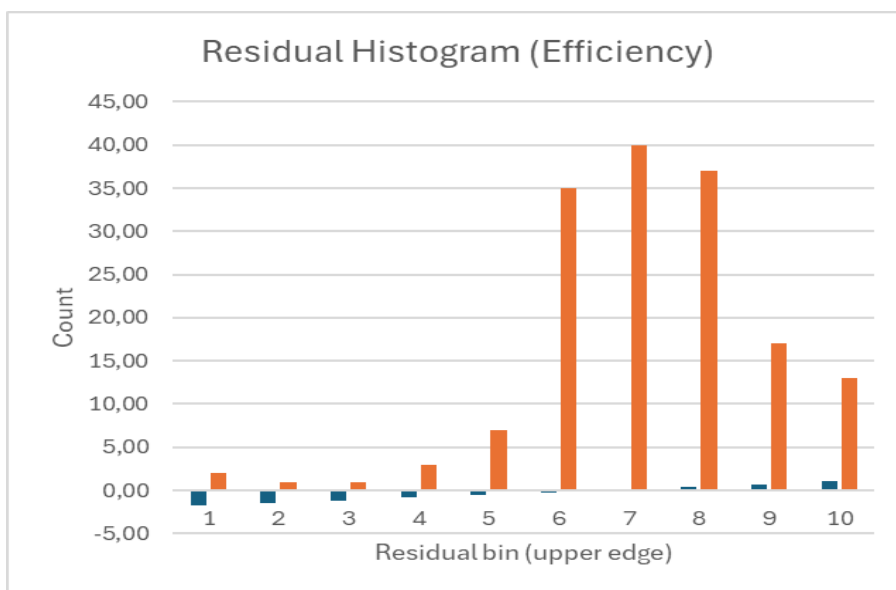


Figure 39: *Residual histogram for Efficiency*

Appendix G: Multiple linear regression robustness check tables

Satisfaction

Metric	Full sample	Without outliers
Valid N	156	155
Outliers removed		1
R ²	0,230	0,309
Adj. R ²	0,205	0,285
F	8,983	13,303
df (resid)	150	149
Sig. F	0,000	0,000
Std. error (y)	0,654	0,591

Table 28: Model Fit Comparison (Satisfaction)

Predictor	β (Full)	SE (Full)	t (Full)	p (Full)	β (No outliers)	SE (No outliers)	t (No outliers)	p (No outliers)	$\Delta\beta$
Intercept	3,868	0,054	71,891	0,000	3,884	0,049	79,846	0,000	0,016
TTF centered	0,450	0,076	5,914	0,000	0,492	0,069	7,121	0,000	0,042
Utilization centered	0,101	0,062	1,622	0,107	0,128	0,056	2,279	0,024	0,027
TTF $\times$ Utilization	0,018	0,070	0,262	0,794	0,061	0,063	0,960	0,339	0,043
Task complexity centered	-0,035	0,068	-0,511	0,610	-0,049	0,061	-0,800	0,425	-0,014
Experience centered	-0,052	0,051	-1,030	0,305	-0,029	0,046	-0,632	0,528	0,023

Table 29: Coefficient Comparison (Satisfaction)

Performance

Metric	Full sample	Without outliers
Valid N	160	153
Outliers removed		7
R ²	0,312	0,376
Adj. R ²	0,290	0,355
F	13,969	17,726
df (resid)	154	147
Sig. F	0,000	0,000
Std. error (y)	0,773	0,696

Table 30: Model Fit Comparison (Performance)

Predictor	β (Full)	SE (Full)	t (Full)	p (Full)	β (No outliers)	SE (No outliers)	t (No outliers)	p (No outliers)	$\Delta\beta$
Intercept	3,475	0,063	55,130	0,000	3,506	0,058	60,370	0,000	0,031
TTF centered	0,379	0,088	4,330	0,000	0,360	0,084	4,270	0,000	-0,019
Utilization centered	0,134	0,073	1,822	0,070	0,155	0,067	2,314	0,022	0,022
TTF $\times$ Utilization	-0,024	0,070	-0,340	0,734	-0,026	0,065	-0,401	0,689	-0,002
Task complexity centered	0,266	0,078	3,407	0,001	0,295	0,074	3,973	0,000	0,029
Experience centered	0,045	0,060	0,745	0,458	0,031	0,055	0,554	0,581	-0,014

Table 31: Coefficient Comparison (Performance)

Activity

Metric	Full sample	Without outliers
Valid N	160	159
Outliers removed		1
R ²	0,320	0,359
Adj. R ²	0,297	0,338
F	14,463	17,160
df (resid)	154	153
Sig. F	0,000	0,000
Std. error (y)	0,634	0,608

Table 32: Model Fit Comparison (Activity)

Predictor	β (Full)	SE (Full)	t (Full)	p (Full)	β (No outliers)	SE (No outliers)	t (No outliers)	p (No outliers)	$\Delta\beta$
Intercept	3,656	0,052	70,699	0,000	3,666	0,050	73,825	0,000	0,010
TTF centered	0,466	0,072	6,494	0,000	0,483	0,069	6,997	0,000	0,016
Utilization centered	0,082	0,060	1,367	0,174	0,083	0,058	1,446	0,150	0,001
TTF $\times$ Utilization	-0,057	0,057	-1,002	0,318	-0,034	0,055	-0,618	0,538	0,023
Task complexity centered	0,076	0,064	1,181	0,239	0,114	0,062	1,833	0,069	0,038
Experience centered	-0,100	0,049	-2,030	0,044	-0,083	0,047	-1,757	0,081	0,017

Table 33: Coefficient Comparison (Activity)

Communication and collaboration

Metric	Full sample	Without outliers
Valid N	160	159
Outliers removed		1
R ²	0,249	0,273
Adj. R ²	0,225	0,249
F	10,239	11,468
df (resid)	154	153
Sig. F	0,000	0,000
Std. error (y)	0,771	0,740

Table 34: Model Fit Comparison (communication and collaboration)

Predictor	β (Full)	SE (Full)	t (Full)	p (Full)	β (No outliers)	SE (No outliers)	t (No outliers)	p (No outliers)	$\Delta\beta$
Intercept	3,613	0,063	57,489	0,000	3,633	0,061	60,016	0,000	0,020
TTF centered	0,324	0,087	3,713	0,000	0,376	0,085	4,432	0,000	0,052
Utilization centered	0,121	0,073	1,662	0,098	0,117	0,070	1,675	0,096	-0,004
TTF $\times$ Utilization	0,001	0,070	0,020	0,984	-0,007	0,067	-0,098	0,922	-0,008
Task complexity centered	0,223	0,078	2,859	0,005	0,190	0,075	2,530	0,012	-0,032
Experience centered	0,054	0,060	0,896	0,372	0,034	0,058	0,594	0,553	-0,019

Table 35: Coefficient Comparison (Communication and collaboration)

Efficiency and flow

Metric	Full sample	Without outliers
Valid N	160	158
Outliers removed		2
R ²	0,275	0,342
Adj. R ²	0,252	0,320
F	11,710	15,806
df (resid)	154	152
Sig. F	0,000	0,000
Std. error (y)	0,518	0,463

Table 36: Model Fit Comparison (Efficiency and flow)

Predictor	β (Full)	SE (Full)	t (Full)	p (Full)	β (No outliers)	SE (No outliers)	t (No outliers)	p (No outliers)	$\Delta\beta$
Intercept	3,808	0,042	90,192	0,000	3,840	0,038	100,781	0,000	0,033
TTF centered	0,318	0,059	5,418	0,000	0,249	0,055	4,520	0,000	-0,069
Utilization centered	0,051	0,049	1,035	0,302	0,047	0,044	1,062	0,290	-0,004
TTF $\times$ Utilization	0,014	0,047	0,304	0,762	-0,012	0,043	-0,270	0,787	-0,026
Task complexity centered	0,125	0,052	2,397	0,018	0,214	0,049	4,389	0,000	0,089
Experience centered	-0,025	0,040	-0,624	0,533	-0,028	0,036	-0,761	0,448	-0,002

Table 37: Coefficient Comparison (Efficiency and flow)

Appendix H: Literature Review Search Summary

Three separate literature reviews were conducted to support the theoretical grounding of this study. The first review focused on generative AI coding assistants and their use in software development. The second review examined the SPACE framework as a multidimensional approach to developer productivity. The third review focused on the Task–Technology Fit (TTF) framework as an explanatory lens for technology use and performance.

Although these reviews are related, they target distinct research traditions and bodies of literature. As a result, each review employed a tailored search strategy, including different keyword combinations, databases, and inclusion and exclusion criteria. The following sections summarize the search strategies used for each literature review.

Generative AI Coding Assistants Literature Review

Inclusion/ Exclusion Criteria

Search Strategy

The approach for identifying relevant articles from the IEEE and ACM Digital Library databases, involved applying a defined set of keywords and search limitations. Figure 40 illustrates the process involved in searching relevant articles in the ACM and IEEE databases using the PRISMA flow diagram.

The keywords, limitations and the final search strings are outlined below:

Keywords:

- Generative AI
- Software Development
- AI coding assistants
- AI assistants

Limitations:

The use of AI coding assistants in software development has developed rapidly over the past two years; therefore, the literature review was limited to articles published from 2024 onward. As Scopus returned only a limited number of relevant studies, the search was narrowed to the IEEE and ACM databases.

Final search string used:

IEEE Database (using organization account)

“coding assistants” or “AI assistants”
--

Published year: 2024-2026

ACM Digital Library Database

“coding assistants” or “AI coding assistants”

Date of search: 04/02-2026 to 30/4-2026

Published year: 2024-2026

Summary of Search

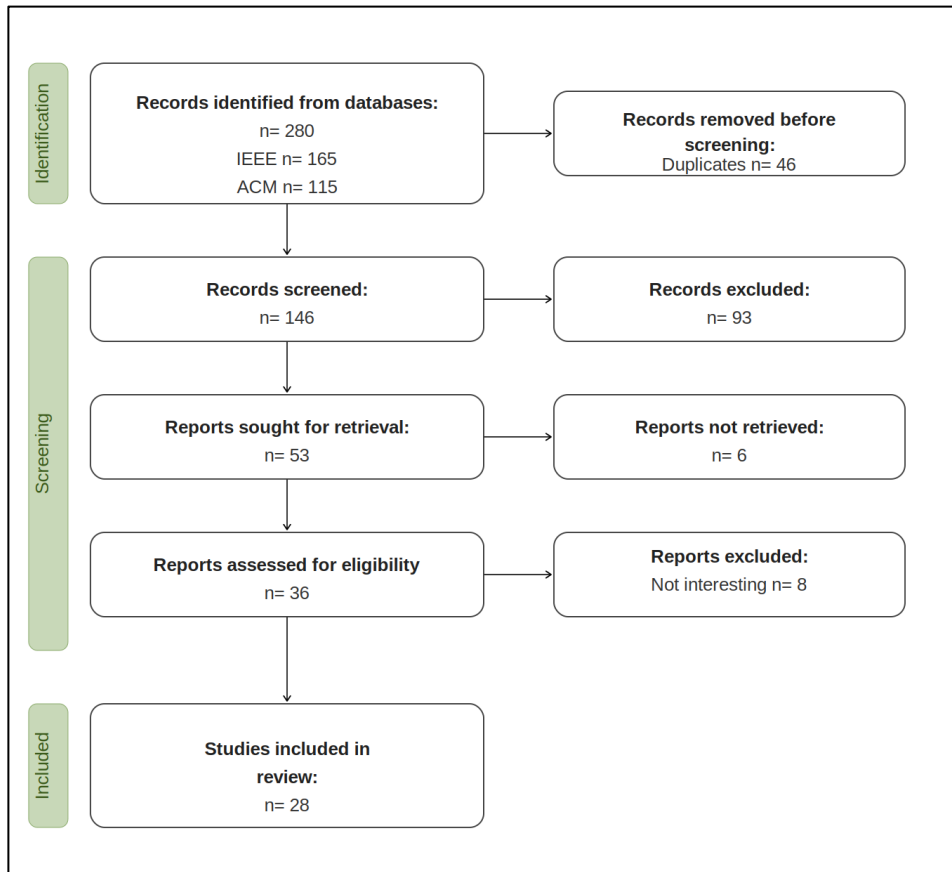


Figure 40: PRISMA diagram for coding assistants literature search

SPACE Framework Literature Review

Inclusion/ Exclusion Criteria

Search Strategy

The approach for identifying relevant articles from the IEEE and ACM Digital Library databases, involved applying a defined set of keywords and search limitations. Figure 41 illustrates the process involved in searching relevant articles in the SCOPUS and IEEE databases using the PRISMA flow diagram.

The keywords, limitations and the final search strings are outlined below:

Keywords:

- Generative AI
- Software Development
- SPACE Framework
- AI coding assistants

Limitations:

SPACE framework used for the evaluation of AI coding assistants is relatively recent. There were limited articles in Scopus and therefore the search was limited ACM and arXiv databases (peer reviewed).

Final search string used:

arXiv Database

“SPACE framework” AND “AI”

ACM Digital Library Database

“SPACE framework” AND “Productivity”

Date of search: 04/02-2026 to 30/4-2026

Published year: 2024-2026

Summary of Search

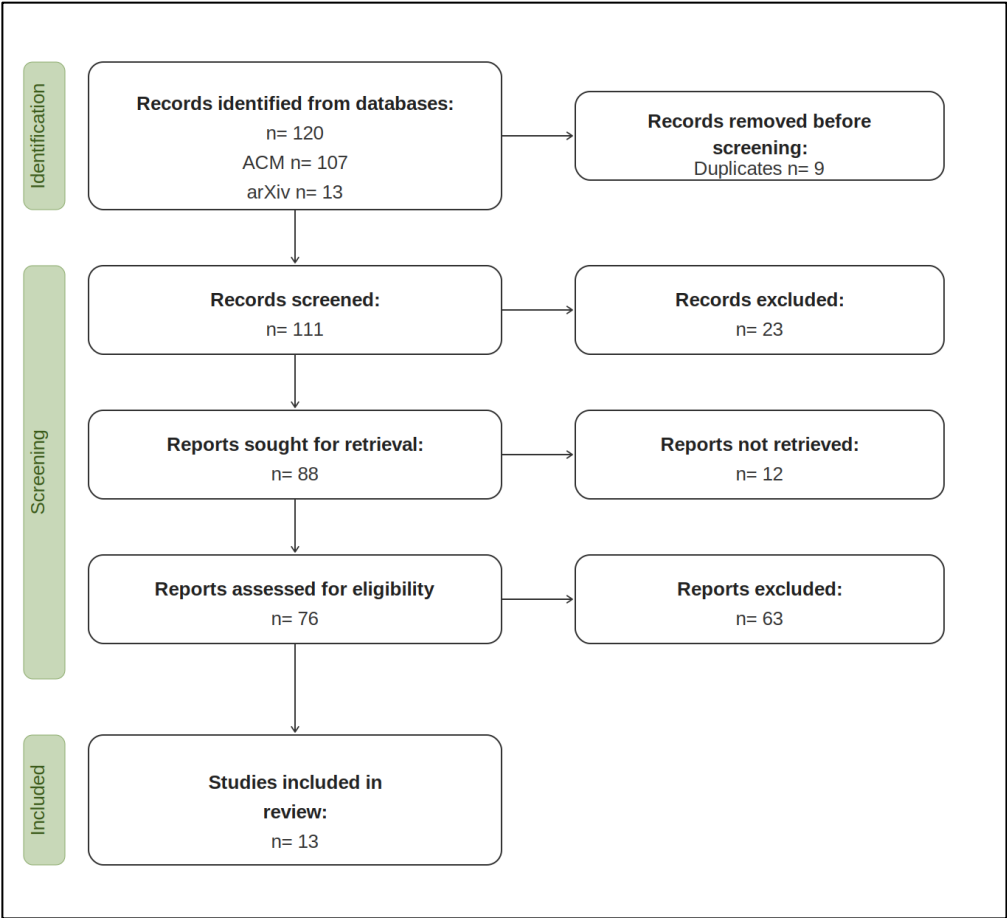


Figure 41: PRISMA flow diagram for SPACE framework literature search

Task-Technology Fit Literature Review

Search Strategy

The approach for identifying relevant articles from Scopus, One search and Google scholar databases, involved applying a defined set of keywords and search limitations. Figure 42 illustrates the PRISMA flow diagram for the search. The keywords, limitations and the final search strings are outlined below:

Keywords:

- "task-technology fit"
- "task technology fit"
- TTF
- "information systems"
- software
- technology
- "system use"
- "software development"

Limitations:

The limitations differed between the different databases, differences can be seen in the final search strings.

Final search string used:

SCOPUS Database:

(TITLE-ABS-KEY ("task-technology fit" OR "task technology fit" OR TTF) AND TITLE-ABS-KEY ("information systems" OR software OR technology OR "system use" OR "software development")) AND (LIMIT-TO (DOCTYPE , "ar") OR LIMIT-TO (DOCTYPE , "cp") OR LIMIT-TO (DOCTYPE , "ch") OR LIMIT-TO (DOCTYPE , "re")) AND (LIMIT-TO (LANGUAGE , "English")) AND (LIMIT-TO (EXACTKEYWORD , "Task Technology Fit") OR LIMIT-TO (EXACTKEYWORD , "Task-technology Fit")))

Google Scholar Database:

"task-technology fit" AND "software development" (source:"MIS Quarterly" OR source:"Information Systems Research" OR source:"European Journal of Information Systems" OR source:"Information Systems Journal" OR source:"Journal of the Association for Information Systems" OR source:"Journal of Information Technology" OR source:"Journal of Management Information Systems" OR source:"Journal of Strategic Information Systems")
--

One search database:

"task-technology fit" OR "task technology fit" OR TTF)
--

Date of search: 04/02-2026 to 30/4-2026

Published year: 1995-2026

Summary of Search

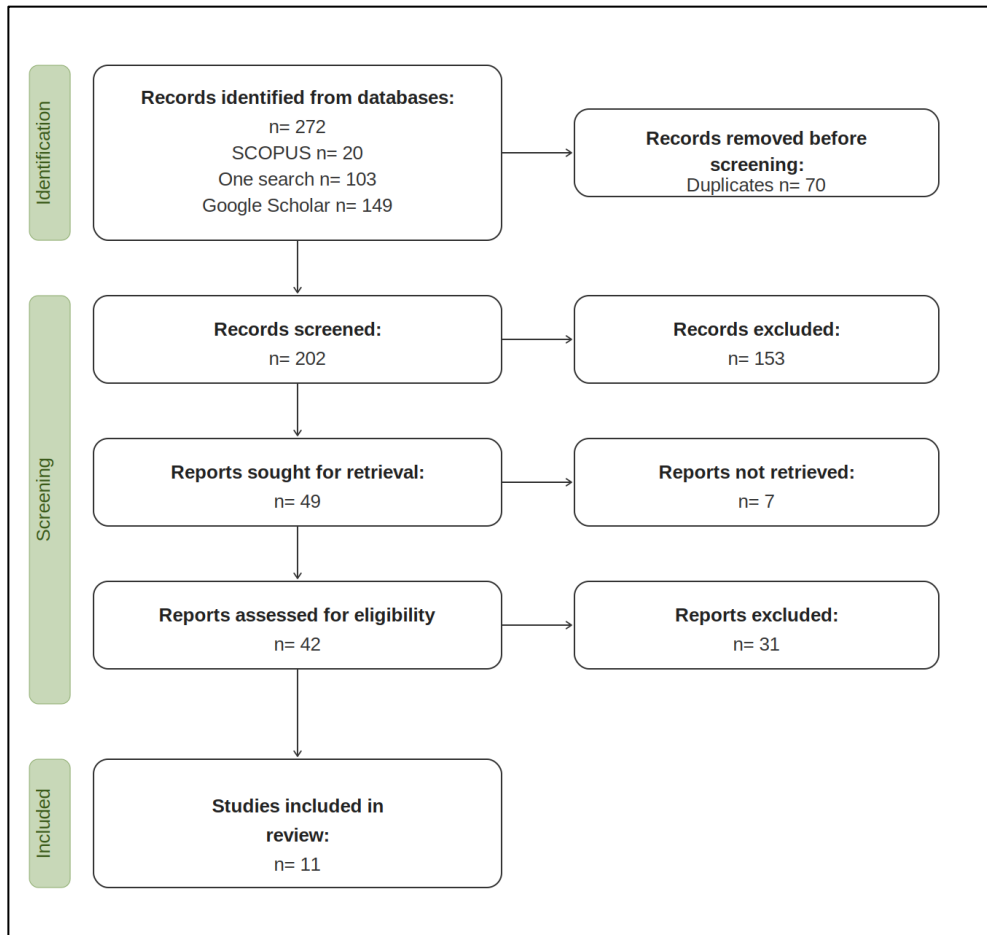


Figure 42: PRISMA flow diagram for TTF literature search